

Table of Lab Programs

Sl. No.	Programs/Exercise/Topic.
2.	AWS Identity Access Management (IAM) User and Group creation. Enable AWS IAM MFA. Create an AWS Account Alias (for Alternate Sign-in URL)
3.	Amazon S3 – Introduction, Bucket Creation and upload objects (files).
4.	Amazon S3 – Static Website Hosting (Multi-Page website), Versioning, Cross-Region Replication rule.(any 1 in 3)
5.	Overview of EC2. To Launch a Windows EC2 Instance and Connect via RDP Client.
6.	Launch a Linux EC2 Instance and Connect using SSH through PowerShell and PuTTY on Windows machine. Launch a Linux EC2 Instance and Connect using SSH through Linux terminal on Linux machine. Launch a Linux EC2 Instance and Connect through EC2 connect option on the console.(any 1 in 3)
7.	Hosting a static website on EC2 instance: - Manual Installation of Apache (httpd) Web Server on EC2 for Static Website Hosting. - Launching an EC2 Instance with User Data Script to Automatically Install Apache and Host a Static Website.
8.	Custom AMI - Create a Custom AMI from a Working EC2 Instance. - Launch an EC2 Instance using a Custom AMI. - Delete the custom AMI [Deregister and delete snapshot]
9.	Mini-Project: Hosting a Multi-Page Website using EC2 and S3 Submission: 11-11-2025 Marks: 5
10.	Creation and Configuration of a Custom AWS Virtual Private Cloud (VPC) [Including Public and Private Subnets, Internet Gateway, NAT Gateway, Route Tables]
11.	Launching and Configuring EC2 Instances for Web Server in Public Subnet and Database Server in Private Subnet Using NAT Gateway for Outbound Access
13.	Stress Testing a Linux EC2 Instance (CPU Load Test)
18.	Creating and Operating a NoSQL Key-Value Database using Amazon DynamoDB
19.	ElastiCache (Redis) as an In-Memory Cache
22.	CloudFormation – Launch EC2 (Amazon Linux 2023) + Install Apache using UserData
23.	Static Content Pulled from S3 using CloudFormation [EC2 + S3]
24.	AWS Lambda : Input Processing, Business Logic Execution, and CloudWatch Logging.
25.	Event-Driven Notification System using AWS Lambda and Amazon SNS
26.	Analyze data in a CSV File stored in S3 Using Amazon Athena (No Server)

Date: 9-10-2025

Exercise-2: AWS Identity Access Management (IAM) User and Group creation. Enable AWS IAM MFA. Create an AWS Account Alias (for Alternate Sign-in URL)

AWS Identity and Access Management (IAM) is a security service that helps you control who can access your AWS resources and what actions they can perform. It is a global AWS service. It allows you to securely manage users, groups, roles, and permissions in your AWS account.

Concept	Description
Root User	The account owner who created the AWS account. Has full access and should be used only for account setup.
Group	A collection of IAM users that share the same permissions. For example, a “Developers” group or “Students” group.
User	A person or application that interacts with AWS (e.g., student1, admin, developer). Each user has its own username, password, and access keys.
Policy	A JSON document that defines what actions are allowed or denied (e.g., “allow S3 read access”).
Role	A set of permissions that can be temporarily assumed by a user, service, or application — often used by EC2 instances or Lambda functions.

STEPS for AWS IAM User Group Creation

Step 1: Sign in to AWS Console

- Log in to your AWS Management Console using an administrator account.
- From the Services menu, search for IAM and open it.

Step 2: Open Groups Section

- In the IAM dashboard, look at the left sidebar.
- Click on “User groups” → then click “Create group”.

Step 3: Name the Group

- Enter a Group name (example: Developers, Admins, Students, etc.).
- Group names must be unique within your account.

Step 4: Attach Permissions Policies

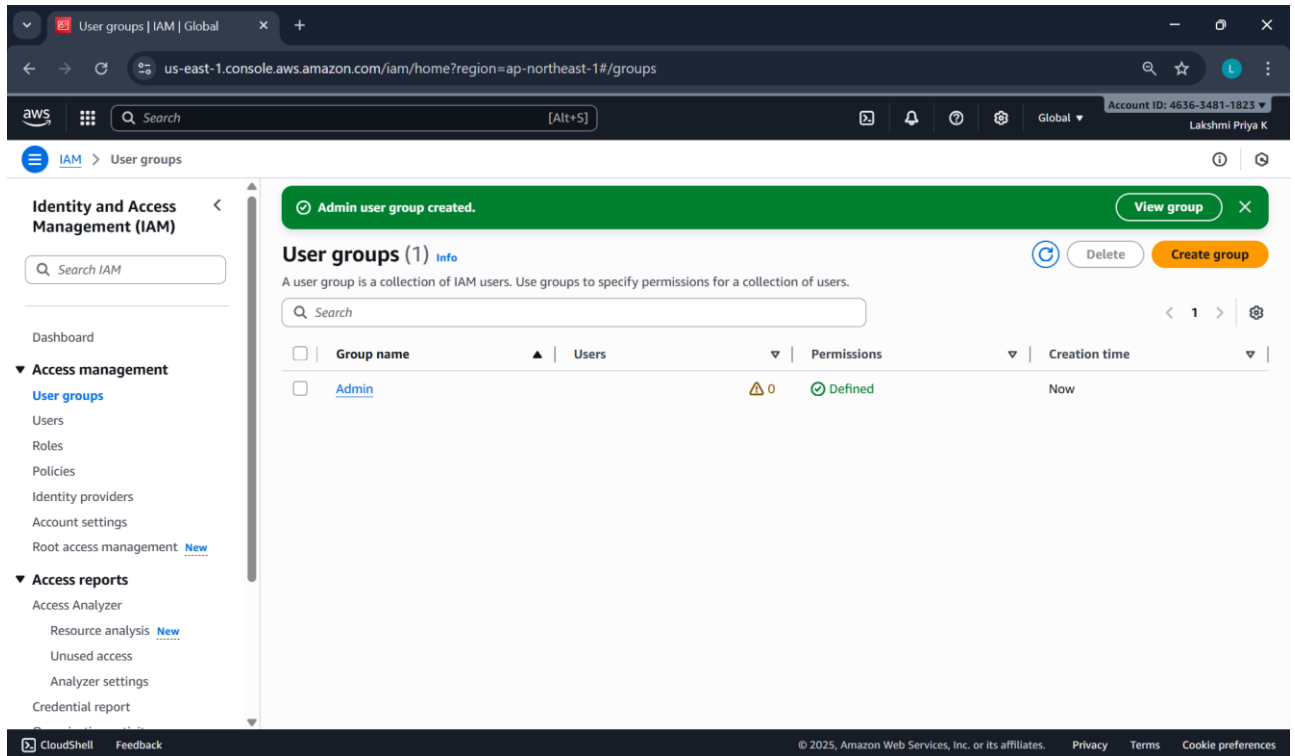
- You can attach IAM policies to define what members of the group can do. Select the following:
 - AdministratorAccess → Full access to all services.
 - IAMUserChangePassword

Step 5: Review and Create

- Review all details (group name + permissions).
- Click “Create group” to finalize.

Group Successfully Created

- The new group now appears in the IAM dashboard.
- Any user added to this group automatically inherits all permissions attached to the group.



STEPS for AWS IAM User Creation

Step 1: Sign in to AWS Management Console

- Login to the AWS Management Console using your root user credentials.
- In the search bar, type IAM and open IAM service.

Step 2: Navigate to Users Section

- In the IAM dashboard's left panel (info panel), click on Users.
- Then click on the "Add users" button.

Step 3: Set User Details

- Enter a user name.
- Checkbox – 'Provide user access to the AWS Management Console'.
- Select 'I want to create an IAM user' option

Step 4: Set Permissions

You can grant permissions to the new user in three ways:

1. Add user to group – Assign predefined permission groups.
2. Copy permissions from an existing user.
3. Attach existing policies directly.

Recommended: Use IAM **groups** to manage permissions easily.

Step 5: Set User Details and Tags (Optional)

- Add tags like Department: MCA or Role: Faculty/Student for easy identification.
- Click Next to review.

Step 6: Review and Create User

- Review all settings (user name, permissions, tags).
- Click Create user.

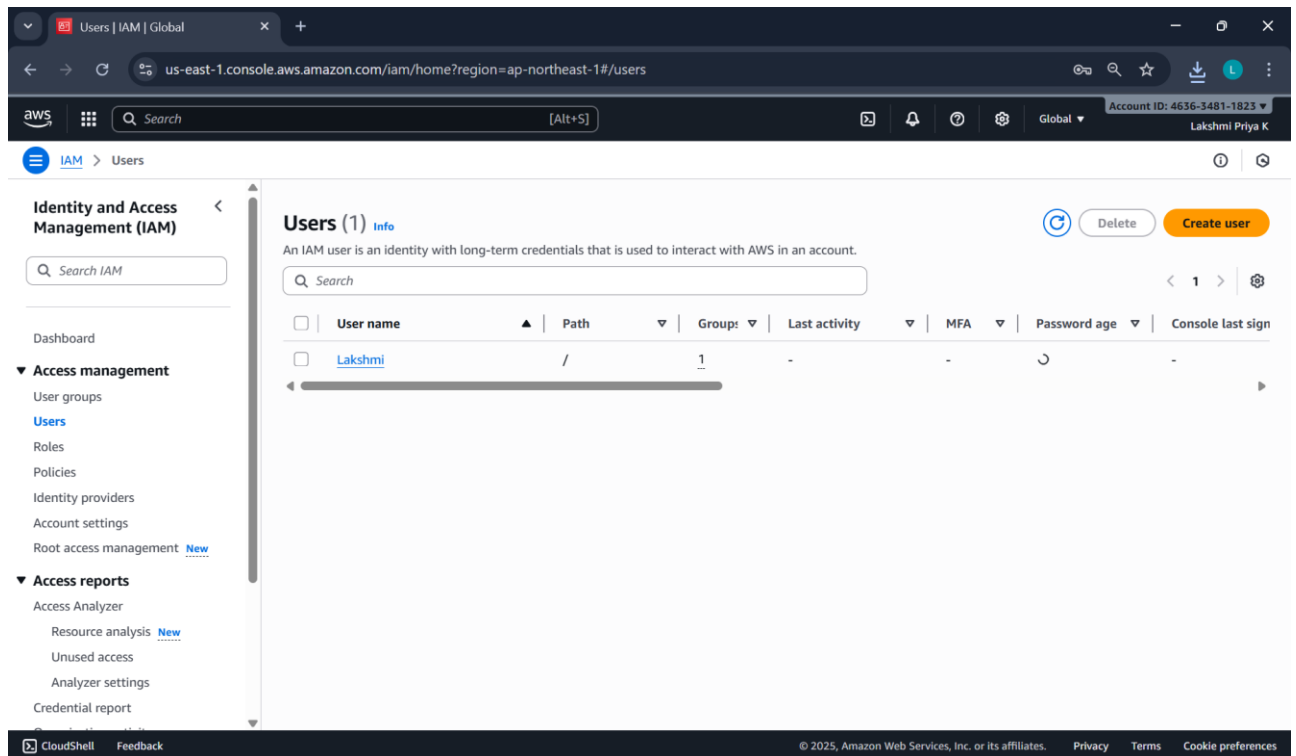
Step 7: Save Credentials

- After creation, AWS displays:
 - Console sign-in URL
 - Username
 - Password

NOTE: Download or copy these credentials immediately — they cannot be retrieved later.

Step 8: Test the User Login

- Visit the IAM login URL provided (unique for your AWS account).
- Log in with the newly created username and password.
- Verify access and permissions.



Enable AWS IAM MFA

Step 1: Sign in to AWS Console as root user

Step 2: Open IAM Dashboard

- In the search bar, type IAM and select IAM (Identity and Access Management).
- From the left navigation pane, select Users.

Step 3: Select a User

- Click the user name for whom you want to enable MFA.
- This opens the User Summary page.

Step 4: Go to Security Credentials Tab

- Click the Security credentials tab.
- Scroll down to the section “Multi-factor authentication (MFA)”.

Step 5: Assign MFA Device

- Click “Assign MFA device”.
- Choose the MFA type:
 1. Virtual MFA device (e.g., Google Authenticator, Authy — most common)
 2. Security key (hardware-based, e.g., YubiKey)
 3. Authenticator app on phone

Step 6: Configure Virtual MFA

- If you select Virtual MFA device:
 1. Open your Google Authenticator or Authy app on your phone.
 2. Scan the QR code shown on the AWS screen.
 3. The app starts generating 6-digit codes.

Step 7: Verify MFA

- Enter two consecutive codes from your app in the verification fields.
- Click “Assign MFA”.

Step 8: Confirm Setup

- You will see a green checkmark confirming MFA is successfully assigned.
- The user now requires MFA each time they sign in.

The screenshot displays the AWS IAM console interface for a user named 'Lakshmi'. At the top, a green notification banner indicates that a 'Passkey MFA device' has been successfully assigned. The main content area is divided into several sections. The 'Summary' section provides key details about the user, including their ARN, console access status (Enabled with MFA), and the date they were created. Below this, the 'Security credentials' tab is selected, showing the 'Console sign-in' section. This section includes a console sign-in link, the console password (which was updated 5 minutes ago), and the last console sign-in status (Never). The left sidebar shows the navigation menu with options like 'Dashboard', 'Access management', and 'Access reports'.

Create an AWS Account Alias (for Alternate Sign-in URL)

As an IAM user you can sign in using the default URL or create an account alias for it.

An Account Alias gives your AWS account a name instead of using the long numeric Account ID in your sign-in URL.

This makes it easier for IAM users to remember and log in.

Step 1:

- Sign in to the AWS Management Console using root user or an IAM user with administrative privileges.
- In the search bar, type IAM, and open the IAM service.

Step 2:

- In the left navigation pane, scroll down and select Dashboard.

Step 3:

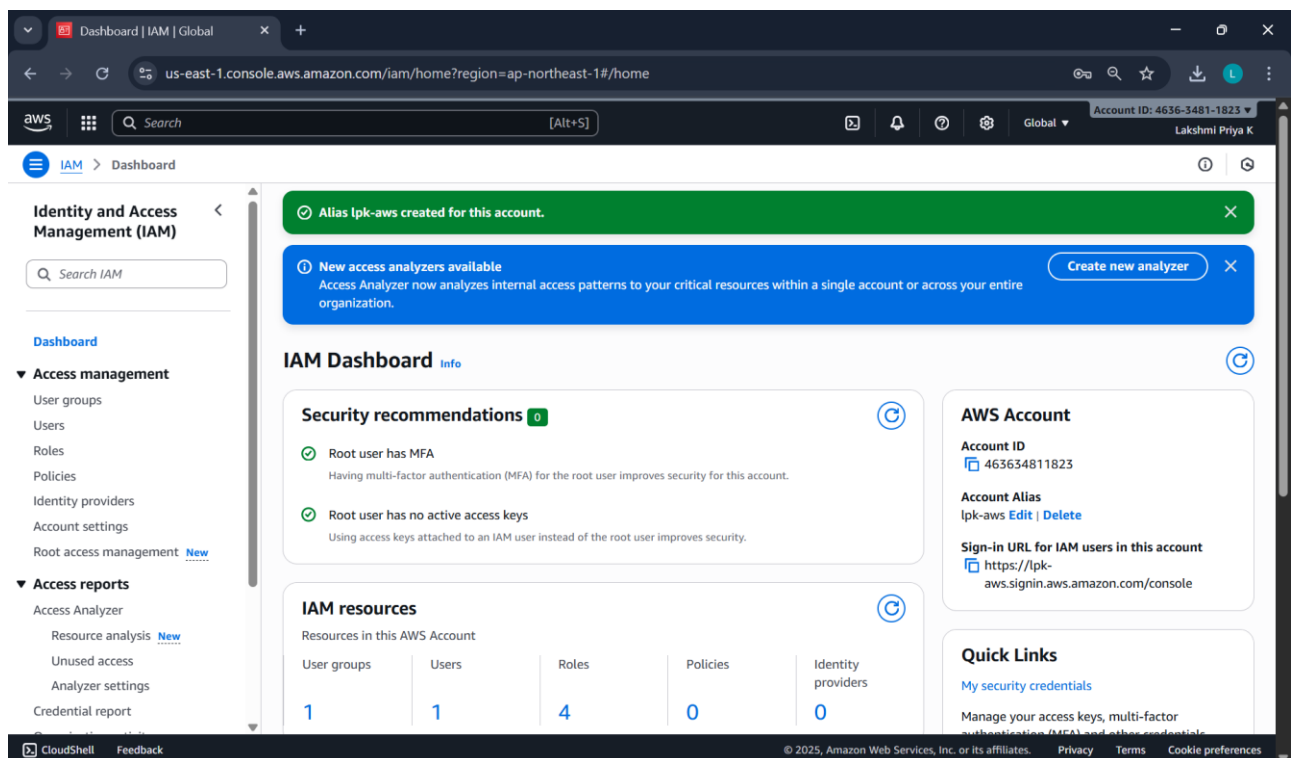
- Under the “AWS Account” section, find “Account Alias”.
- Click on “Create” (or “Edit” if one already exists).

Step 4:

- In the pop-up box, enter your preferred alias name.
- Click “Create alias”.

Step 5:

- Once created, you’ll see a new Sign-in URL displayed.



Date: 14-10-2025

Exercise-3: Amazon S3 – Introduction, Bucket Creation and upload objects (files).

Amazon S3 (Simple Storage Service) is a fully managed, object-based storage service offered by AWS. It allows you to store and retrieve unlimited amounts of data from anywhere on the web. Unlike traditional file systems that store data in folders, S3 stores data as objects inside buckets. Each object has:

- Data (the actual file),
- Metadata (information about the file),
- Unique key (its name within the bucket).

You can store unlimited data in Amazon S3.

However, each AWS account can create up to 100 buckets by default (can be increased by request).

Each object (file) can be up to 5 TB (terabytes) in size.

Single upload limit: 5 GB (may vary), but larger files can be uploaded using Multipart Upload.

Amazon S3 is designed for:

- Durability: 99.999999999% (11 nines) – this means your data is extremely safe.
- Availability: 99.99% uptime – your data is almost always accessible.
- S3 automatically stores multiple copies of your data across multiple Availability Zones in a region.

Storage classes: S3 offers different storage classes depending on cost and frequency of access.

Storage Class	Description
S3 Standard	For frequently accessed data (default).
S3 Intelligent-Tiering	Automatically moves data between frequent - infrequent access tiers.
S3 Standard-IA	For infrequently accessed data but still quickly available.
S3 Glacier	For archival storage (low cost, slower retrieval).
S3 Glacier Deep Archive	For long-term archival (lowest cost).

Each object can be accessed through a **unique URL**.

S3 is commonly used for:

- Backup and archival
- Static website hosting
- Application data storage
- Media content delivery

Feature	Description
Buckets	Top-level containers for storing objects.
Objects	Actual files stored in S3 (e.g., images, HTML, PDFs).
Region	Each bucket is created in a specific AWS region.
Versioning	Maintains multiple versions of the same object for recovery.
Static Website Hosting	Allows you to host HTML pages directly from an S3 bucket.

Steps to Create an S3 Bucket and Upload an Image

Step 1: Open the S3 Service

- Sign in to your AWS Management Console.

- In the search bar, type S3 and click Amazon S3.

Step 2: Create a New Bucket

- Click “Create bucket”.
- Bucket type: General purpose
- Bucket name: Enter a name (Bucket names must be globally unique and lowercase.)
- AWS Region: Choose the region nearest to you.
- Scroll down and uncheck:
 - “Block all public access” → Uncheck this (for viewing file publicly).
 - Confirm the warning checkbox.
- Keep all other settings as default.
- Click Create bucket.

Step 3: Upload an Image File

1. Click on the newly created bucket name.
2. Click Upload → Add files.
3. Choose any image file from your computer.
4. Scroll down and click Upload.

Step 4: Make the Object Public

By default, S3 objects are private. To view them in a browser, make the file public.

1. After upload, go to the Objects tab inside your bucket.
2. Select the uploaded file.
3. Click Actions → Make public using ACL (or Object actions → Make public, depending on console version).
4. Confirm.

Step 5: Copy the Object URL

1. Open the object again by clicking its name.
2. Scroll down to the **Object URL** section.
3. Copy this URL and open it in a new browser tab.

The image is displayed directly from the S3 bucket — meaning AWS S3 is serving that object over HTTP.

Make objects public - S3 bucket

ap-northeast-1.console.aws.amazon.com/s3/buckets/amzn-s3-lpkawdemo1/object/edit_public_read_access?region=ap-northeast-1&bucketType=general&showversions=false

aws

Search

[Alt+S]

CloudShell

Feedback

Account ID: 4636-3481-1823

Lakshmi Priya K

Asia Pacific (Tokyo)

☰

☰

☰

☑ Successfully edited public access

View details below.

✕

Make public: status

Close

ⓘ After you navigate away from this page, the following information is no longer available.

Summary

Source

s3://amzn-s3-lpkawdemo1

Successfully edited public access

☑ 1 object, 62.1 KB

Failed to edit public access

0 objects

Failed to edit public access

Configuration

⊖ Failed to edit public access (0)

🔍 Find objects by name

Name

▲

Folder

▼

Type

▼

Last modified

▼

Size

▼

Error

▼

No objects failed to edit

CloudShell

Feedback

© 2025, Amazon Web Services, Inc. or its affiliates.

Privacy

Terms

Cookie preferences

Date: 16-10-2025, 23-10-2025

Exercise-4: Amazon S3 – Static Website Hosting (Multi-Page website), Versioning, Cross-Region Replication rule.

Amazon S3 Static Website Hosting

Amazon S3 can host a static website – [a website consisting of only HTML, CSS, JavaScript, images, etc. – no server-side scripting like PHP or Python].

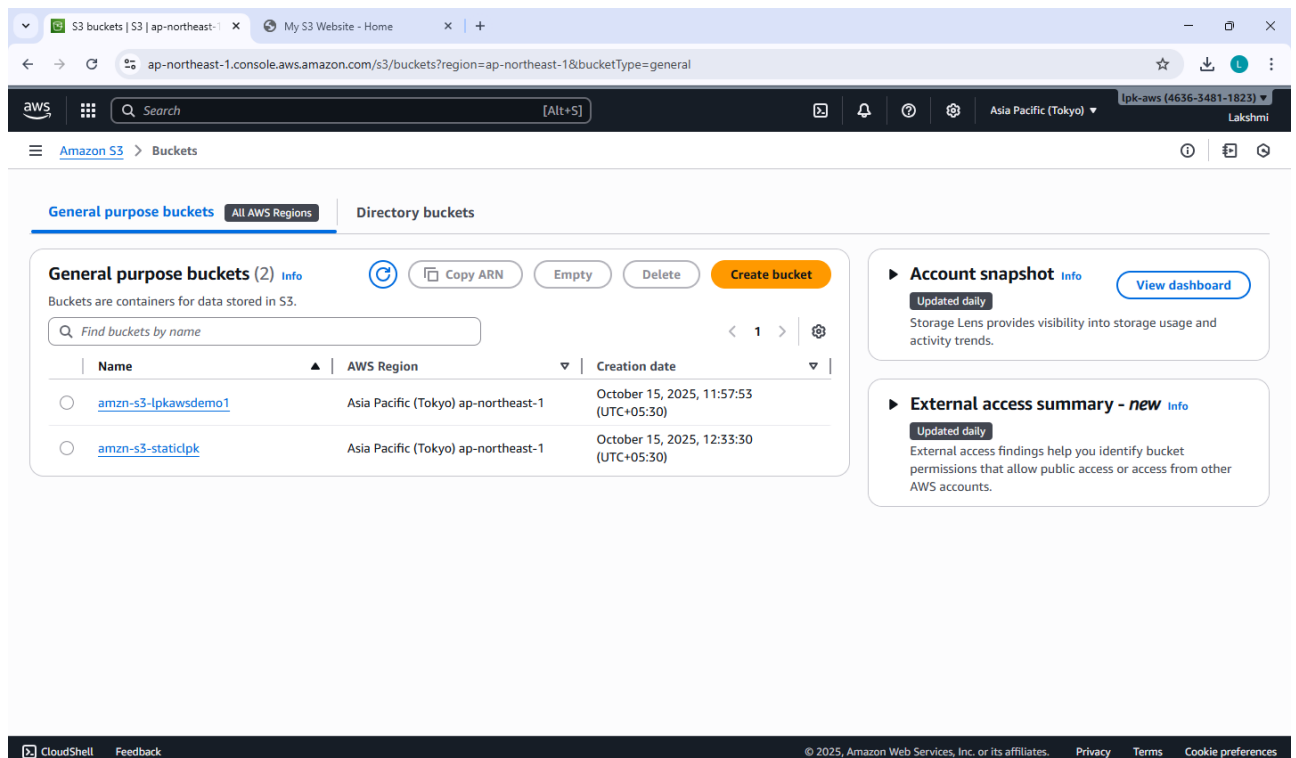
When you enable “Static Website Hosting,” your S3 bucket acts like a web server, and AWS provides a public website URL to access it.

You can create a multi-page static website (e.g., index.html, about.html, contact.html) and upload it to S3. Links within these pages allow users to navigate between them just like a normal website.

Steps to Create a Multi-Page Static Website on S3

Step 1: Create an S3 Bucket

- Open the AWS Management Console → Navigate to S3.
- Click Create bucket.
- Select Bucket type: General purpose
- Enter a unique bucket name (e.g., my-static-web-demo).
- Select ACLs enabled under Object Ownership section.
- Uncheck “Block all public access” → acknowledge.
- Click Create bucket.



Step 2: Prepare Website Files

Before uploading, organize your files in a folder structure as follows:

my-website/

```
|
|
|— index.html
|— about.html
|— contact.html
|— error.html
|— images/
    |— banner.jpg
```

Each HTML file should include navigation links.

Step 3: Upload Website Files

- Open your S3 bucket → Click Upload.
- Add all files and folders (HTML, CSS, JS, images).
- Click Upload to store them in S3.

Step 4: Enable Static Website Hosting

- Go to the **Properties** tab of the bucket.
- Scroll down to Static website hosting → Click Edit.
- Choose Enable and select 'Host a static website'.
- Set:
 - Index document: index.html
 - Error document: error.html
- Click Save changes.

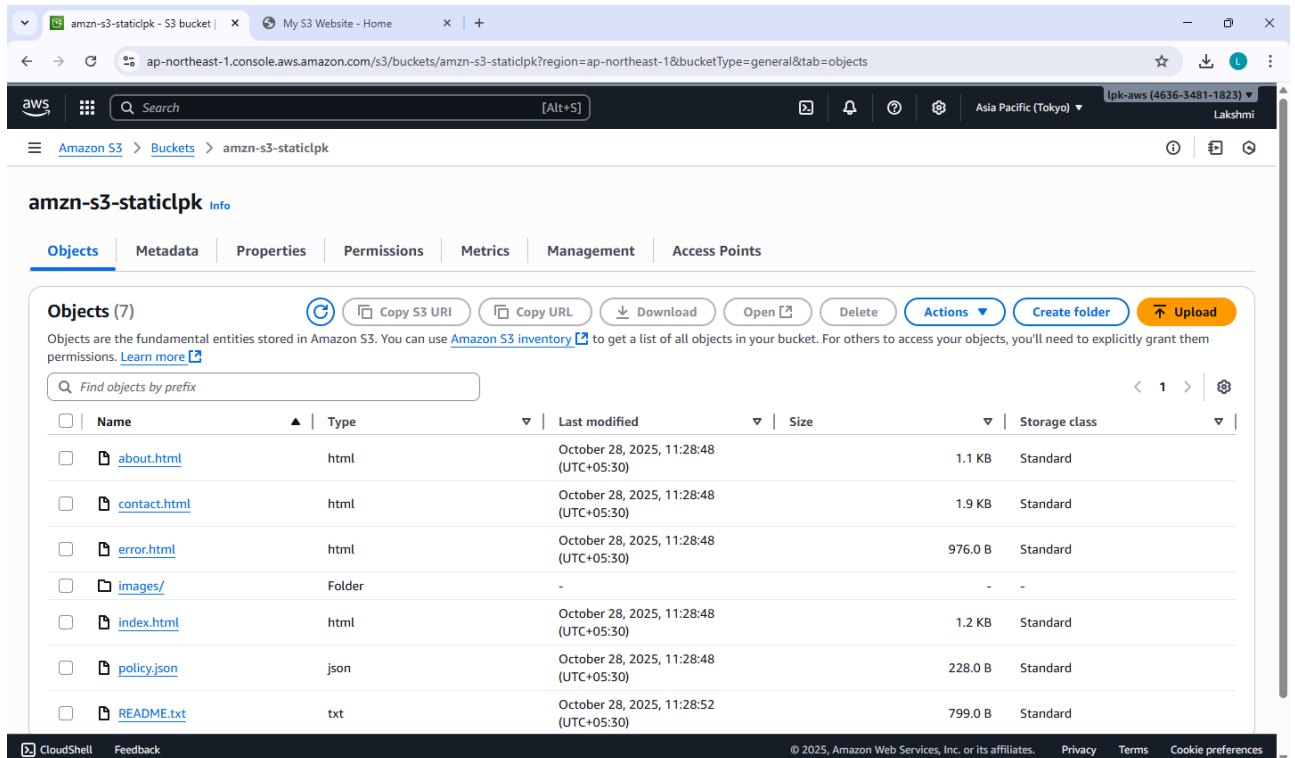
Step 5: Make Files Public (Bucket Policy)

By default, your files are private. To make them public:

- Go to the **Permissions** tab → Bucket Policy → Edit.
- Paste the following policy (replace bucket name):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Sid": "PublicReadGetObject",
      "Effect": "Allow",
      "Principal": "*",
      "Action": "s3:GetObject",
      "Resource": "arn:aws:s3:::my-static-web-demo/*"
    }
  ]
}
```

- Save the changes.



Step 6: Access Your Website

- Go to the Properties tab → Scroll to Static website hosting.
- Copy the Bucket Website Endpoint URL.
- Paste it into your browser — your homepage (index.html) should appear.
- Use the header links to navigate between pages (About, Contact, etc.).

Bucket website endpoint

When you configure your bucket as a static website, the website is available at the AWS Region-specific website endpoint of the bucket. [Learn more](#)

<http://amzn-s3-staticlpk.s3-website-ap-northeast-1.amazonaws.com>

When Will the Error Page Be Shown?

If a user enters a wrong URL or tries to access a file that doesn't exist (e.g., /abc.html), Amazon S3 automatically displays the file you set as the **Error document** (error.html).

Amazon S3 Versioning

Versioning allows you to keep multiple versions of an object in a bucket.

If a file is accidentally deleted or overwritten, you can recover the previous version.

Each version gets a unique version ID.

Steps to Enable Versioning

- Go to your S3 bucket
- Open the Properties tab
- Scroll to Bucket Versioning
- Click Edit → Enable
- Click Save changes

Now whenever you upload a file with the same name, S3 will keep both versions.

Note: You can view versions by clicking “List versions” in the bucket objects page.
--

To Restore or Delete a Specific Version

- Click the object name → Versions
- Select the desired version → Download / Delete
(Deleting only adds a delete marker — older versions are still stored.)

Cross-Region Replication (CRR) – is a rule created on S3

CRR automatically copies objects from one S3 bucket (source) to another (destination) in a different AWS Region.

Used for disaster recovery, compliance, or low-latency access in another region.

Requires Versioning to be enabled on both buckets.

Steps to Set Up CRR

NOTE: Enable Versioning on both:

Source bucket

Destination bucket

Step 1: Choose a different region before you create Destination bucket
Create Destination Bucket first.

Step 2: Create Source Bucket

Give replication permission:

Source bucket → Management tab → Replication rules → Create rule

Step 3: Create Replication Rule page

Enter a Replication rule name

Status: Enabled

Source bucket section:

Choose a rule scope: select “Apply to all objects in the bucket”

Destination:

Select “Choose a bucket in this account”

Bucket name: Select the destination bucket

IAM role:

Select “Create new role”

[An IAM role gives Amazon S3 permission to replicate objects from your source bucket to your destination bucket.

S3 replication won’t work unless you assign (or create) a role with the right permissions.

IAM Role: Either create a new role automatically or choose an existing one]

Step 4: Save

Any new objects uploaded to the source bucket will automatically replicate to the destination region.

Replication rule list - S3 bucket

ap-northeast-1.console.aws.amazon.com/s3/management/s3-amz-source/replication?region=ap-northeast-1&bucketType=general

aws Search [Alt+S] Asia Pacific (Tokyo) lpk-aws (4636-3481-1823) Lakshmi

Amazon S3 > Buckets > s3-amz-source > Replication rules

✓ Replication configuration successfully updated
If changes to the configuration aren't displayed, choose the refresh button. Changes apply only to new objects. To replicate existing objects with this configuration, choose **Create replication job**.

Replication rules Info

Replication enables automatic and asynchronous copying of objects across buckets in the same or different AWS Regions. A replication configuration is a set of rules that define what options should be applied to a group of objects during replication.

Replication configuration settings Create replication job Edit

Configuration settings affect all replication rules in the bucket.

Source bucket
s3-amz-source

Source Region
Asia Pacific (Tokyo) ap-northeast-1

IAM role
[s3crr_role_for_s3-amz-source](#)

Replication rules (1) View details Edit rule Delete Actions Create replication rule

Use replication rules to define options you want Amazon S3 to apply during replication such as server-side encryption, replica ownership, transitioning replicas to another storage class, and more. [Learn more](#)

CloudShell Feedback © 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Note: Replication is not retroactive — only new uploads after enabling CRR are copied.

Reminder – Resource cleanup – to release/delete/terminate the resources created.

Date: 29-10-2025

Exercise-5: Overview of EC2, To Launch a Windows EC2 Instance and Connect via RDP Client

Amazon Elastic Compute Cloud (EC2) is a core AWS Compute service that lets you run virtual servers (instances) in the cloud.

Amazon EC2 is an Infrastructure as a Service (IaaS) offering from AWS.

It allows you to launch virtual machines to host applications and manage them remotely – wherever you are in the world.

Key concepts:

Instance - A virtual machine running in the AWS cloud.

AMI (Amazon Machine Image) - A pre-configured template that includes: OS (Linux, Windows, etc.), Application software, other configurations.

Instance Type - Defines hardware power:

Family	Example	Use Case
General Purpose	t2.micro	Basic web apps
Compute Optimized	c5.large	High-performance computing
Memory Optimized	r5.large	Databases, analytics
Storage Optimized	i3.large	Data warehousing
GPU Instances	g4dn.xlarge	ML/AI, graphics

EBS (Elastic Block Store) - Persistent storage for your EC2 instance. Acts like a hard drive — data remains even after instance stops. Types: SSD, HDD, etc.

Security Groups - Virtual firewalls controlling inbound and outbound traffic.

Example: Allow HTTP (port 80), SSH (port 22), HTTPS (port 443)

Key Pair - Used for secure login (SSH for Linux, RDP for Windows). Consists of a public key (stored in AWS) and private key (.pem) that you download.

Common Ways to Access EC2

- SSH (Linux instances)
- RDP (Windows instances) - Use Remote Desktop with Administrator password.
- User Data Script: Run automation commands during instance launch.

EC2 Use Cases

- Hosting static or dynamic websites
- Deploying web servers (Apache/Nginx)
- Running applications, APIs, or databases
- Machine Learning model hosting
- Batch processing jobs

Pricing Models

- On-Demand: Pay per hour/second; flexible.

- Reserved Instances: 1–3 year commitment; cheaper.
- Spot Instances: Unused capacity; up to 90% cheaper.
- Free Tier: t2.micro or t3.micro free for 6 months.

Lifecycle of an Instance

Step	Description
Launch	Choose AMI, type, key, security group
Running	Accessible and operational
Stop	Instance paused, EBS persists
Start	Boot again from same EBS
Terminate	Deleted permanently, data lost unless backed up

Two types of IPv4 addresses

When you launch an EC2 instance, AWS automatically assigns two types of IPv4 addresses depending on your network settings (VPC, subnet, etc.):

1. Private IPv4 Address

- Purpose: Used for internal communication within the same VPC (Virtual Private Cloud).
- Assigned Automatically: Yes, by AWS from the subnet's private IP range.
- Visibility: Not accessible from the Internet.
- Use Case:
 - Instance-to-instance communication inside AWS (e.g., web server ↔ database server).
 - Internal services that do not need public internet access.
- Persistence: The private IP remains attached to the instance until it is terminated.

2. Public IPv4 Address

- Purpose: Used for communication over the Internet.
- Assigned Automatically:
 - Yes, if your subnet is public (i.e., auto-assign public IP enabled).
 - No, if it's a private subnet.
- Visibility: Accessible from the Internet.
- Use Case:
 - Accessing the instance via SSH or HTTP from your local system.
 - Hosting web applications publicly.
- Persistence:
 - The public IP changes each time you stop/start the instance.
 - To make it permanent, you can assign an Elastic IP (static public IP).

Remote Desktop Protocol [RDP], is a secure communication protocol developed by Microsoft that allows a user to connect to and control another computer remotely. It establishes an encrypted channel to transmit keyboard and mouse inputs from the client to the remote computer and send screen information back to the client, providing a graphical user interface for remote access.

An **RDP client** is the software or app that you use to make this remote connection. It connects to a remote Windows server or Windows EC2 instance that is running an RDP server (which listens on port **3389**). It is pre-installed in Windows.

Steps to Launch a Windows EC2 Instance and Connect via RDP Client

Step 1: Sign in to AWS Management Console

Select the **Region** closest to you.

Step 2: Open the EC2 Dashboard

In the AWS Console, search for **EC2** in the search bar.

Click on EC2 → Instances → Launch Instance.

Under Name and Tags, give your instance a name.

Step 3: Choose an Amazon Machine Image (AMI)

Under Application and OS Images (Amazon Machine Image) → click Browse more AMIs or choose: Microsoft Windows Server 2022 Base (Free tier eligible).

Step 4: Choose Instance Type

Choose t3.micro (Free-tier eligible).

Step 5: Configure Key Pair

Under Key pair (login), choose an existing key pair or create a new one.

If creating a new key pair:

- Choose type: RSA
- Format: .pem
- Download and save it safely — it's required to decrypt your Windows password later.

Step 6: Configure Network Settings

Leave default VPC and Subnet settings.

Under Firewall (security group) →

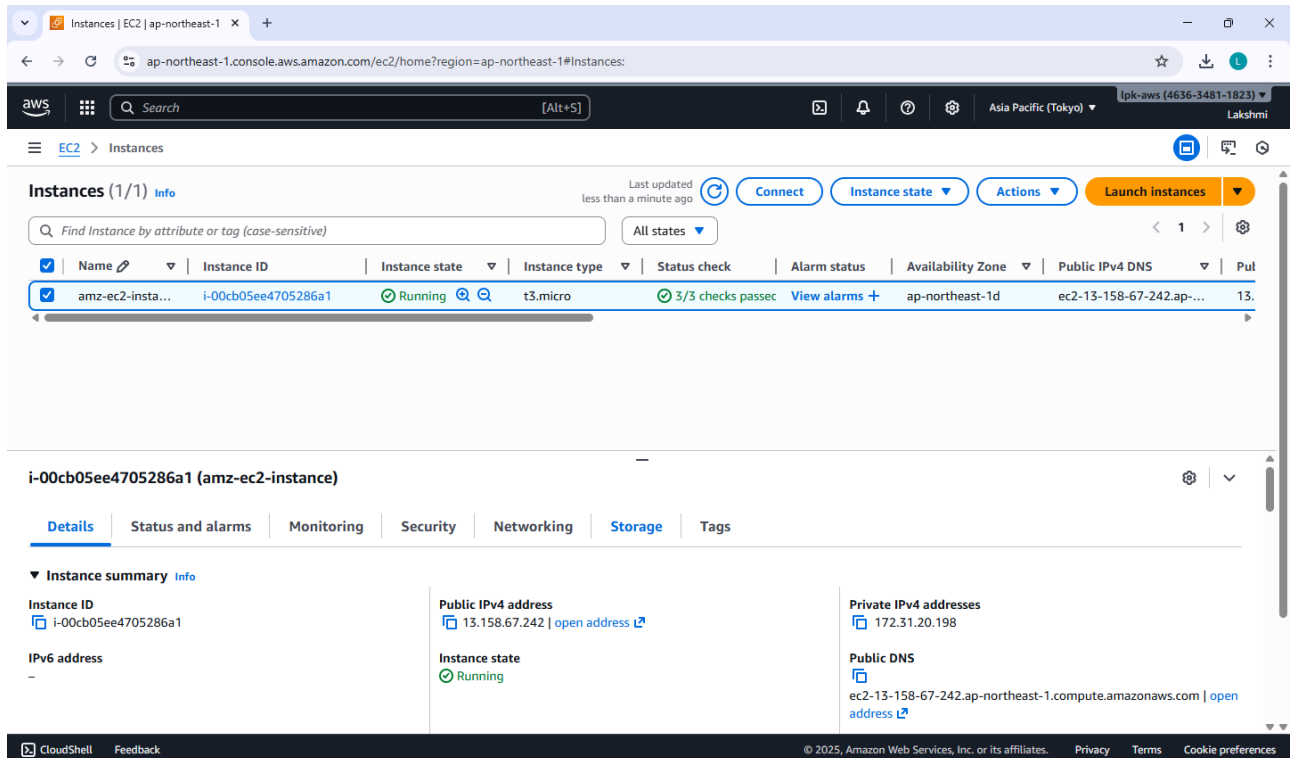
- Select Create security group.
- Allow RDP (port 3389) access from My IP (for better security) or anywhere (0.0.0.0/0).

Step 7: Launch the Instance

Review all configurations.

Click Launch Instance.

Wait until the Instance state changes to Running and Status check = 3/3 passed.



Note:

Wait approximately 5 minutes after instance launch to allow AWS to fully initialize the instance and make the Administrator password available.

When a Windows EC2 instance is first launched, AWS needs a few minutes to:

Initialize the instance, Attach the root volume, Generate the Windows Administrator password (encrypted using your key pair).

Step 8: Get the Administrator Password

Select your running instance → click Connect → choose RDP Client tab.

Click Get Password (you must wait about 4 minutes after launch).

Upload your .pem key file and click Decrypt Password.

Copy the Public IPv4 address and Administrator Password shown.

Step 9: Connect Using RDP Client

On Windows system:

1. Open Remote Desktop Connection (from Start Menu).
2. Enter your instance's Public IPv4 address.
3. Click Connect → Enter:
 - Username: Administrator
 - Password: *(the decrypted password from AWS)*
4. Click OK → accept the certificate → the remote Windows desktop opens!

Step 10: Verify Connection

- You should now see a Windows Server desktop running inside your local window.
- You can open File Explorer, browse settings, or install software (within the free-tier limits).

Note:

- Always **Stop** (not Terminate) the instance when not in use to avoid charges.

- RDP uses port 3389, so ensure it's open in the security group.
- Avoid sharing your decrypted password or key pair.

The screenshot displays the AWS Management Console interface for the 'Instances' page in the 'ap-northeast-1' region. A green notification bar at the top indicates 'Successfully initiated stopping of i-00cb05ee4705286a1'. Below this, the 'Instances (1/1)' table shows a single instance with the ID 'i-00cb05ee4705286a1', which is in a 'Stopped' state. The instance type is 't3.micro' and it is located in the 'ap-northeast-1d' availability zone. A 'Remote Desktop Connection' dialog box is open in the foreground, showing the 'Computer' field with the value '13.158.67.242' and the 'User name' field with the value 'Administrator'. The dialog also includes a 'Connect' button and a 'Help' button. The console interface includes a search bar, a 'Connect' button, and various tabs for instance details such as 'Details', 'Status and alarms', 'Monitoring', 'Security', 'Networking', 'Storage', and 'Tags'.

Date: 30-10-2025

Exercise-6: Launch a Linux EC2 Instance and Connect using SSH through PowerShell/Linux Terminal and PuTTY on Windows.

Note: Choosing the Correct Key Pair Format

While creating a Key Pair, you are asked to select the Private Key File Format - either .pem or .ppk. The correct choice depends on the operating system and the method you will use to connect to your EC2 instance.

Scenario	Key File Format	Explanation
Using PuTTY on Windows	.ppk	The .ppk file is specific to the GUI based PuTTY application, a popular SSH client for Windows system.
Using PowerShell on Windows, Linux terminal on Linux	.pem	The .pem file is the default AWS key format used by the OpenSSH client available in PowerShell (Windows), Linux, and macOS terminals. Used for CLI.

Steps to Launch a Linux EC2 Instance and Connect using SSH through PowerShell/Linux

Step 1: Sign in to AWS Management Console

select the nearest AWS Region.

Step 2: Open EC2 Service

In the search bar, type EC2 and click EC2 Dashboard.

Select Instances → Launch Instance.

Step 3: Configure Instance Details

Under Name and Tags, give your instance a name, e.g., *Linux-SSH-Demo*.

Under Application and OS Images (AMI) → select Amazon Linux 2 AMI (Free tier eligible).

Under Instance type, choose t3.micro (Free-tier eligible).

Step 4: Create or Select a Key Pair

Under Key pair (login) → choose Create new key pair.

Choose:

- Key pair type: RSA
- Private key file format: .pem (for SSH via PowerShell/Linux)

Click Create key pair → the .pem file will automatically download.

Save it securely on your local machine (you'll need it for SSH login).

Step 5: Configure Network Settings

Under Network settings, leave the defaults.

In Firewall (security group) → select Create security group.

Ensure SSH (port 22) is allowed:

- Type: SSH
- Protocol: TCP
- Port Range: 22
- Source: My IP (recommended for security) or Anywhere (0.0.0.0/0).

Step 6: Launch the Instance

Review the configuration → click Launch Instance.

Wait for the Instance State to show Running and Status Checks: 3/3 passed.

Step 7: Get the Public IP Address

Select your instance → In the summary section →

Copy the Public IPv4 address — you'll use this to connect.

Step 8: Connect using SSH

(A) On Windows using PowerShell

- Open **PowerShell** (search "PowerShell" from the Start menu).
- Navigate to the folder where your .pem file is saved:
`cd "C:\Users\<YourName>\Downloads"`
- Connect using the SSH command:
`ssh -i "keyfile.pem" ec2-user@<Public-IP-address>`
- When prompted, type **yes** to continue connecting.
- You'll now be logged into your EC2 Linux terminal.

(B) On Linux Terminal (Ubuntu / macOS)

- Open **Terminal**.
- Navigate to the directory where your .pem file is stored.
- Set proper permission for the key file:
`chmod 400 keyfile.pem`
- Connect to the instance:
`ssh -i keyfile.pem ec2-user@<Public-IP-address>`
- Type **yes** when prompted.
- You will be connected to your EC2 instance remotely.

Step 9: Verify Connection

- Once connected, the command prompt will appear as:
`[ec2-user@ip-172-31-xx-xx ~]$`
- Try a few commands:
`uname -a` and `sudo yum update -y` to confirm access.

Step 10: Stop Instance after Use

Go to the EC2 console.

Select your instance → Instance State → Stop Instance (to avoid charges).

Instances | EC2 | ap-northeast-1

ap-northeast-1.console.aws.amazon.com/ec2/home?region=ap-northeast-1#Instances:

Successfully initiated stopping of i-0c89644c13d2635ac

Instances (1/3) Info

Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance State
putty-linux-ex2	i-0c89644c13d2635ac	Stopped
amz-ec2-inst...	i-00cb05ee4705286a1	Terminated
linux-vm	i-0a91ba9163822c79d	Running

i-0a91ba9163822c79d (linux-vm)

Details Status and alarms Monitoring

Instance summary Info

Instance ID
i-0a91ba9163822c79d

IPv6 address
-

ec2-user@ip-172-31-20-179:~

```
Microsoft Windows [Version 10.0.26100.6899]
(c) Microsoft Corporation. All rights reserved.

C:\Users\MCA048\Downloads>ssh -i "pem-linux.pem" ec2-user@13.114.179.136
The authenticity of host '13.114.179.136 (13.114.179.136)' can't be established.
ED25519 key fingerprint is SHA256:Alg80k7qQLFP4CGm4tQHYI7X2aELrP/PScc3PaRSrmM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '13.114.179.136' (ED25519) to the list of known hosts.

_ _ _ _ _
#           Amazon Linux 2023
_ _ _ _ _
#           https://aws.amazon.com/linux/amazon-linux-2023
_ _ _ _ _

[ec2-user@ip-172-31-20-179 ~]$ uname -a
Linux ip-172-31-20-179.ap-northeast-1.compute.internal 6.1.156-177.286.amzn2023.x86_64
#1 SMP PREEMPT_DYNAMIC Tue Oct 21 22:16:56 UTC 2025 x86_64 x86_64 x86_64 GNU/Linux
[ec2-user@ip-172-31-20-179 ~]$
```

Running

ec2-13-114-179-136.ap-northeast-1.compute.amazonaws.com | open address

Instances | EC2 | ap-northeast-1

ap-northeast-1.console.aws.amazon.com/ec2/home?region=ap-northeast-1#Instances:

Instances (1/4) Info

Find Instance by attribute or tag (case-sensitive)

Name	Instance ID	Instance State
win-win	i-0ea5...	Stopped
lin-lin	i-0e959d66790f1116a	Running
win-lin-putty	i-0410...	Stopped
win-lin	i-0d0cf...	Stopped

i-0e959d66790f1116a (lin-lin)

Details Status and alarms

Instance summary Info

Instance ID
i-0e959d66790f1116a

IPv6 address
-

ec2-user@ip-172-31-21-44:~

```
mca048@mcalab1:~/Downloads$ sudo ssh -i "lin-lin.pem" ec2-user@13.230.41.96
[sudo] password for mca048:

_ _ _ _ _
#           Amazon Linux 2023
_ _ _ _ _
#           https://aws.amazon.com/linux/amazon-linux-2023
_ _ _ _ _

Last login: Tue Nov  4 06:40:53 2025 from 49.206.243.162
[ec2-user@ip-172-31-21-44 ~]$
```

Running

ec2-13-230-41-96.ap-northeast-1.compute.amazonaws.com | open address

Steps to Install PuTTY on Windows

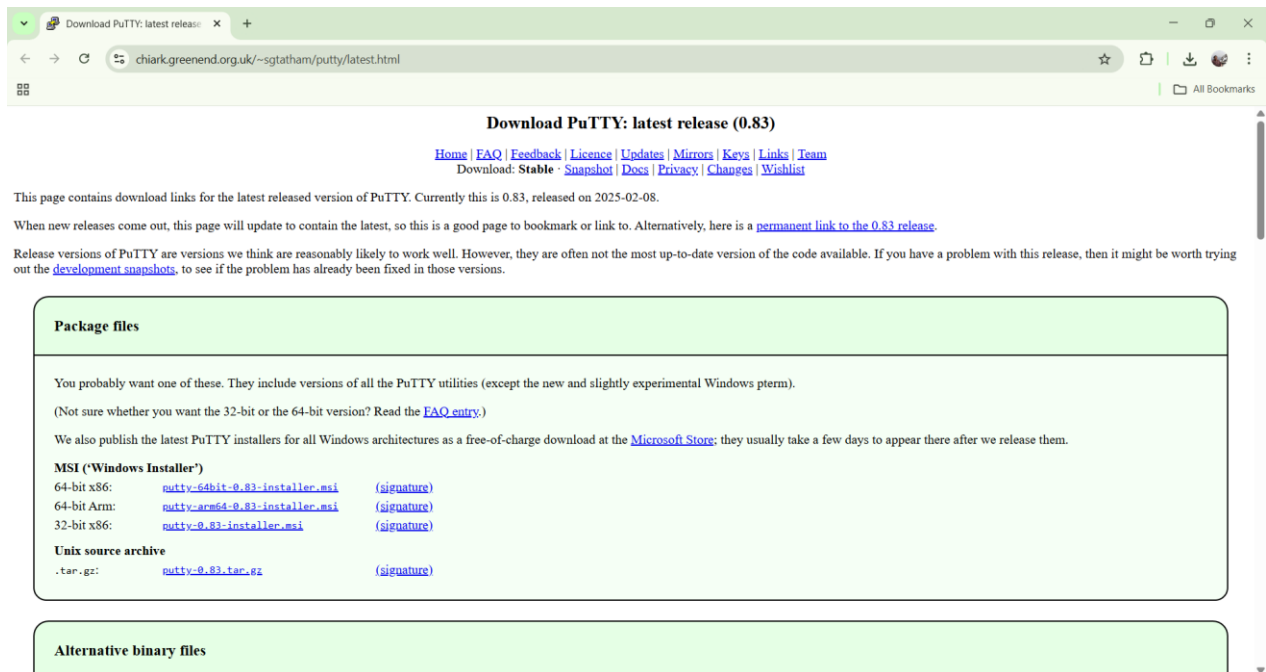
PuTTY is a client program for the SSH, Telnet and Rlogin network protocols. These protocols allow you to interact with a remote server as if you were sitting right in front of it.

It is primarily used in the Windows platform.

In an era where cloud computing and remote servers are the norm, being able to securely connect and interact with these servers is vital. It provides a secure and reliable way to connect to these remote systems. It supports a range of network protocols including the secure ones like SSH.

Use the correct, safe download link: Official download page: This is the only genuine PuTTY source.

<https://www.chiark.greenend.org.uk/~sgtatham/putty/latest.html>

A screenshot of a web browser displaying the PuTTY download page. The browser's address bar shows the URL 'chiark.greenend.org.uk/~sgtatham/putty/latest.html'. The page title is 'Download PuTTY: latest release (0.83)'. Below the title, there are links for 'Home', 'FAQ', 'Feedback', 'Licence', 'Updates', 'Mirrors', 'Keys', 'Links', and 'Team'. A line of text indicates the download is 'Stable', with links for 'Snapshot', 'Docs', 'Privacy', 'Changes', and 'Wishlist'. A paragraph states that the page contains download links for the latest released version of PuTTY, currently 0.83, released on 2025-02-08. Another paragraph mentions that when new releases come out, the page will update to contain the latest, and provides a 'permanent link to the 0.83 release'. A third paragraph explains that release versions of PuTTY are versions thought to be reasonably likely to work well, but they are often not the most up-to-date version of the code available, and suggests trying out 'development snapshots' if there's a problem. The page features two main sections: 'Package files' and 'Alternative binary files'. The 'Package files' section includes a note about the utilities and a link to the FAQ entry. It also mentions that the latest PuTTY installers for all Windows architectures are available as a free-of-charge download at the Microsoft Store. Under the 'MSI (Windows Installer)' heading, there are three links: 'putty-64bit-0.83-installer.msi' (signature), 'putty-arm64-0.83-installer.msi' (signature), and 'putty-0.83-installer.msi' (signature). Under the 'Unix source archive' heading, there is a link for 'putty-0.83.tar.gz' (signature). The 'Alternative binary files' section is currently empty.

Under the **Package files** section, look for:

MSI ('Windows Installer')

64-bit x86: **putty-64bit-0.83-installer.msi**

Click the link: **putty-64bit-0.83-installer.msi**

Package files

You probably want one of these. They include versions of all the PuTTY utilities.

(Not sure whether you want the 32-bit or the 64-bit version? Read the [FAQ](#).)

We also publish the latest PuTTY installers for all Windows architectures as follows:

MSI ('Windows Installer')

64-bit x86:	putty-64bit-0.83-installer.msi	(signature)
64-bit Arm:	putty-arm64-0.83-installer.msi	(signature)
32-bit x86:	putty-0.83-installer.msi	(signature)

Unix source archive

.tar.gz:	putty-0.83.tar.gz	(signature)
----------	-----------------------------------	-----------------------------

Once it downloads, open the file and follow: Next → Next → Install → Finish

After installation, you will have:

- PuTTY – to connect to your EC2 instance via SSH
- PuTTYgen – to convert .pem to .ppk (if needed)
- Pageant – optional key manager

You can open PuTTY by typing **PuTTY** in the Windows search bar/start menu.

Steps to Launch a Linux EC2 Instance and Connect using PuTTY on Windows

Step 1: Sign in to AWS Management Console

Select your nearest AWS Region.

Step 2: Open the EC2 Service

In the AWS Console search bar, type EC2 and select EC2 Dashboard.
Click Instances → Launch Instance.

Step 3: Configure Instance Details

Under Name and Tags, enter an instance name, e.g., *Linux-PuTTY-Demo*.
Under Application and OS Images (AMI) → choose Amazon Linux 2 AMI (Free tier eligible).
Under Instance Type, select t2.micro (Free tier eligible).

Step 4: Create or Select a Key Pair

Under Key pair (login) → select Create new key pair.
Choose the following:

- Key pair type: RSA
- Private key file format: .ppk (*for PuTTY on Windows*)

Click Create key pair → a .ppk file will download automatically.
Save it securely — this file is required to connect later.

Step 5: Configure Network Settings

Under Network settings, leave default VPC/Subnet settings.
Under Firewall (security group):

- Select Create security group.
- Ensure SSH (port 22) is allowed:
 - Type: SSH
 - Protocol: TCP
 - Port Range: 22
 - Source: anywhere.

Step 6: Launch the Instance

Review all configurations.

Click Launch Instance.

Wait until the Instance State changes to Running and Status Check = 3/3 passed.

Step 7: Obtain the Public IP

Select your instance → In the summary section →

Copy the Public IPv4 address or Public DNS (IPv4) — you'll use this to connect from PuTTY.

Step 8: Connect using PuTTY

Open **PuTTY** on your Windows system.

In the **Host Name (or IP address)** field, enter: ec2-user@<Public-IP-address>

In the **Category** list on the left, expand: Connection → SSH → Auth → Credentials

Click **Browse** → locate and select your .ppk key file downloaded earlier.

Click **Open**.

When prompted, click **Accept** to trust the host.

A terminal window opens — you're now connected to your EC2 Linux instance!

Step 9: Verify Connection

Once connected, your prompt should look like: [ec2-user@ip-172-31-xx-xx ~]\$

Try verifying: uname -a or update packages: sudo yum update -y

Step 10: Stop the Instance

Return to the EC2 Dashboard.

Select your instance → click Instance State → Stop Instance.

This prevents charges when you're not using it.

Note

- Use .ppk format key when connecting with **PuTTY (Windows)**.
- Use .pem format key when connecting with **PowerShell / Linux / macOS terminal**.
- Both keys serve the same purpose — secure authentication to your EC2 instance.

The screenshot displays the AWS Management Console interface for an EC2 instance. The instance is named 'putty-linux-ex2' and is currently in the 'Running' state. It is a t3.micro instance type located in the ap-northeast-1 region. A terminal window is open, showing the command 'ssh -i my-key-pair ec2-user@ip-172-31-24-194~' and the output of the 'uname -a' command, which shows the kernel version as 6.1.156-177.286.amzn2023.x86_64.

Date: 5-11-2025

Exercise-7: Hosting a static website on EC2 instance.

- Manual Installation of Apache (httpd) Web Server on EC2 for Static Website Hosting.
- Launching an EC2 Instance with User Data Script to Automatically Install Apache and Host a Static Website.

When you create an EC2 instance, it's just a **bare machine** — It does not have the software to host a website yet. To host a website:

- You need a web server software → Apache (httpd)
- You put your website files (HTML, CSS, etc.) in a special folder (usually /var/www/html)
- Apache listens on port 80 (HTTP) or port 443 (HTTPS) for incoming connections

Apache HTTP Server (often called Apache or httpd) is a web server software.

It runs on a computer (like your EC2 instance) and delivers web pages (HTML, images, CSS, etc.) to users over the HTTP/HTTPS protocol.

So, whenever someone types your website's URL (like http://your-ec2-ip/), Apache receives that request and serves your webpage files from your server to the browser.

httpd stands for HTTP Daemon.

A *daemon* in Linux is a background service that keeps running to handle requests.

So, httpd is the background process that runs the Apache web server.

When you install Apache, you're really installing the **httpd service**.

File Locations (default) – Website files go into: /var/www/html

Steps for Manual Installation of Apache (httpd) Web Server on EC2 for Static Website Hosting.

Part 1 – Creating an EC2 Instance

Step 1: Sign in to AWS Management Console

In the search bar, type EC2 and open the EC2 Dashboard.

Step 2: Launch a New Instance

Click “Launch Instance.”

Give a name. (e.g., MyApacheServer)

Step 3: Choose an Amazon Machine Image (AMI)

Select Amazon Linux 2 AMI (Free tier eligible)

Step 4: Choose Instance Type

Choose t3.micro (Free tier eligible).

Step 5: Create / Select Key Pair

Under Key pair (login), choose:

Create new key pair → Give a name → File format = .pem
Download the key file → Keep it safe.

Step 6: Configure Network Settings

Click → Allow SSH traffic
→ Allow HTTP traffic from the internet. (This automatically opens port 80 for web access).

Step 7: Launch Instance

Review → Click **Launch Instance**.
Wait for the instance to start.

Step 8: Connect to the Instance

Using powershell type the following commands:

- Open PowerShell (search “PowerShell” from the Start menu).
- Navigate to the folder where your .pem file is saved:
`cd "C:\Users\<YourName>\Downloads"`
- Connect using the SSH command:
`ssh -i "keyfile.pem" ec2-user@<Public-IP-address>`
- When prompted, type yes to continue connecting.
- You'll now be logged into your EC2 Linux terminal.

Part 2 – Manual Installation of Apache (httpd) Web Server

Step 1: Update the Packages

`sudo yum update -y`

Step 2: Install Apache (httpd)

`sudo yum install httpd -y`
(This installs the Apache Web Server package.)

Step 3: Start the Apache Service

`sudo systemctl start httpd`

Step 4: Enable Apache to Start on Boot

`sudo systemctl enable httpd`

Step 5: Check Apache Status

`sudo systemctl status httpd` [It should show active (running). Press q to exit status view.]

Step 7: Test Apache Server

Copy your Public IPv4 address from the EC2 dashboard.
Paste it into a browser: `http://`
You should see the Apache Test Page

Part 3 – Host a Static Website on Apache

Step 1: Move to Web Root Folder


```
cd /var/www/html
```

Step 2: Create your HTML file

```
sudo nano index.html
```

Step 3: Add HTML content

Paste the following code:

```
<!DOCTYPE html>
<html>
<head>
<title>Welcome to My Website</title>
</head>
<body style="text-align:center; background-color: #f0f0f0;">
<h1>Hello from Apache on AWS EC2!</h1>
<p>This is a static website hosted on Apache web server.</p>
</body>
</html>
```

Press Ctrl + O, Enter, then Ctrl + X to save and exit.

After Pressing Ctrl + O You'll see:

File Name to Write: index.html [Just press Enter to confirm saving with that name].

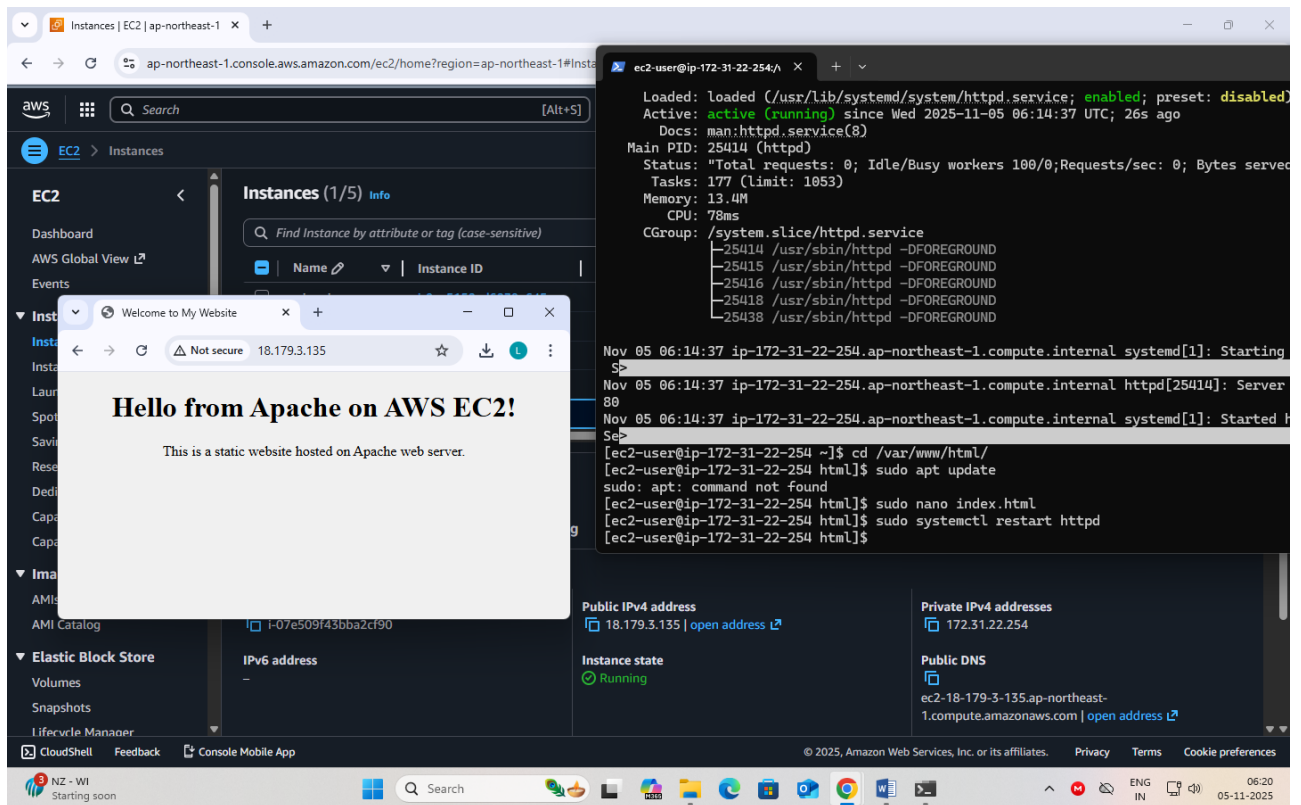
To Exit Nano: press Ctrl + X [This closes the Nano editor and will be back to the Linux prompt]

Step 4: Restart Apache

```
sudo systemctl restart httpd
```

Step 5: View Website

- In browser: <http://<your-ec2-public-ip>>
- You'll see your custom HTML page.



Optional: Add Multiple Pages

- Upload other pages like about.html, contact.html in the same directory.

Steps for launching an EC2 Instance with User Data Script to Automatically Install Apache and Host a Static Website.

In this method, you don't manually install Apache or upload files after connecting. Instead, you write a **shell script** in the **User Data** section (during instance creation). When the EC2 instance starts for the first time, it automatically:

1. Installs Apache (httpd)
2. Starts the service
3. Creates an index.html webpage
4. Hosts your website immediately after launch

Step 1: Sign in to AWS Management Console

Go to EC2 Dashboard → Click Launch Instance

Step 2: Name and OS

Name: AutoApacheWebServer

AMI: Choose Amazon Linux 2 AMI (Free tier eligible)

Instance Type: t3.micro

Step 3: Key Pair

Select existing key pair (or create new) → Format = .pem

Step 4: Network Settings

Allow HTTP traffic from the Internet (port 80)

Allow SSH traffic (port 22)

Step 5: Add User Data Script

Scroll down to Advanced Details → User data box.

This script:

- Updates all packages
- Installs and starts Apache
- Creates a sample index.html webpage in /var/www/html

Paste the following shell script:

```
#!/bin/bash
# Update packages
yum update -y

# Install Apache Web Server
yum install -y httpd

# Start Apache
systemctl start httpd
systemctl enable httpd

# Create website content
echo "<html>
<head> <title>Welcome to My Auto Web Server</title> </head>
<body style='text-align:center; background-color:#e9f5ff;'>
<h1>Welcome to EC2 Instance</h1>
<h2>Apache Installed Automatically via User Data Script</h2>
<p>This is a static website hosted on EC2 using User Data automation.</p>
</body>
</html>" > /var/www/html/index.html
```

Step 6: Launch the Instance

Click Launch Instance

Wait for the status → Running

Step 7: Test Your Web Server

Copy the Public IPv4 address of your instance.

Paste it in your browser: `http://<your-public-ip>`

You should immediately see your webpage without logging into EC2!

Instances | EC2 | ap-northeast-1

ap-northeast-1.console.aws.amazon.com/ec2/home?region=ap-northeast-1#Instances:

EC2

Dashboard

AWS Global View

Events

Instances

Instances (1/6) Info

Find Instance by attribute or tag (case-sensitive)

Name	Instance ID
lin-lin	i-0e959d66790f1116a
win-lin-putty	i-04106ec8d9217c8b1

Welcome to My Auto Web Service

Not secure 35.77.64.161

Welcome to EC2 Instance

Apache Installed Automatically via User Data Script

This is a static website hosted on EC2 using User Data automation.

```
Nov 05 06:36:44 ip-172-31-18-150.ap-northeast-1.compute.internal httpd[3019]: Server con
Nov 05 06:36:44 ip-172-31-18-150.ap-northeast-1.compute.internal systemd[1]: Started htt
~
~
~
lines 1-19/19 (END)
• httpd.service - The Apache HTTP Server
  Loaded: loaded (/usr/lib/systemd/system/httpd.service; enabled; preset: disabled)
  Active: active (running) since Wed 2025-11-05 06:36:44 UTC; 2min 44s ago
  Docs: man:httpd.service(8)
  Main PID: 3019 (httpd)
  Status: "Total requests: 0; Idle/Busy workers 100/0;Requests/sec: 0; Bytes served/s
  Tasks: 177 (Limit: 1053)
  Memory: 13.3M
  CPU: 200ms
  CGroup: /system.slice/httpd.service
          └─3019 /usr/sbin/httpd -DFOREGROUND
             3172 /usr/sbin/httpd -DFOREGROUND
             3173 /usr/sbin/httpd -DFOREGROUND
             3175 /usr/sbin/httpd -DFOREGROUND
             3176 /usr/sbin/httpd -DFOREGROUND
```

Public IPv4 address

35.77.64.161 | open address

Private IPv4 addresses

172.31.18.150

Instance state

Running

Public DNS

ec2-35-77-64-161.ap-northeast-1.compute.amazonaws.com | open address

CloudShell Feedback Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Rain coming In about 2.5 hours

Search

06:40 05-11-2025

Date: 7-11-2025

Exercise-8: Create a Custom AMI from a Working EC2 Instance.

Launch an EC2 Instance using a Custom AMI

Delete the custom AMI

AMI (Amazon Machine Image) is a pre-configured template that contains only Operating System (e.g., Amazon Linux, Ubuntu, Windows) needed to launch an EC2 instance.

When you launch a new EC2 instance, you select an AMI as the base image.

A **Custom AMI** is created from your *own running EC2 instance* after you've installed software, uploaded website files, or applied settings.

Benefits:

1. **Reusability** – Launch multiple identical servers quickly.
2. **Backup** – Acts as a snapshot of your configured instance.
3. **Auto Scaling** – Used by Auto Scaling Groups to create identical instances automatically.
4. **Disaster Recovery** – You can recreate your setup if the original instance fails.
5. **Time-saving** – No need to reinstall Apache or re-upload files each time.

A Custom AMI is like a master copy of your EC2 setup. It ensures your website or app can be duplicated instantly and consistently.

Steps to Create a Custom AMI from a Working EC2 Instance

To capture the current configuration — installed packages, website files, and settings — into a reusable Amazon Machine Image (AMI).

Step 1: Select the running instance

Go to EC2 → Instances.

Select the instance that already hosts your website.

Or create an instance (add user data for web hosting)

Step 2: Create Image

From the Actions → Image and templates → Create image.

Step 3: Enter details

Image name: MyWebsiteAMI

Description: AMI created from configured Apache website instance

Leave “No reboot” unchecked (so the filesystem is consistent).

Step 4: Storage volumes

The root volume will appear automatically; keep defaults unless you need more space.

Step 5: Create image

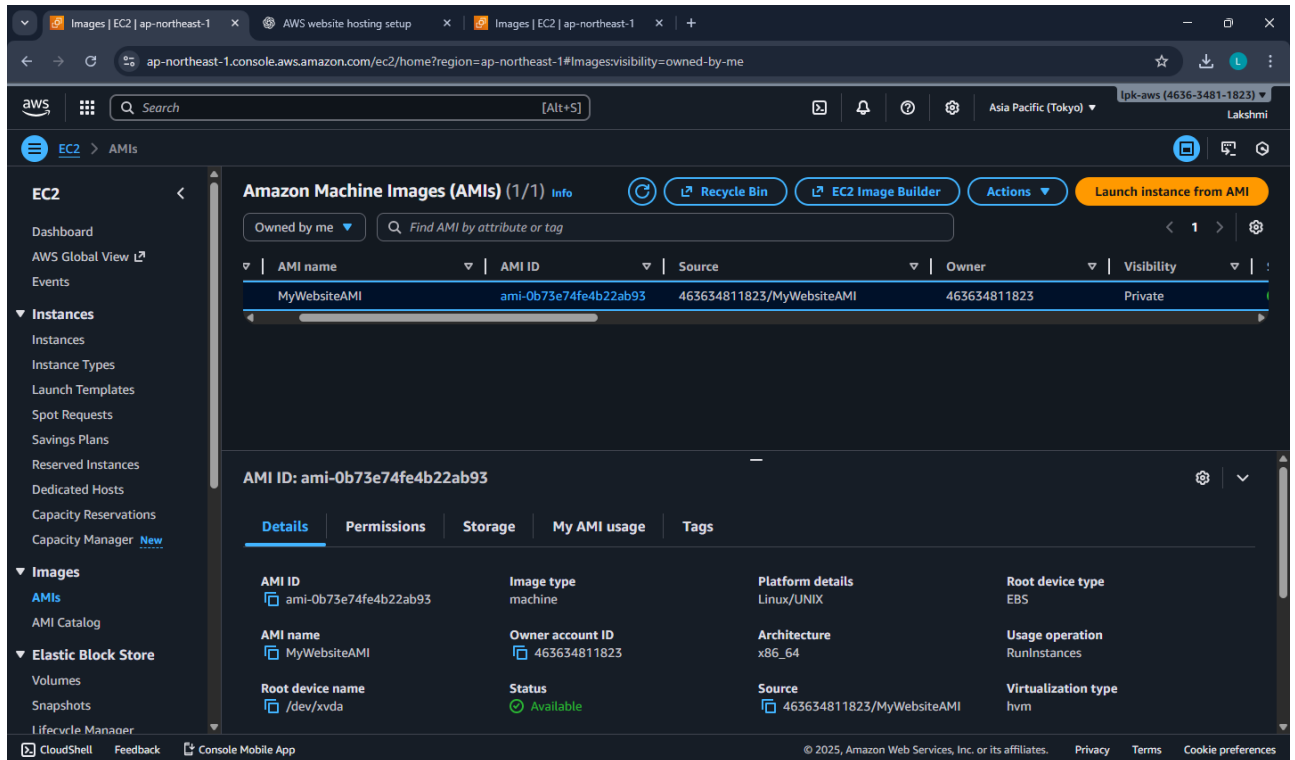
Click **Create image**.

A confirmation message appears; note the **Image ID**.

Step 6: Verify creation

In left panel → **AMIs** → refresh until **Status = Available**.

Your custom AMI is now saved in that region and can be used to launch identical webserver instances.



Steps to Launch an EC2 Instance using a Custom AMI

To launch a new EC2 instance from a previously created Custom Amazon Machine Image (AMI) — containing your configured website and software.

Step 1: Go to AMIs

Open the EC2 Service dashboard.

In the left navigation pane, click AMIs (under “Images”).

Step 2: Select Your Custom AMI

Locate the AMI you created earlier (e.g., MyWebsiteAMI).

Ensure Status = Available.

Step 3: Launch Instance from AMI

Select the AMI → click Launch instance from image.

Step 4: Configure Instance Details

Name: WebServer-from-Custom-AMI

Instance type: t3.micro (Free Tier eligible)

Key pair: Choose an existing .pem key for SSH access.

Network settings:

VPC: Default

Subnet: Public

Security Group: Allow HTTP (80) and SSH (22)

Step 5: Storage (EBS Volume)

Keep default root volume (e.g., 8 GB gp3).

Step 6: Review and Launch

Click Launch instance.

Wait until the instance state becomes Running.

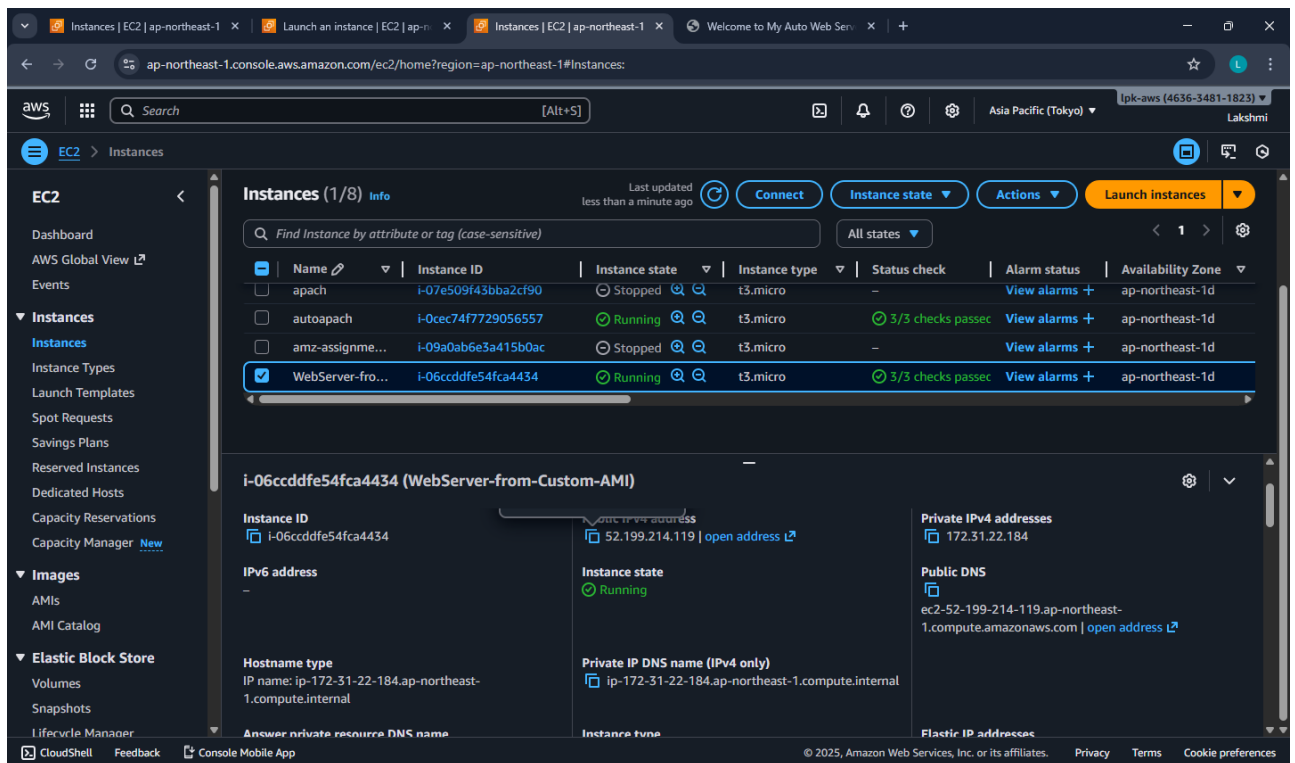
Step 7: Access the Website

Copy the Public IPv4 address from the instance details.

Open in a browser → `http://<Public-IP>`

You should see your same website, confirming the custom AMI works.

A new EC2 instance is successfully launched using the Custom AMI, automatically containing the OS, Apache, configurations, and website files — no manual setup required.



The screenshot displays the AWS Management Console for the 'Instances' page in the 'ap-northeast-1' region. A table lists three instances: 'apach' (Stopped), 'autoapach' (Running), and 'WebServer-from-Custom-AMI' (Running). The 'WebServer-from-Custom-AMI' instance is selected, and its details are shown below. The instance is a 't3.micro' type with a 'Running' state and '3/3 checks passed'. It has a public IPv4 address of '52.199.214.119' and a private IPv4 address of '172.31.22.184'. The public DNS is 'ec2-52-199-214-119.ap-northeast-1.compute.amazonaws.com'. The instance name is 'WebServer-from-Custom-AMI' and its ID is 'i-06ccddfe54fca4434'.

Name	Instance ID	Instance state	Instance type	Status check	Alarm status	Availability Zone
apach	i-07e509f43bba2cf90	Stopped	t3.micro	-	View alarms +	ap-northeast-1d
autoapach	i-0cec74f7729056557	Running	t3.micro	3/3 checks passed	View alarms +	ap-northeast-1d
WebServer-from-Custom-AMI	i-06ccddfe54fca4434	Running	t3.micro	3/3 checks passed	View alarms +	ap-northeast-1d

i-06ccddfe54fca4434 (WebServer-from-Custom-AMI)

Instance ID	Public IPv4 address	Private IPv4 addresses
i-06ccddfe54fca4434	52.199.214.119 open address	172.31.22.184

Instance state
Running

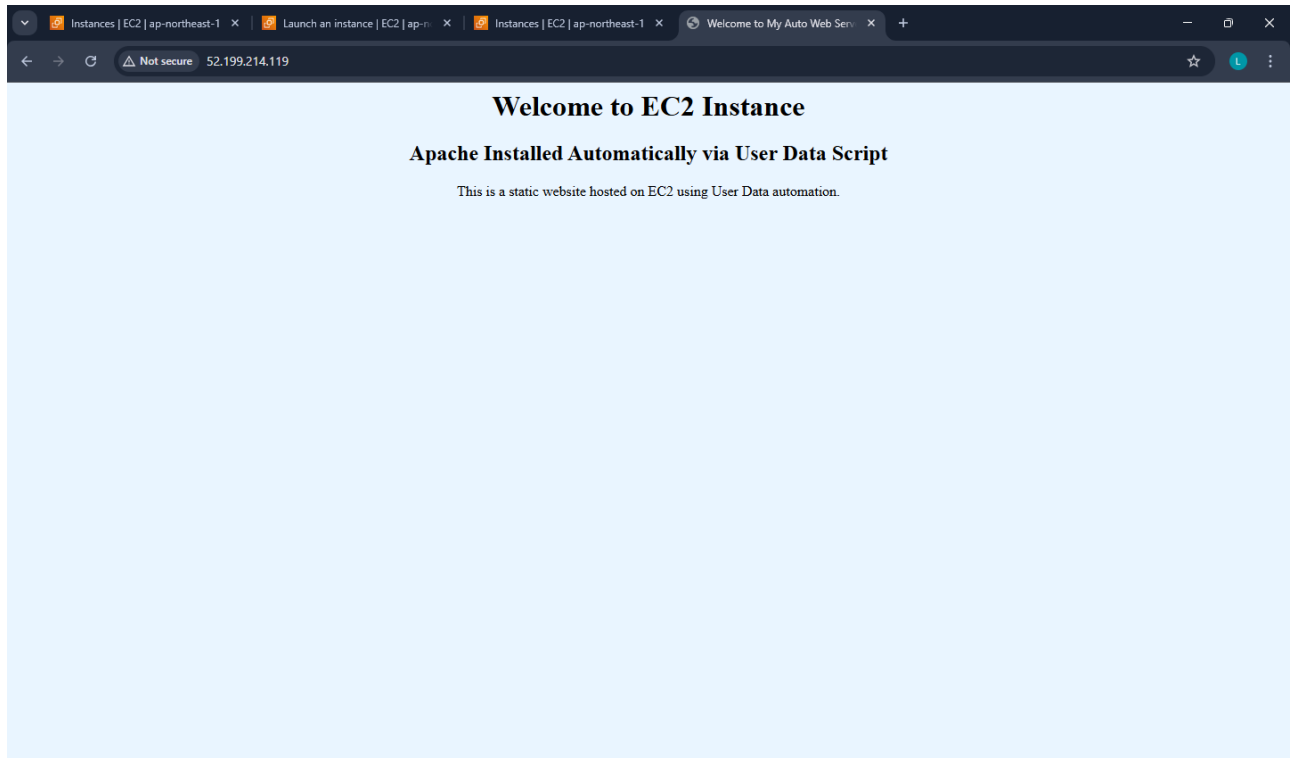
Public DNS
ec2-52-199-214-119.ap-northeast-1.compute.amazonaws.com | open address

Private IP DNS name (IPv4 only)
ip-172-31-22-184.ap-northeast-1.compute.internal

Hostname type
IP name: ip-172-31-22-184.ap-northeast-1.compute.internal

Answer private resource DNS name

Elastic IP addresses



Steps to Delete the custom AMI

Deleting a **custom AMI** involves **deregistering** the image and then **deleting its associated EBS snapshot**.

When you deregister an AMI, it is removed from your account and can no longer be used to launch new instances.

However, the **snapshot** that was created along with the AMI still remains in your storage and continues to incur charges, so you must delete it separately to free up space and stop costs. This ensures your AWS environment remains clean and cost-efficient.

Step 1 – Open EC2 Dashboard

- Sign in to the AWS Management Console.

- Navigate to EC2 service.

- In the left navigation pane, scroll down to Images → AMIs.

Step 2 – Locate Your Custom AMI

- In the Owned by me tab, you will see all AMIs you created (custom AMIs).

- Select the AMI you want to delete.

 - You can identify it by Name or AMI ID.

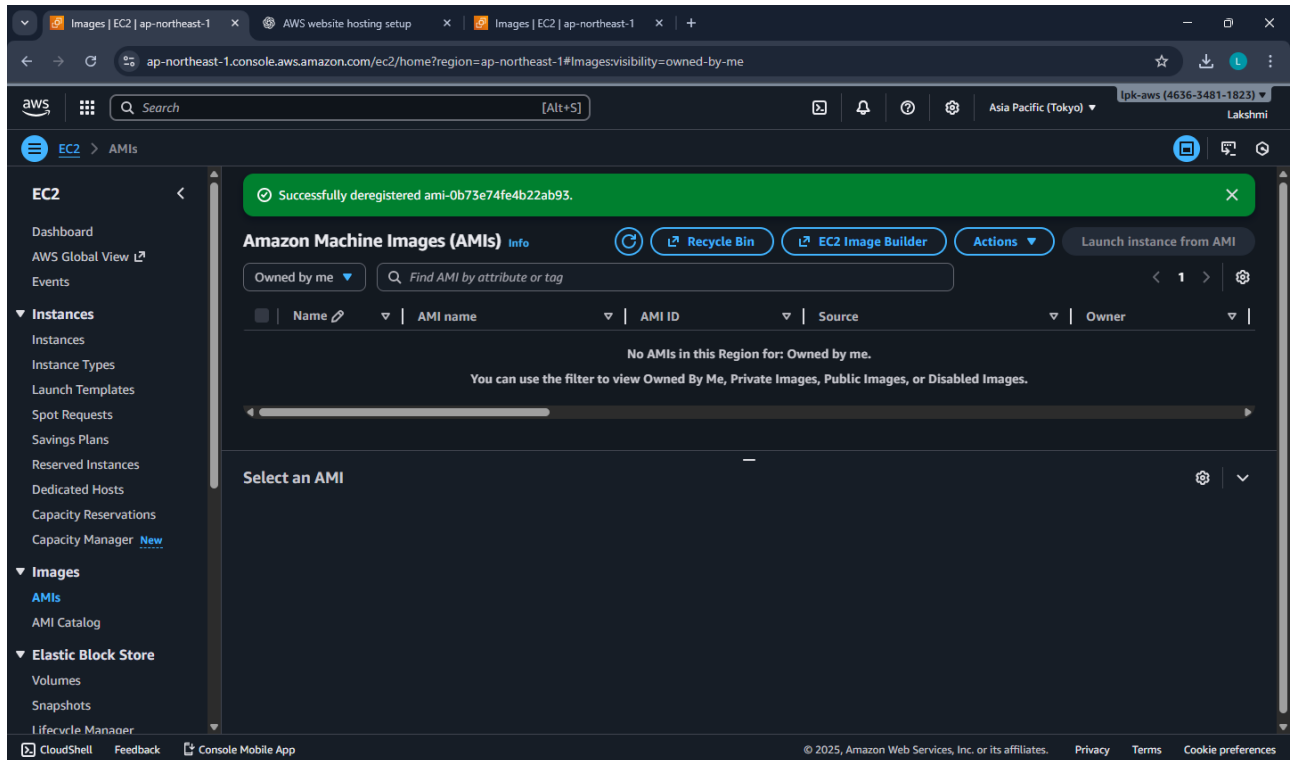
Step 3 – Deregister the AMI

- Select the AMI → Click on Actions dropdown.

- Choose Deregister AMI.

- Confirm by clicking Deregister in the pop-up.

 - This removes the AMI record, but not the underlying snapshot(s).



Step 4 – Delete the Associated Snapshot

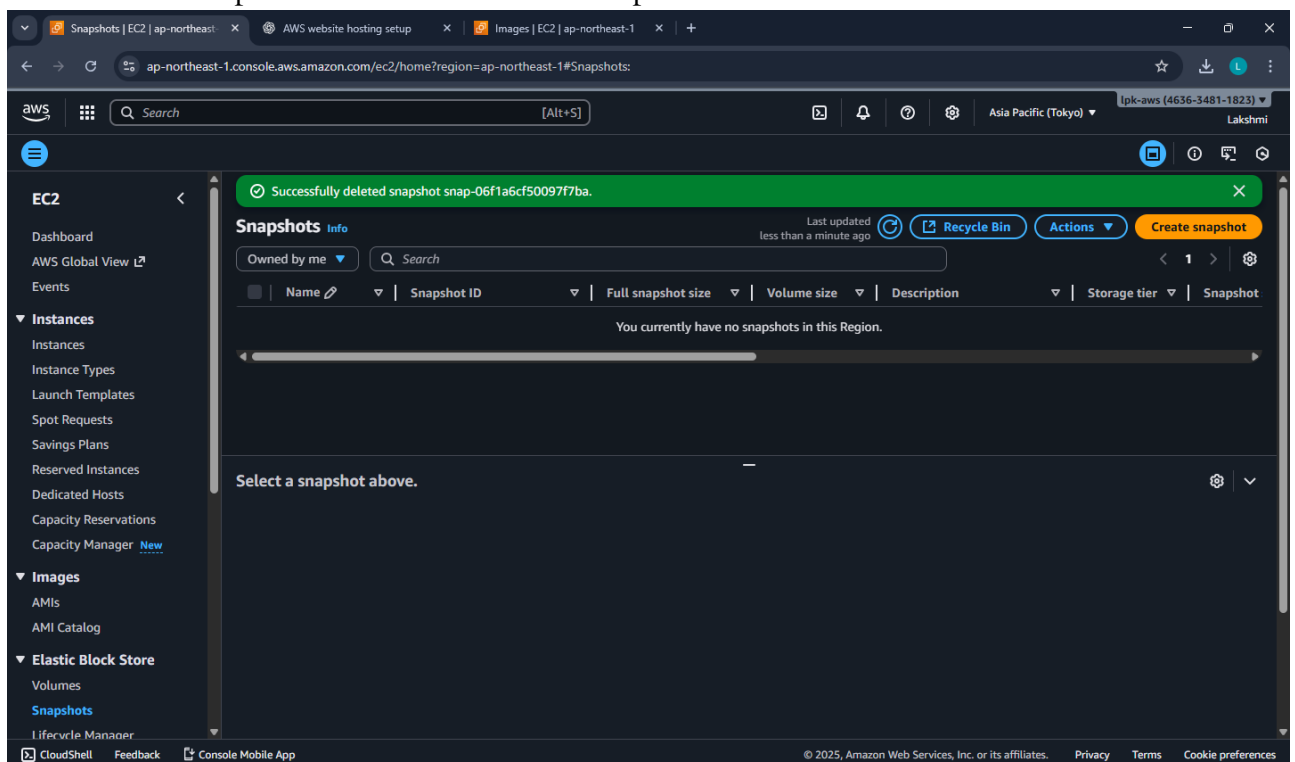
To fully free up storage space (and avoid charges):

In the left navigation pane, click Elastic Block Store → Snapshots.

Find the snapshot linked to your deleted AMI.

You can check the Description column; it usually mentions the AMI ID.

Select the snapshot → Actions → Delete snapshot → Confirm Delete.



- The AMI is now deregistered (unavailable for future instance launches).
- The snapshot is also deleted, freeing up EBS storage and costs.

Date: 11-11-2025

Mini project assessment

Hosting a Multi-Page Website using EC2 and S3

Amazon Web Services (AWS) offers cloud-based resources for hosting and managing web applications.

In this experiment, two AWS services are used together:

- **Amazon S3 (Simple Storage Service):** Used to store image files and make them publicly accessible via HTTPS URLs.
- **Amazon EC2 (Elastic Compute Cloud):** A virtual server used to host and serve the website's HTML and CSS files using **nginx** as the web server.

This setup demonstrates the concept of **decoupling compute and storage** - the website files are served from EC2 while the images are fetched from S3.

Steps:

1. Login to AWS Management Console.

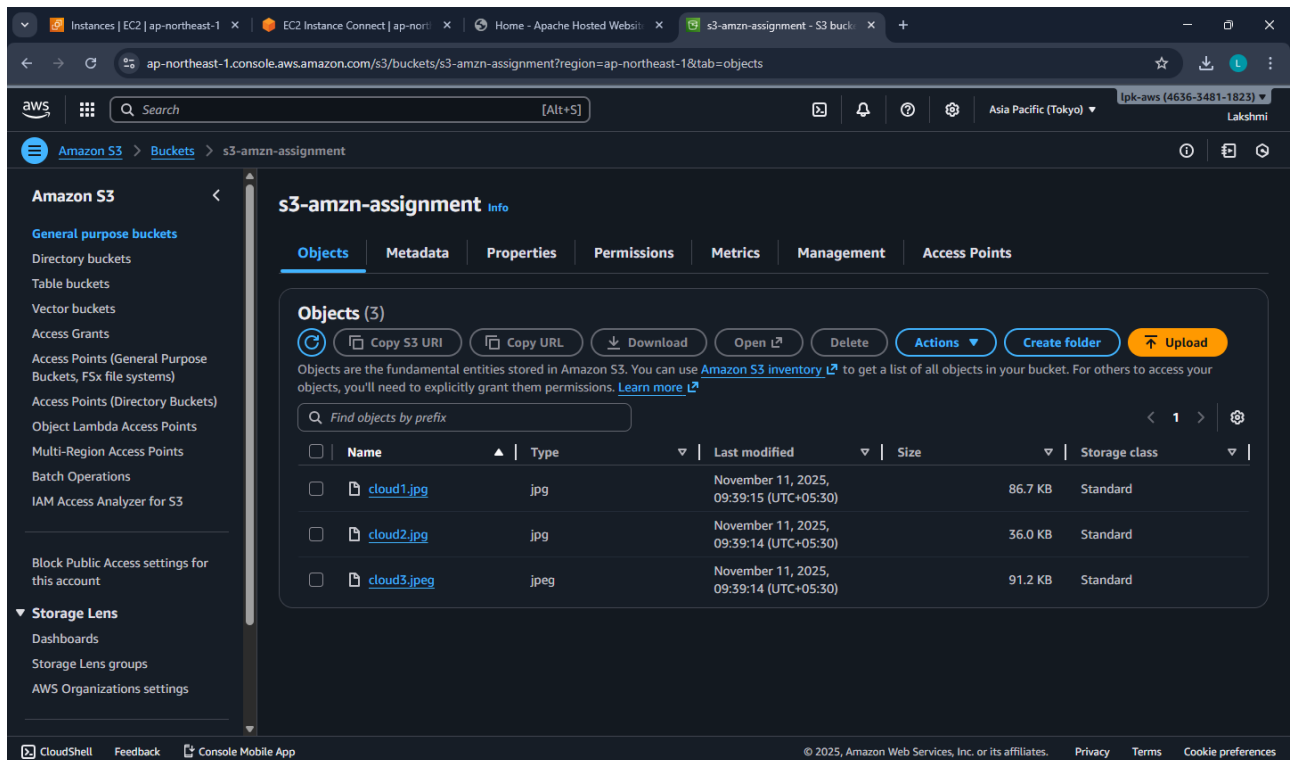
Navigate to the AWS dashboard using your account credentials.

2. Create an S3 Bucket.

- Go to S3 Service → Create bucket.
- Provide a unique bucket name (e.g., *my-website-images-123*).
- Select the region (e.g., *us-east-1*).
- Uncheck "Block all public access".
- Click **Create bucket**.

3. Upload Images to S3.

- Upload 3 images (cloud1.jpg, cloud2.jpg, cloud3.jpg).
- Use **Object actions** → **Make public using ACL** (or apply a bucket policy) to make the images publicly accessible.
- Note the **Object URLs** of each image.



4. Launch an EC2 Instance.

- Open EC2 Service → Launch instance.
- Choose **Amazon Linux 2025 AMI** and instance type **t2.micro** (Free Tier).
- Create or select an existing key pair.
- Allow **HTTP (80)** and **SSH (22)** in the Security Group.
- In **Advanced details** → **User data**, paste your files or upload your files.

This script:

- Updates all packages
- Installs and starts Apache

```
#!/bin/bash
# Update the system
sudo yum -y update

# Install Apache (httpd)
sudo yum -y install httpd

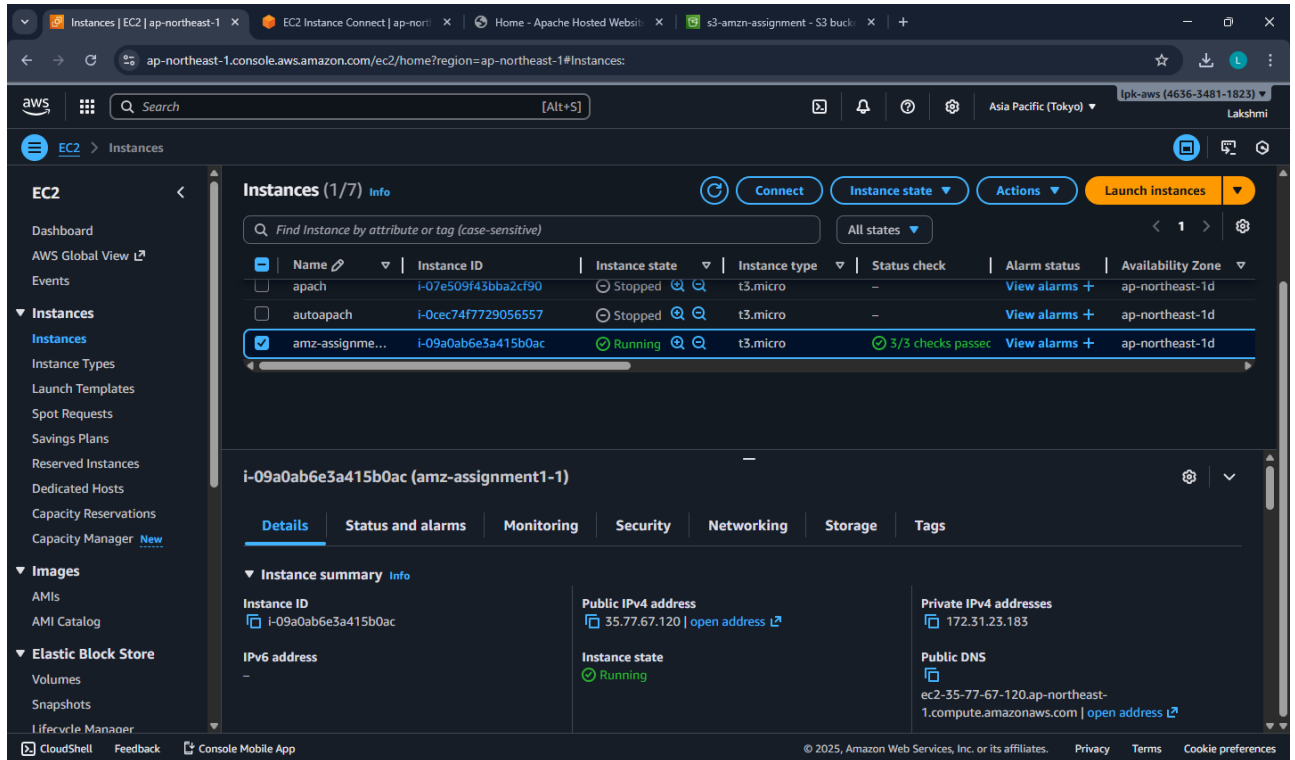
# Enable Apache to start on boot
sudo systemctl enable httpd

# Start the Apache service
sudo systemctl start httpd

sudo chown -R apache:apache /var/www/html
```

```
sudo chmod -R 755 /var/www/html
sudo systemctl restart httpd
```

- Launch the instance.



5. Create the files in the instance.

- Connect to the instance and log in as ec2-user
- Navigate to /var/www/html
- Create index.html, gallery.html, styles.html as required.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Home - Apache Hosted Website</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="bg-animation"></div>
  <div class="particles" id="particles"></div>

  <nav>
```

```

<div class="nav-container">
  <div class="logo"> ⚡ CloudHost</div>
  <div class="nav-links">
    <a href="index.html">Home</a>
    <a href="about.html">About</a>
    <a href="gallery.html">Gallery</a>
  </div>
</div>
</nav>

<div class="container">
  <section class="home-section">
    <div class="hero-content">
      <h1>Apache Hosted Website</h1>
      <p class="subtitle">Cloud-powered infrastructure combining AWS EC2, Apache, and
S3 storage</p>

      <div class="tech-stack">
        <div class="tech-item">
          <h3>AWS EC2</h3>
          <p>High-performance hosting</p>
        </div>
        <div class="tech-item">
          <h3>Apache Server</h3>
          <p>Industry standard</p>
        </div>
        <div class="tech-item">
          <h3>Amazon S3</h3>
          <p>Scalable storage</p>
        </div>
      </div>
    </div>
  </section>
</div>

<footer>
  <p>© 2024 Apache Hosted Website | Powered by AWS Cloud Infrastructure</p>
</footer>

<script src="script.js"></script>
</body>
</html>

```

about.html

```

<!DOCTYPE html>
<html lang="en">
<head>

```

```

<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>About - Apache Hosted Website</title>
<link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="bg-animation"></div>
  <div class="particles" id="particles"></div>

  <nav>
    <div class="nav-container">
      <div class="logo"> ⚡ CloudHost</div>
      <div class="nav-links">
        <a href="index.html">Home</a>
        <a href="about.html">About</a>
        <a href="gallery.html">Gallery</a>
      </div>
    </div>
  </nav>

  <div class="container">
    <section class="about-section">
      <div class="about-content">
        <div class="about-text">
          <h2>About This Project</h2>
          <p>A cutting-edge demonstration of cloud infrastructure showcasing the seamless integration of AWS services to create a robust, scalable web hosting solution.</p>
          <p>This architecture separates compute from storage, ensuring optimal performance, cost-efficiency, and scalability for modern web applications.</p>
        </div>
        <div class="features-grid">
          <div class="feature-box">
            <h4>☁ AWS EC2</h4>
            <p>Elastic compute hosting with high availability and performance</p>
          </div>
          <div class="feature-box">
            <h4>⚡ Apache Web Server</h4>
            <p>Reliable content delivery with industry-standard software</p>
          </div>
          <div class="feature-box">
            <h4>📦 Amazon S3</h4>
            <p>Scalable object storage for assets and media files</p>
          </div>
          <div class="feature-box">
            <h4>🧠 Modern Architecture</h4>
            <p>Best practices in cloud computing and web design</p>
          </div>
        </div>
      </div>
    </section>
  </div>

  <footer>

```

```
<p>© 2024 Apache Hosted Website | Powered by AWS Cloud Infrastructure</p>
</footer>

<script src="script.js"></script>
</body>
</html>
```

gallery.html

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Gallery - Apache Hosted Website</title>
  <link rel="stylesheet" href="styles.css">
</head>
<body>
  <div class="bg-animation"></div>
  <div class="particles" id="particles"></div>

  <nav>
    <div class="nav-container">
      <div class="logo"> ⚡ CloudHost</div>
      <div class="nav-links">
        <a href="index.html">Home</a>
        <a href="about.html">About</a>
        <a href="gallery.html">Gallery</a>
      </div>
    </div>
  </nav>

  <div class="container">
    <section class="gallery-section">
      <div class="gallery-header">
        <h2>Gallery</h2>
        <p>Images served directly from Amazon S3</p>
      </div>

      <div class="gallery-grid">
        <div class="gallery-item">
          
        </div>
        <div class="gallery-item">
          
        </div>
      </div>
    </section>
  </div>
```



```

<div class="gallery-item">
  
</div>
</div>
</section>
</div>

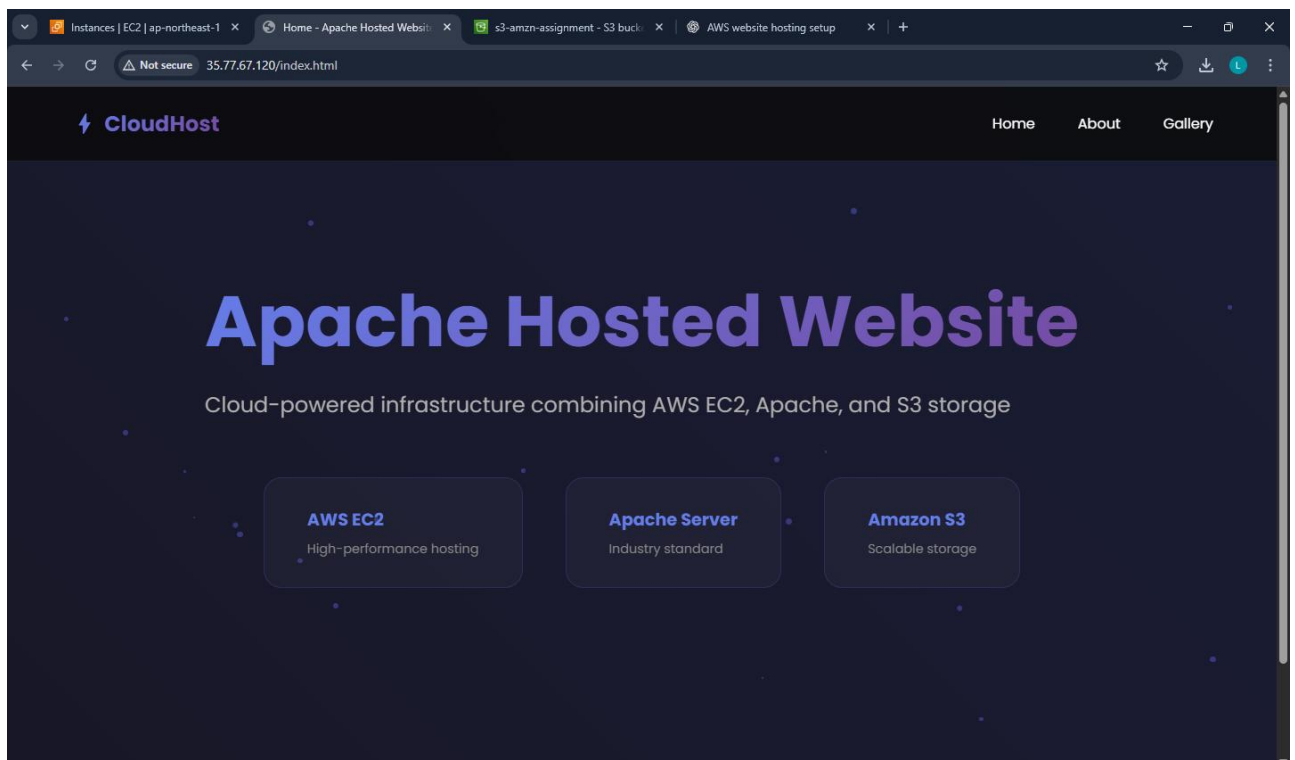
<footer>
  <p>© 2024 Apache Hosted Website | Powered by AWS Cloud Infrastructure</p>
</footer>

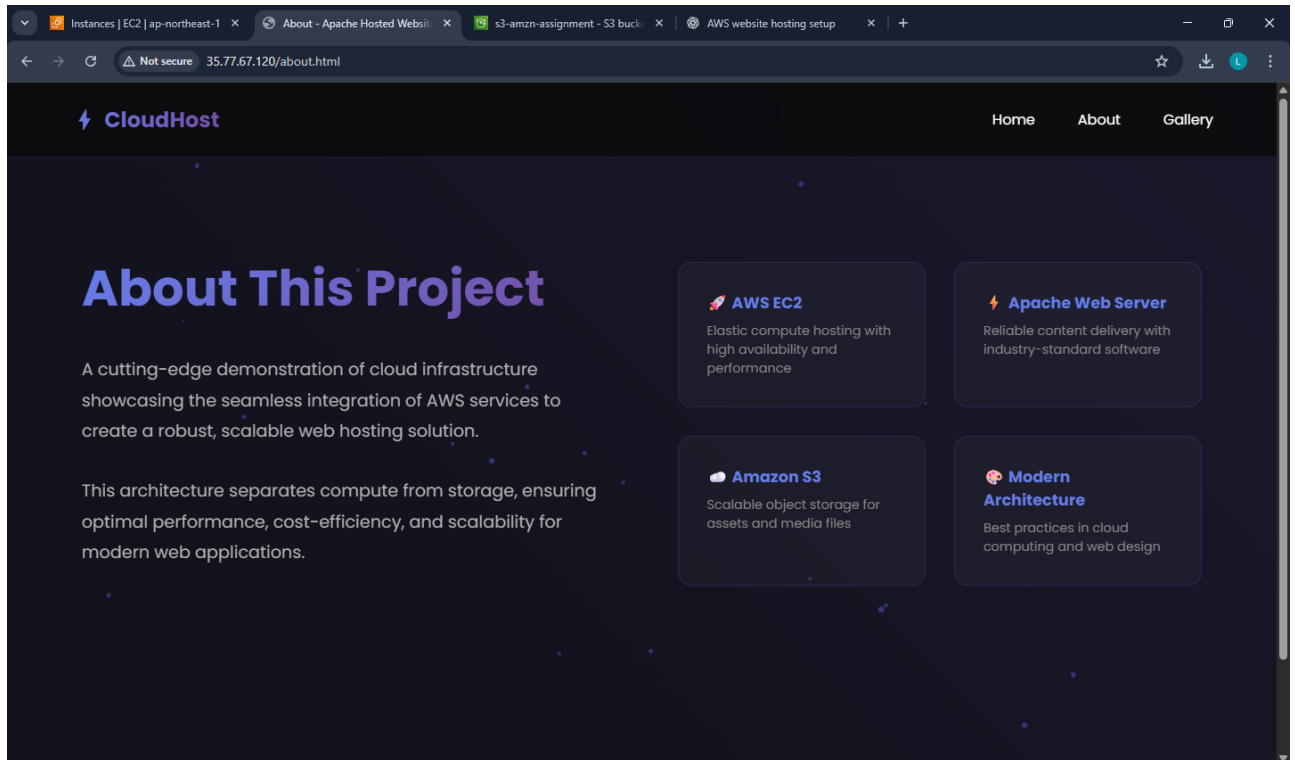
<script src="script.js"></script>
</body>
</html>

```

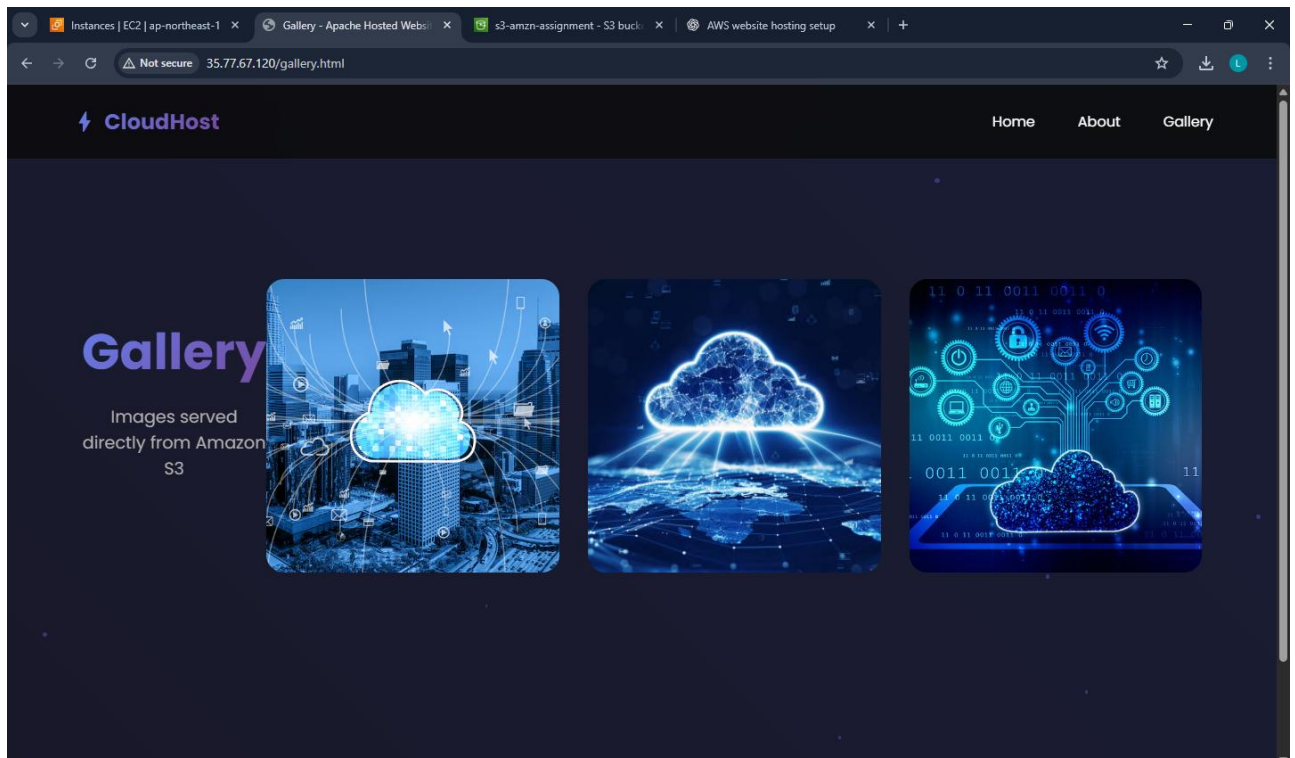
6. Access the Website.

- Once the instance state is “running”, copy the **Public IPv4 address**.
- Open a web browser and visit:
http://<PublicIPv4>
- The home page will display links to *Home*, *About*, *Gallery*.





- The *Gallery* page loads the images directly from the S3 bucket via HTTPS links.



7. Verify Hosting.

- Confirm that HTML pages are served by EC2 (nginx).
- Verify that all images load successfully from S3.

- If required, restart nginx using:
`sudo systemctl restart nginx`

Result:

The website was successfully hosted on AWS by using an **EC2 instance** for serving static HTML/CSS files and an **S3 bucket** for storing and displaying image assets. This experiment demonstrates the integration of multiple AWS services to deploy a simple and scalable web application.

Date: 12-11-2025

Exercise 9: Creation and Configuration of a Custom AWS Virtual Private Cloud (VPC)

[Including Public and Private Subnets, Internet Gateway, NAT Gateway, Route Tables]

Virtual Private Cloud [VPC]

It is a logically isolated section of the AWS Cloud where you can launch your AWS resources (like EC2 instances, databases, etc.) in a customized, secure network environment — similar to having your own private data center inside AWS.

It is a virtual network dedicated to your AWS account.

It gives you complete control over your networking environment, including IP address ranges, subnets, route tables, and network gateways.

Components of a VPC

Component	Description
CIDR Block (IP Range)	The range of IP addresses for your VPC (e.g., 10.0.0.0/16).
Subnets	Smaller divisions inside your VPC; can be Public (accessible from internet) or Private (internal only).
Internet Gateway (IGW)	Allows internet access for resources in public subnets.
Route Tables	Define how traffic is directed between subnets and gateways.
NAT Gateway / NAT Instance	Enables instances in private subnets to connect to the internet without being exposed.
Security Groups	Virtual firewalls that control inbound/outbound traffic at the instance level.
Network ACLs (Access Control Lists)	Additional firewall at the subnet level.
VPC Peering	Connects two VPCs so they can communicate privately.

Default VPC and Custom VPC

Characteristics of Default VPC

When you first create an AWS account, AWS automatically creates a default VPC for you in each region.

Feature	Description
Best For	Beginners, quick testing, learning environments, or temporary setups.
Created Automatically	One Default VPC per AWS Region (created by AWS).
Ready to Use	You can launch EC2 instances immediately — no setup needed.

CIDR Block	Always uses 172.31.0.0/16.
Subnets	One default subnet in each Availability Zone within the region.
Internet Connectivity	Each default subnet is a public subnet (has a route to Internet Gateway).
Route Table	Already configured to connect to the Internet Gateway.
Security Groups & NACLs	Default ones are automatically created and allow basic communication.

Characteristics of Custom VPC

A Custom VPC is created manually by the user to have complete control over the network configuration.

Feature	Description
Best For	Production environments, enterprise setups, or multi-tier architectures.
Created Manually	You define the VPC and its settings yourself.
CIDR Block	You can choose your own IP range (e.g., 10.0.0.0/16).
Subnets	You decide how many subnets, and whether they are public or private.
Internet Connectivity	You attach your own Internet Gateway.
Route Tables	Must be created and configured manually.
Security	You can create custom Security Groups and NACLs as needed.

Subnets

- Subdivisions inside a VPC, used to organize resources.
- Each subnet belongs to one Availability Zone.
- Two types:
 - **Public Subnet** → Connected to Internet Gateway; for web servers.
 - **Private Subnet** → No direct internet access; for databases or internal apps.
- CIDR examples:
 - Public: 10.0.1.0/24
 - Private: 10.0.2.0/24

Internet Gateway (IGW)

- A gateway that connects your VPC to the Internet.
- Required for instances in a public subnet to receive internet traffic.
- Must be attached to the VPC and referenced in the route table.
- Supports bi-directional communication (inbound and outbound).

NAT Gateway

- Enables outbound internet access for private subnet instances.
- Allows downloads and updates (e.g., OS patches) without exposing private IPs.
- Deployed inside a public subnet.
- Needs an Elastic IP for internet access.
- Traffic is one-way: private → internet only (not vice versa).

Route Tables

- Define rules (routes) that determine where network traffic goes.
- Each subnet must be associated with one route table.
- Common routes:
 - For public subnet → 0.0.0.0/0 → Internet Gateway
 - For private subnet → 0.0.0.0/0 → NAT Gateway
- Ensures proper separation between public and private networks.

Security Groups

- **Instance-level firewalls** controlling inbound and outbound traffic.
- Stateful: if inbound traffic is allowed, corresponding outbound is automatically allowed.
- Rules are based on protocol, port number, and source/destination.
- Example:
 - WebServer-SG: allows HTTP(80), HTTPS(443), SSH(22)
 - Database-SG: allows MySQL(3306) from WebServer-SG only

Network ACL (Access Control List)

- **Subnet-level firewall**, acts as an additional layer of security.
- **Stateless**: inbound and outbound rules must be defined separately.
- Default NACL allows all traffic; custom NACLs can be restrictive.
- Used for fine-grained control or compliance environments.

EC2 Instances

- Virtual machines running inside your subnets.
- Public subnet → hosts Web Server (Apache).
- Private subnet → hosts Database Server (MySQL).
- Public EC2s get a public IP; private ones use private IPs only.
- Controlled by their Security Groups and Route Tables.

Elastic IP (EIP)

- A **static, public IPv4 address** that can be attached to an EC2 or NAT Gateway.

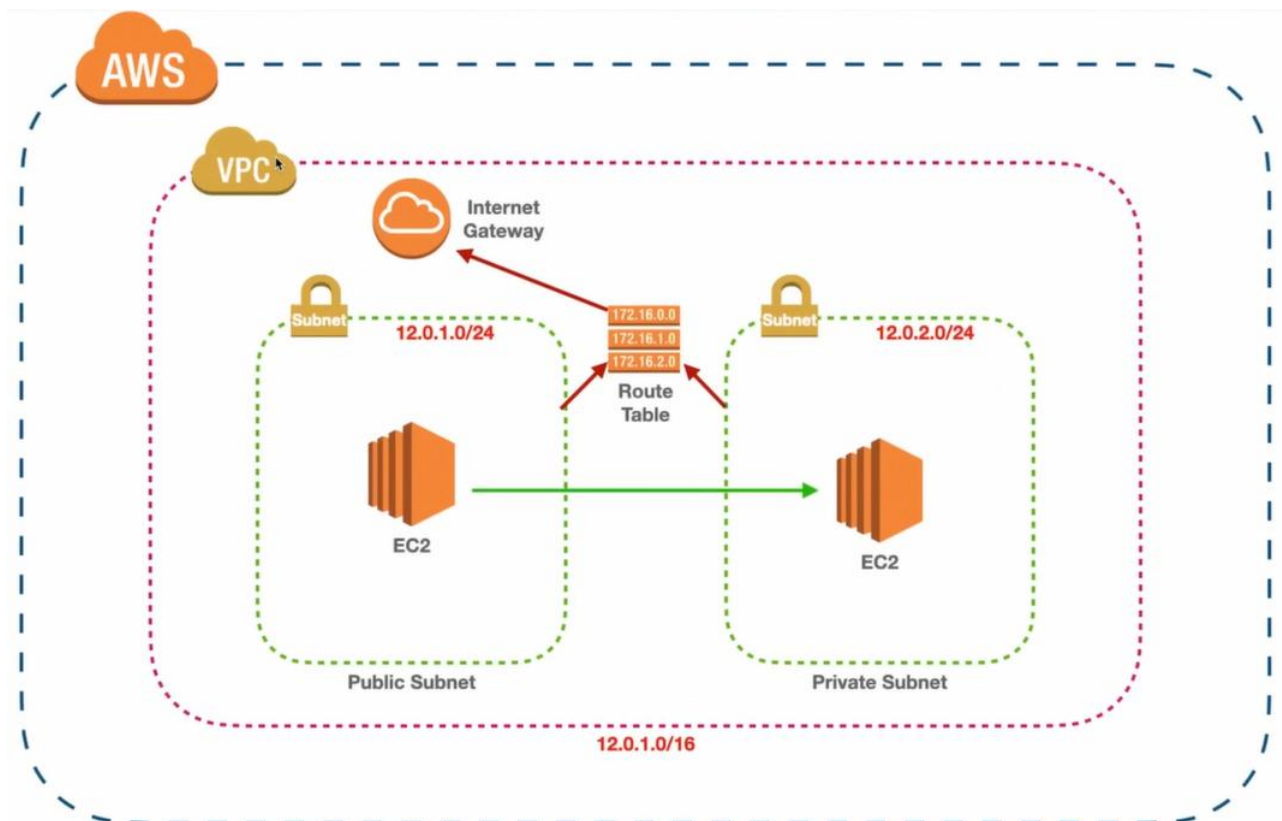
- Remains constant even if the instance is stopped or restarted.
- Useful for **NAT Gateways**, load balancers, or fixed server access.

Example: Two-Tier Architecture

Tier	Subnet	Function	Internet Access
Web Tier	Public Subnet	Hosts front-end web server	Yes (via IGW)
Database Tier	Private Subnet	Hosts back-end database	Outbound only (via NATGW)

Create a new VPC for a web application:

- One **Public Subnet** for web servers (via Internet Gateway)
- One **Private Subnet** for databases (via NAT Gateway)
- Custom route tables and tighter security rules.



Objective

To design and configure a **Virtual Private Cloud (VPC)** in AWS with:

- One **Public Subnet** for web servers (via **Internet Gateway**)
- One **Private Subnet** for databases (via **NAT Gateway**)
- Separate **Route Tables** for public and private subnets

Step 1 – Create a New VPC

- Open AWS Management Console → VPC.
- Click Create VPC.
- Choose VPC only option.
- Enter:
 - Name → MyWebAppVPC
 - IPv4 CIDR block → 10.0.0.0/16
 - Tenancy → Default
- Click Create VPC.

This allocates a private IP range for your virtual network.

Step 2 – Create Subnets

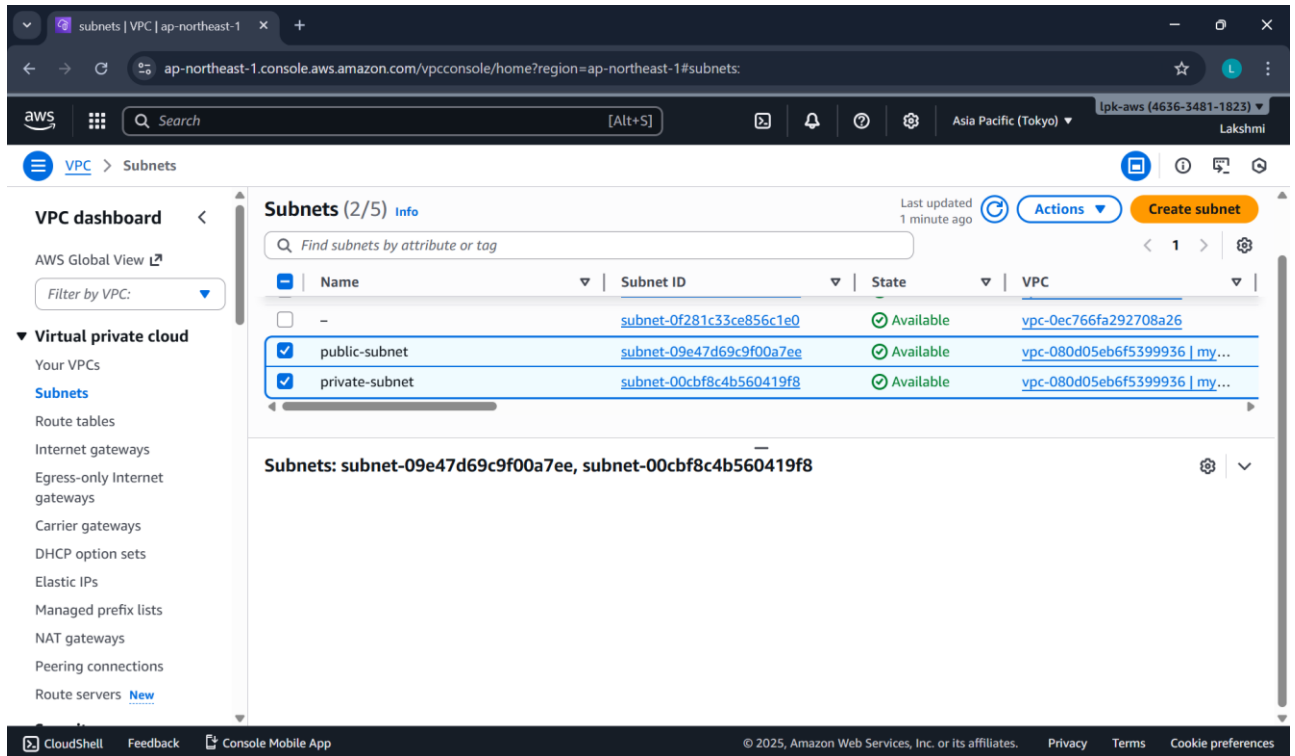
We'll create two subnets inside the VPC.

(a) Public Subnet

- Go to Subnets → Create subnet.
- Select VPC: MyWebAppVPC.
- Choose Availability Zone A.
- Enter:
 - Name → PublicSubnet
 - IPv4 CIDR block → 10.0.1.0/24
- Click Create subnet.

(b) Private Subnet

- Click Create subnet.
- Select same VPC.
- Choose Availability Zone B.
- Enter:
 - Name → PrivateSubnet
 - IPv4 CIDR block → 10.0.2.0/24
- Click Create subnet.



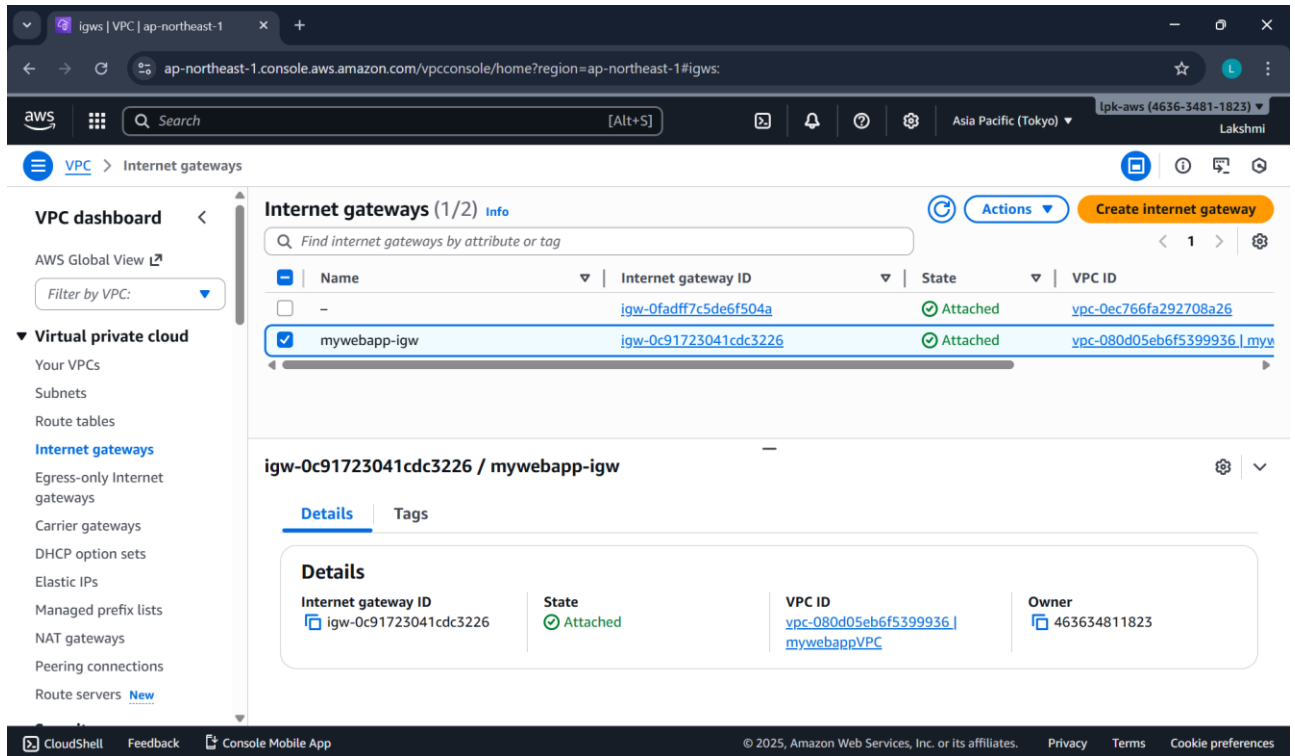
Step 3 – Create and Attach an Internet Gateway

- In the left navigation pane, scroll down and click on “Internet Gateways”.
- Internet Gateways → Create Internet Gateway.
 - Name: MyWebApp-IGW
- Click Create Internet Gateway

At this point, the IGW exists but is not yet connected to your VPC.

- **Attach the Internet Gateway to Your VPC**
 - Select the IGW you just created (checkbox).
 - Click on the “Actions” drop-down.
 - Choose “Attach to VPC.”
 - From the list, select your VPC name, e.g. MyWebAppVPC.
 - Click “Attach Internet Gateway.”

Now the IGW is linked to your VPC and the public subnet can reach the internet.



Step 4 – Configure Route Table for IGW

Public Route Table

- Go to Route Tables → Create route table.
 - Name → PublicRT
 - VPC → MyWebAppVPC
- Under Routes → Edit routes → Add route
 - Destination: 0.0.0.0/0
 - Target: Internet Gateway (MyWebApp-IGW)
- Under Subnet Associations → Edit subnet associations → Select PublicSubnet → Save.

Now the PublicSubnet has internet access.

Step 5 – Create a NAT Gateway

NOTE: NAT Gateway must always be created in a public subnet

- Go to NAT Gateways → Create NAT Gateway.
- Name → NAT-GW
- Choose:
 - Subnet → PublicSubnet
 - Elastic IP Allocation ID → Click Allocate Elastic IP
- Click Create NAT Gateway.

This allows instances in private subnet to access the internet (e.g., for updates) without being exposed.

The screenshot displays the AWS Management Console for the 'ap-northeast-1' region. The left sidebar shows the 'VPC dashboard' with a 'NAT gateways' link highlighted. The main content area shows a list of NAT gateways with one entry, 'natGW', which is in an 'Available' state. Below the list, the details for 'nat-07ee9ee4ed81a3c03 / natGW' are shown, including its ID, ARN, connectivity type (Public), and primary public and private IPv4 addresses.

Name	NAT gateway ID	Connectivity...	State	State message	Primary
natGW	nat-07ee9ee4ed81a3c03	Public	Available	-	13.231.95.185

nat-07ee9ee4ed81a3c03 / natGW			
Details			
NAT gateway ID nat-07ee9ee4ed81a3c03	Connectivity type Public	State Available	State message -
NAT gateway ARN arn:aws:ec2:ap-northeast-1:463634811823:natgateway/nat-07ee9ee4ed81a3c03	Primary public IPv4 address 13.231.95.185	Primary private IPv4 address 10.0.1.88	Primary network interface ID eni-0f741f16af39bde68
Subnet	Created	Deleted	

Note: NAT Gateway is a **paid resource**.

Step 6 – Configure Route Table for IGW

Private Route Table

- Create another Route Table → Name PrivateRT.
- Under Routes → Edit routes → Add route
 - Destination: 0.0.0.0/0
 - Target: NAT Gateway (MyWebApp-NATGW)
- Under Subnet Associations → Select PrivateSubnet → Save.

PrivateSubnet traffic goes through the NAT Gateway.

ap-northeast-1

ap-northeast-1.console.aws.amazon.com/vpconsole/home?region=ap-northeast-1#RouteTables:sort=tag:Name

Search [Alt+S]

Asia Pacific (Tokyo) lpk-aws (4636-3481-1823) Lakshmi

VPC > Route tables

VPC dashboard

AWS Global View

Filter by VPC:

Virtual private cloud

- Your VPCs
- Subnets
- Route tables**
- Internet gateways
- Egress-only Internet gateways
- Carrier gateways
- DHCP option sets
- Elastic IPs
- Managed prefix lists
- NAT gateways
- Peering connections
- Route servers [New](#)

Route tables (5) Info

Last updated less than a minute ago

Find route tables by attribute or tag

☐ Name ☐ Route table ID ☐ Explicit subnet associ... ☐ Edge associations ☐ Main

☐	-	rtb-0eb92eb038bb3ca8b	-	-	Yes
☐	-	rtb-02981954e3df99ca2	-	-	Yes
☐	privateRT	rtb-03812a6c51f7b2260	subnet-00cbf8c4b56041...	-	No
☐	publicRT	rtb-0900332f21eb9f1c6	subnet-09e47d69c9f00a...	-	No

Select a route table

CloudShell Feedback Console Mobile App

© 2025, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Note:

Each subnet in a VPC can be associated with **only one route table**.

If a subnet is not explicitly associated with a custom table, it will automatically use the **Main Route Table** created by default with the VPC.

DATE: 14-11-2025

EXERCISE-11

Launching and Configuring EC2 Instances for Web Server in Public Subnet and Database Server in Private Subnet Using NAT Gateway for Outbound Access

Step 1 — Launch Windows instance in Public Subnet

- Go to EC2 → Launch Instance.
- Choose:
 - AMI → Windows Server (Free tier eligible)
 - Instance type → t2.micro / t3.micro
- Select:
 - VPC → MyWebAppVPC
 - Subnet → PublicSubnet
 - Auto assign Public IP → Enable
- Security Group:
 - Create WebInstance-SG
 - Allow:
 - RDP (3389) → Anywhere (for practice)
- Launch instance using key pair (.pem).

Step 2 — Launch Windows instance in Private Subnet

- Click Launch Instance.
- Choose:
 - Windows Server AMI
- Select:
 - VPC → MyWebAppVPC
 - Subnet → PrivateSubnet
 - Auto assign Public IP → Disable
- Security Group:
 - Create DBInstance-SG
 - Allow RDP only from WebInstance-SG
- Launch instance.

The DBInstance is now **fully isolated** and cannot be accessed directly from the internet.

Step 3 — Connect to Public Windows Instance

1. Select WebInstance → Click Connect.
2. Choose RDP Client tab.
3. Click Get Password.
4. Upload your .pem key pair.
5. It shows the decrypted Windows Administrator password.
6. Copy the Public IP into a document for further use.

Connect using RDP

- Open Remote Desktop Connection (mstsc).
- In the dialog:
 - Computer → Public IP of WebInstance
- Click Connect.
- Credentials dialog:
- Username → Administrator
- Password → Paste decrypted password
- Click OK.

You are now inside the public Windows instance.

Step 4 — connect to private windows instance

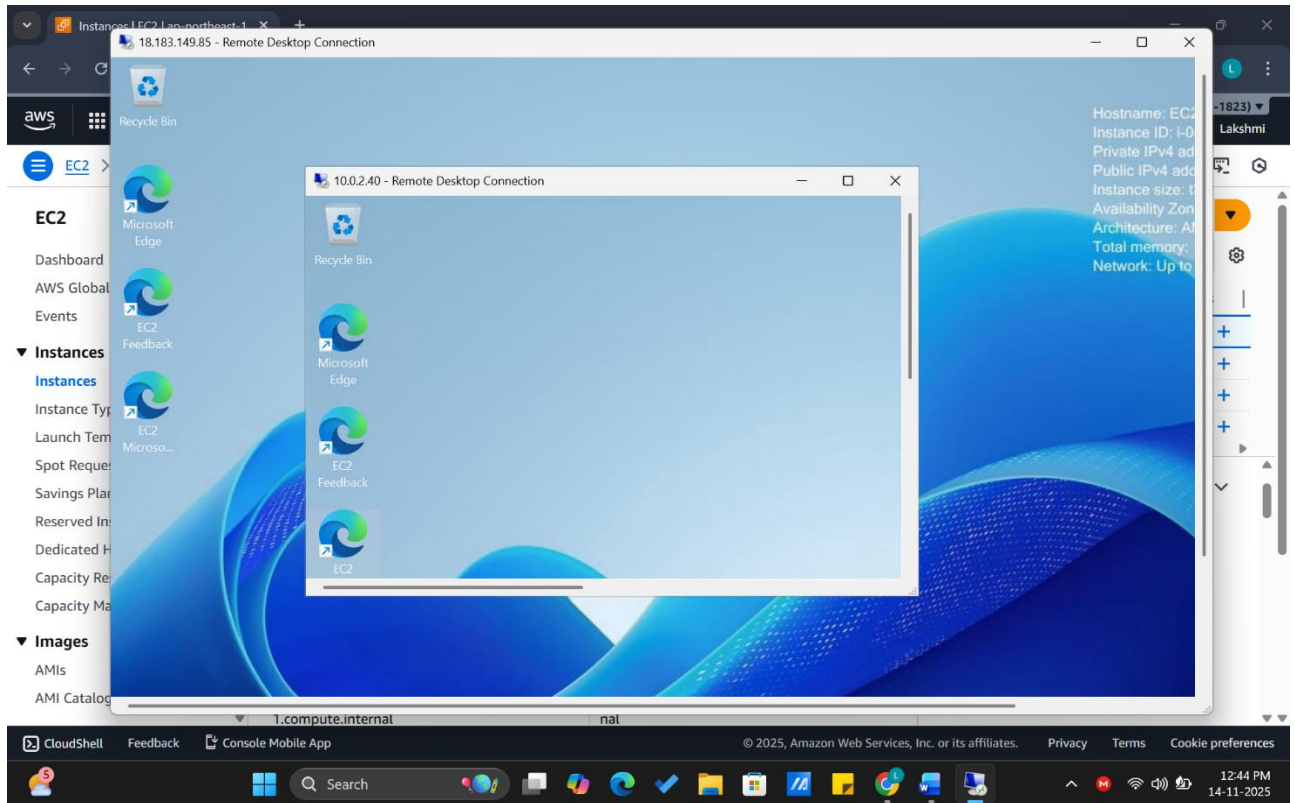
We cannot connect directly from your laptop → No Public IP.

We must connect **from within WebInstance** (jump server / bastion host).

Remote Login from WebInstance → DBInstance

- While logged into WebInstance, open Remote Desktop Connection. [**mstsc**]
- Enter:
 - Computer → Private IP of DBInstance
- Click Connect.
- Credentials:
 - Username → Administrator
 - Password → Paste the decrypted password (same key pair)
- Click OK.

You are now inside the private Windows instance.



Step 5 — testing NAT gateway connectivity (Verify Internet Access)

From WebInstance (Public Subnet)

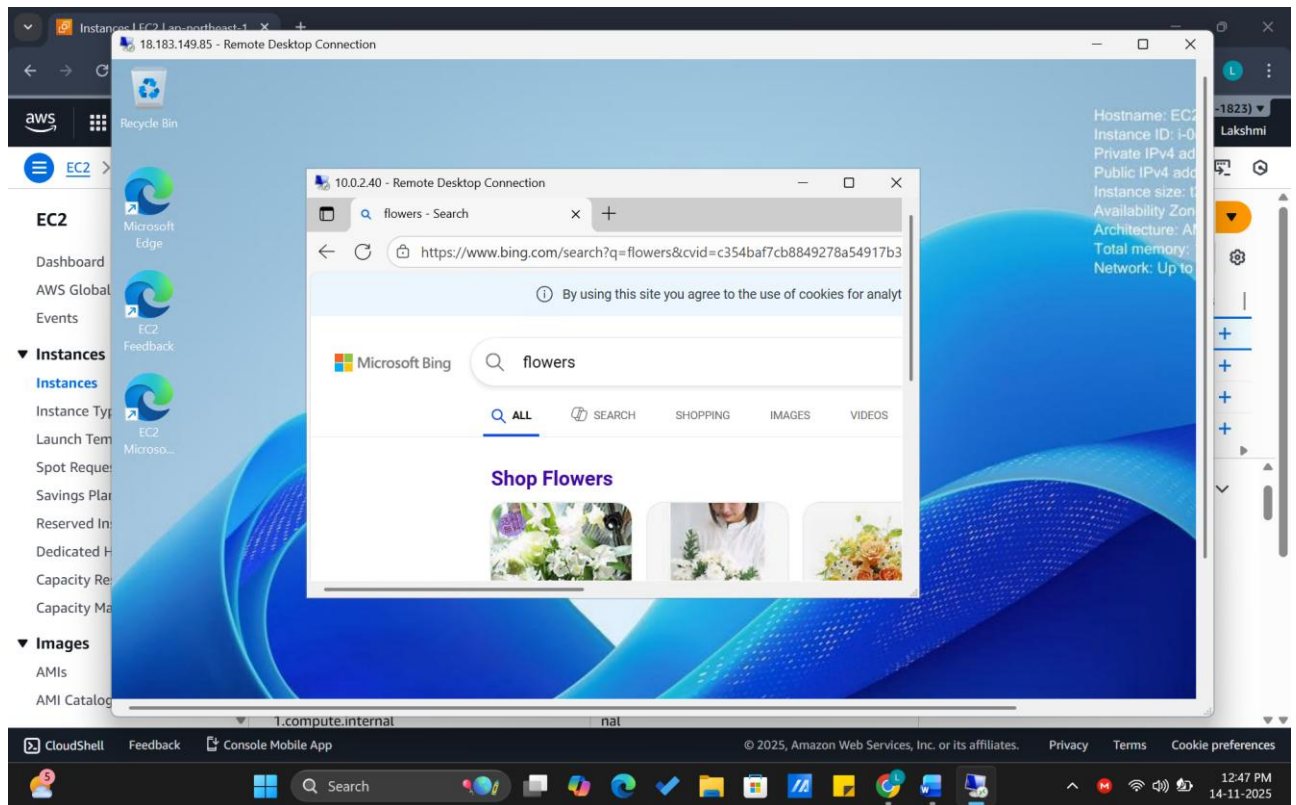
- Open a browser → Internet should work (through IGW).

From DBInstance (Private Subnet)

- Open browser → Internet should also work (through NAT Gateway).
- Internet icon will show "Internet Access".

Security Check

- DBInstance cannot receive inbound traffic from internet.
- Only outbound is allowed via NAT (safe for DB servers).



Test: From DBInstance (Private Subnet) → ping Google (Outbound Allowed)

Steps

Connect to DBInstance (using RDP from WebInstance).

Open Command Prompt inside DBInstance.

Type: ping google.com

Expected Result

- It will successfully ping google.com
- You will see replies like:
Reply from 142.250.xxx.xxx: bytes=32 time=20ms TTL=115

Why does this work?

- Outbound traffic is allowed from Private Subnet → NAT Gateway → Internet.
- NAT Gateway acts as a proxy for the private instance.
- So the private instance can reach internet,
but internet cannot reach the private instance.

What will NOT work?

From the Private DBInstance: ping <Your Own Public IP>
or anyone trying to ping: ping <DBInstance Private IP>

Both will fail because:

- Inbound traffic is blocked
- DBInstance has no Public IP
- SG allows inbound only from WebInstance-SG

Private instance → internet = YES (via NAT Gateway)

Internet → private instance = NO (fully blocked)

Because NAT Gateway is **one-way** only.

Elastic IP address

- A static IP address is a permanent address that doesn't change. You can manually configure a device to have a static IP address.
- An Elastic IP address is static and has to be used in a specific Region, it cannot be moved to a different Region.
- An Elastic IP address comes from Amazon's pool of IPv4 addresses.
- To use an Elastic IP address, you first allocate one to your account, and then associate it with your instance or a network interface.
- When you associate an Elastic IP address with an instance or its primary network interface, if the instance already has a public IPv4 address associated with it, that public IPv4 address is released back into Amazon's pool of public IPv4 addresses and the Elastic IP address is associated with the instance instead.
- You can disassociate an Elastic IP address from a resource, and then associate it with a different resource.
- A disassociated Elastic IP address remains allocated to your account until you explicitly release it.
- You are charged for all Elastic IP addresses in your account, regardless of whether they are associated or disassociated with an instance.
- Static IP addresses are especially important in cases where a device has to be quickly found over the internet on a permanent basis.
- Web Servers: A website must have one or more static IP addresses to be assigned to the domain always point to the correct server.

DATE: 20-11-25

Exercise—13 Stress Testing a Linux EC2 Instance (CPU Load Test)

Objective:

To generate high CPU load on an Amazon EC2 Linux instance using the stress-ng tool and observe CPU utilization in CloudWatch.

Step 1 — Launch a Linux EC2 Instance

1. Log in to the AWS Console.
2. Go to **EC2 → Instances → Launch Instance**.
3. Name: **StressTest-EC2**
4. AMI: **Amazon Linux 2023 (Free Tier eligible)**
5. Instance type: **t2.micro / t3.micro**
6. Key pair: Select or create a .pem key.
7. Security Group:
 - Allow **SSH (22)** from My IP.
 - Allow **HTTP (80)** from anywhere (optional).
8. Launch the instance.

Step 2 — Connect to the EC2 Instance

Use Windows PowerShell

Navigate to the folder where pem file is located and type the following command:

```
ssh -i "your-key.pem" ec2-user@<public-ip>
```

Step 3 — Install Apache (Optional, to verify the instance is working)

```
sudo yum install httpd -y
```

```
sudo systemctl start httpd
```

```
sudo systemctl enable httpd
```

Create a test web page:

```
echo "<h1>EC2 Stress Test Demo — $(hostname)</h1>" | sudo tee /var/www/html/index.html
```

STEP 4 — Install and Run the Stress Testing Tool (stress-ng)

(For Amazon Linux 2023 EC2 Instances)

Move to the ec2-user home directory

(It is recommended to run stress-ng from the home folder)

```
cd ~
```

Install stress-ng

Amazon Linux 2023 includes stress-ng directly through dnf, so install it using:

```
sudo dnf install stress-ng -y
```

Run a CPU Stress Test

Generate high CPU load using 4 CPU workers for 2 minutes:

```
stress-ng --cpu 4 --timeout 120
```

Expected Result

- Terminal becomes busy during the 120-second load test
- After completion, you will see:

successful run completed in 120.02s

- CPU usage will spike to 90–100% (you can check using top)

Step 5 — Run Stress Test (CPU Load Generation)

Move to home directory to avoid permission issues:

```
cd ~
```

Run CPU stress for 2 minutes using 4 CPU workers:

```
stress-ng --cpu 4 --timeout 120
```

You should see messages like:

```
stress-ng: info: dispatching hogs: 4 cpu
```

```
stress-ng: info: successful run completed in 120.02s
```

This will increase CPU usage to ~100% for the duration.

Step 6 — Observe CPU Utilization in CloudWatch

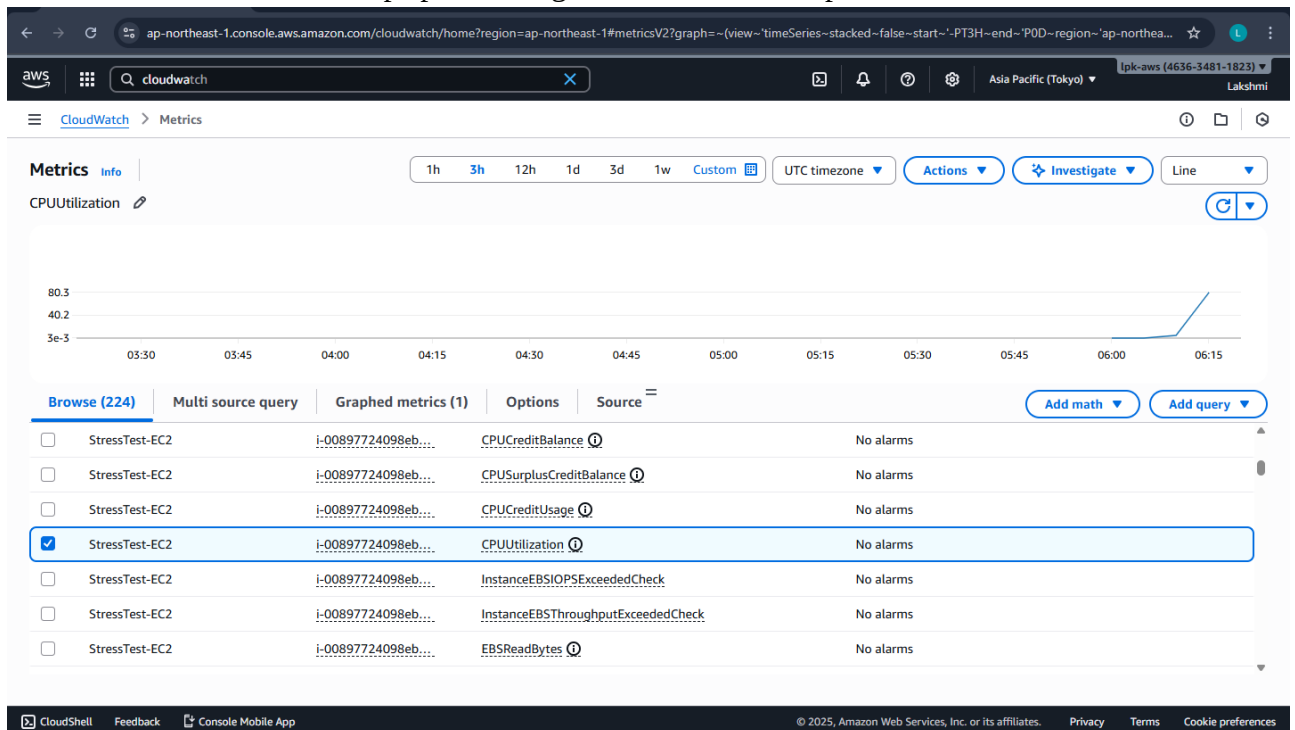
Go to CloudWatch → Metrics → EC2 → Per-Instance Metrics

Select:

CPUUtilization → InstanceId → your instance

View graph in 1-minute or 5-minute intervals.

You should see a sharp spike during the 2-minute stress period.



Step 7 — Stop or Terminate the Instance

To avoid charges:

Option A — Stop the instance (safe):

Actions → Instance State → Stop

Option B — Terminate (permanently delete):

Actions → Instance State → Terminate

Expected Output

- CPU Utilization in CloudWatch should reach 90–100%.
- Stress test completes without errors.
- Students understand how CPU load affects EC2 metrics.

DATE: 28-12-25

Exercise–18: Creating and Operating a NoSQL Key-Value Database using Amazon DynamoDB

To create an Amazon DynamoDB table and perform CRUD operations (Create, Read, Update, Delete) using the AWS Management Console, and understand Partition Key, Sort Key, Query vs Scan, and table cleanup.

Phase 1: Create DynamoDB Table

Create a DynamoDB table to store Student Records.

Store items like: USN, Name, Semester, Course, Attendance, IA1Marks

Step 1: Open DynamoDB

- Login to AWS Console
- Search → type DynamoDB → open Amazon DynamoDB

Step 2: Create a Table

- Left menu → Tables
- Click Create table
- Fill details:

Table details

- Table name: MCA_StudentLabInternals
- Partition key (PK): USN (Type: String)
- Sort key (SK): CourseCode (Type: String)
(Enable sort key option by setting sort key)

Why this design?

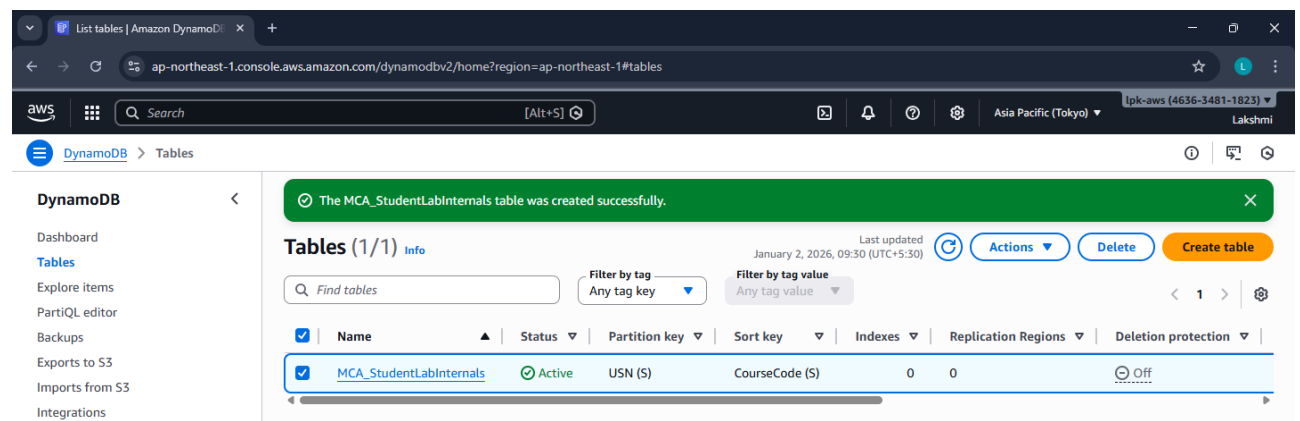
- One student can have multiple courses → Sort key separates records per course.
- This creates a composite primary key (PK + SK).

Step 3: Table settings

- Under Table settings, choose:
Default settings (recommended for lab)
- Ensure Capacity mode is: On-demand (default already selected)

Step 4: Create

- Click Create table
- Wait until table status becomes Active



Phase 2: Insert Items (Create Data)

Step 5: Open Table and Add Items

- Click the table: MCA_StudentLabInternals
- Click: Explore table items
- Click Create item

Step 6: Add Item 1 (Student 1 - Course 1)

- You will see attributes:
 - USN (String)
 - CourseCode (String)

Enter:

- USN = 1MS24MCA001
- CourseCode = CCL301

Now add additional attributes (click **Add new attribute**):

- Name (String) = Arun
- Semester (Number) = 3
- Attendance (Number) = 86
- IA1Marks (Number) = 18

Click **Create item**

Step 7: Add Item 2 (Same student - another course)

Repeat Create item with:

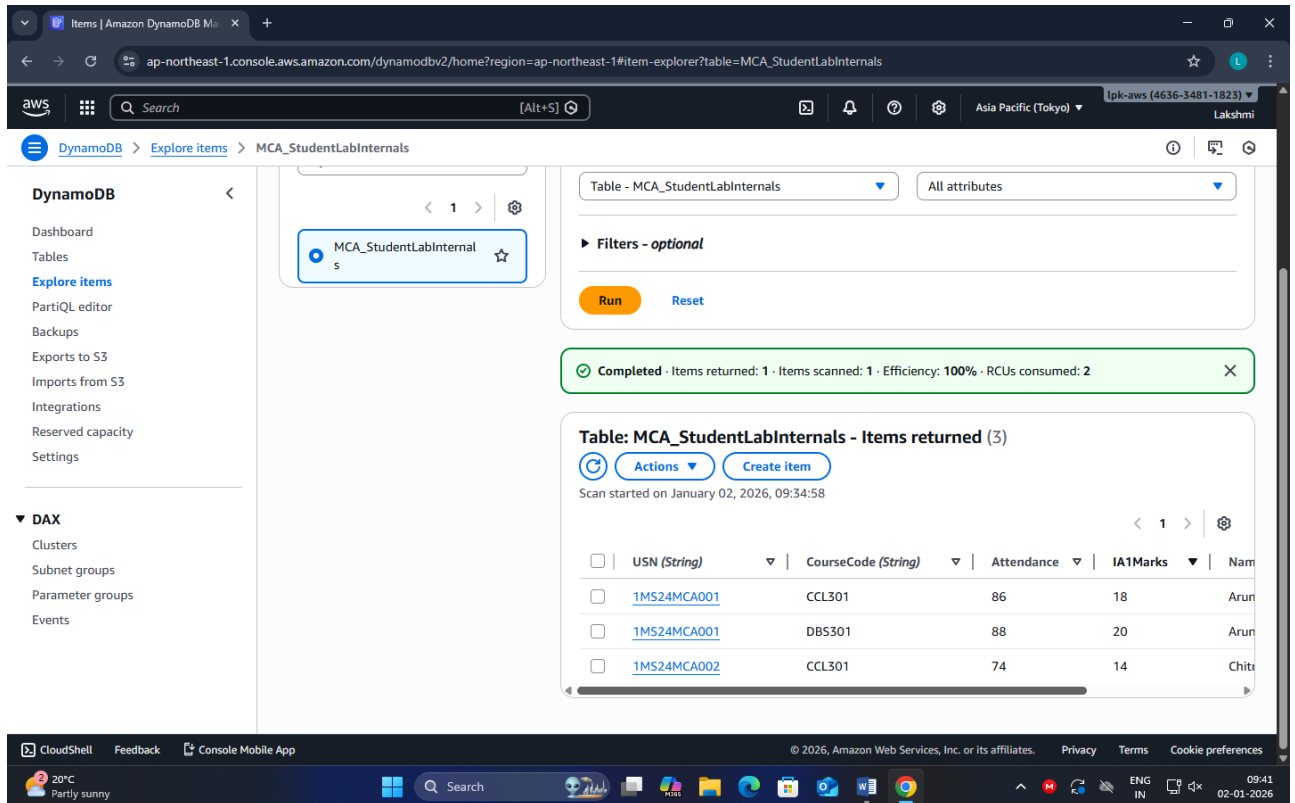
- USN = 1MS24MCA001
- CourseCode = DBS301
- Name = Arun
- Semester = 3
- Attendance = 88
- IA1Marks = 20

Step 8: Add Item 3 (Another student)

Create item:

- USN = 1MS24MCA002
- CourseCode = CCL301
- Name = Chitra
- Semester = 3
- Attendance = 74
- IA1Marks = 14

You should now see 3 items in the table list.



Phase 3: Read Data (Get / Query / Scan)

Step 9: Get a single item (exact PK + SK)

- In “Explore table items”, use search/filter:
- Use USN = 1MS24MC001 and CourseCode = CCL301
- Open the item → verify all attributes

Key concept:

To uniquely identify an item in a PK+SK table, you must provide **both**.

Step 10: Query – fetch all courses for one student

- Click Run query
- Set Partition key condition: USN equals 1MS24MCA001
- Run

Expected output:

- Items for Arun in both CCL301 and DBS301

Key concept:

Query works only with Partition Key (and optional Sort Key conditions). It is efficient.

Step 11: Scan (Not for large tables)

- Click Scan
- Run scan without filters

Expected output:

- All items (Arun + Chitra)

Key concept:

Scan reads the entire table → slow/costly for big tables.

The screenshot shows the Amazon DynamoDB console interface. On the left, there's a navigation menu with options like Dashboard, Tables, Explore Items, PartQL editor, Backups, Exports to S3, Imports from S3, Integrations, Reserved capacity, Settings, DAX, Clusters, Subnet groups, Parameter groups, and Events. The main area displays the 'MCA_StudentLabInternals' table. The 'Partition key' is 'USN' and the 'Sort key - optional' is 'CourseCode'. A query is executed with filters for 'USN' and 'CourseCode'. The results show two items returned, both for 'USN' '1MS24MCA001' but with different 'CourseCode' values: 'CCL301' and 'DBS301'. The 'Attendance' and 'IA1Marks' values are also displayed for each item.

USN (String)	CourseCode (String)	Attendance	IA1Marks	Name	Semester
1MS24MCA001	CCL301	86	18	Arun	3
1MS24MCA001	DBS301	88	20	Arun	3

Phase 4: Update Data

Step 12: Update IA1 marks of Chitra for CCL301

- Open item where:
USN = 1MS24MCA002
CourseCode = CCL301
- Click Edit
- Change: IA1Marks from 14 to 16
- Click Save changes

Verify updated value appears.

Phase 5: Delete Data

Step 13: Delete one item (Arun's DBS301 record)

- Select item:
 - USN = 1MS24MCA001
 - CourseCode = DBS301
- Click **Delete**
- Confirm delete

Now total items should reduce by 1.

The screenshot shows the AWS Management Console for DynamoDB. The left sidebar lists navigation options like Dashboard, Tables, Explore items, etc. The main content area shows the 'MCA_StudentLabInternals' table. A green notification bar at the top states 'Selected items have been deleted successfully.' Below this, the 'Scan or query items' section is active, showing a scan of the table. A green bar indicates the scan is 'Completed' with 2 items returned. Below this, a table titled 'Table: MCA_StudentLabInternals - Items returned (1)' shows the scan results. The table has columns for USN (String), CourseCode (String), Attendance, IA1Marks, Name, and Semester.

Phase 6: Cleanup (Must Do to avoid charges)

Step 15: Delete the Table

- DynamoDB → Tables
- Select MCA_StudentLabInternals
- Click Delete
- Type confirmation (if asked) and delete

Ensure table disappears from list.

The screenshot shows the AWS Management Console for DynamoDB, specifically the 'Tables' page. A green notification bar at the top states 'The request to delete the "MCA_StudentLabInternals" table has been submitted successfully.' Below this, the 'Tables (1/1)' list shows the 'MCA_StudentLabInternals' table with a status of 'Deleting'. The table's details, including its name, status, partition key, sort key, indexes, replication regions, and deletion protection, are visible.

DATE: 03-12-25

Exercise–19: ElastiCache (Redis) as an In-Memory Cache

To implement Amazon ElastiCache (Redis) as an in-memory caching service by deploying a Redis cluster and performing basic cache operations from an EC2 instance.

Application-Level Data Flow (Logical)

Step 1: User requests data

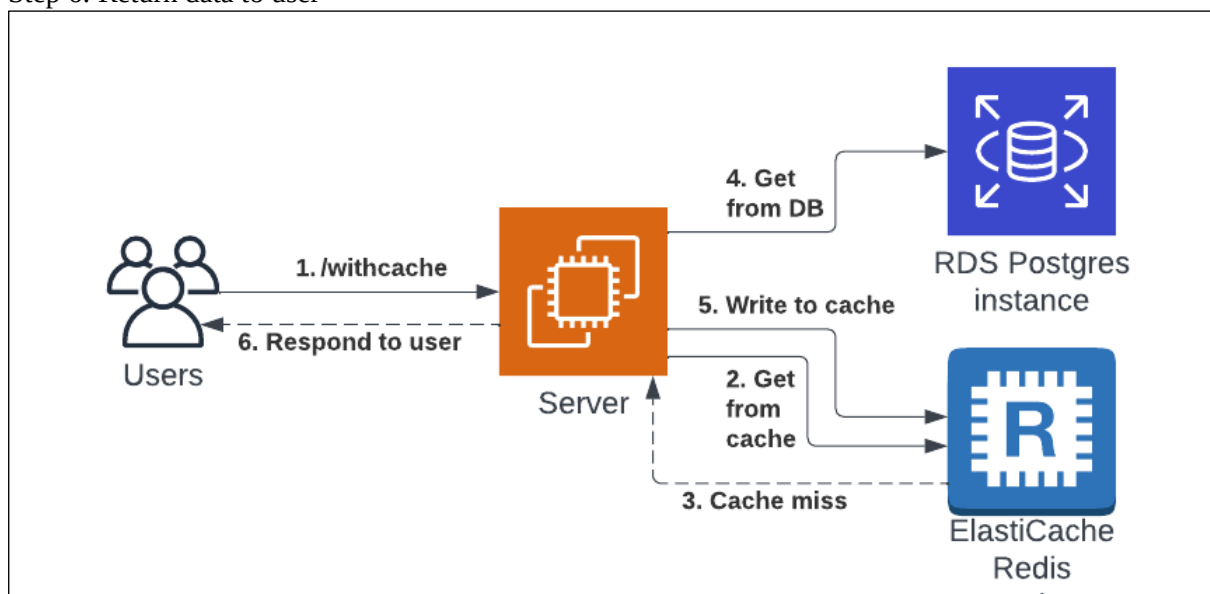
Step 2: Application checks ElastiCache (Redis)

Step 3: If data exists → return from cache (FAST)

Step 4: If data does not exist → fetch from RDS

Step 5: Store fetched data in ElastiCache

Step 6: Return data to user



This exercise provides **exposure** to an industry-used in-memory data store.

Redis improves performance by serving frequently accessed data from memory and is typically used **in front of databases** in real-world applications.

Feature	RDS	ElastiCache
Storage	Disk	RAM
Data lifetime	Permanent	Temporary
Access speed	Slower	Very fast
TTL support	No	Yes

Phase-1 → EC2 creation + valkey-cli installation

Phase-2 → Redis cache creation

Phase-3 → Connect EC2 → Redis engine and run commands

Phase-1: Launch EC2 Client using Amazon Linux 2023 (for Valkey / Redis Client)

Purpose of Phase-1

To create an EC2 instance using **Amazon Linux 2023** that will act as a **client machine** to connect to Amazon ElastiCache (Redis OSS) using **valkey-cli**.

Step 1: Select AWS Region

- Choose ONE region and stick to it for the entire exercise
ap-south-1 (Mumbai) (*recommended*)
- Ensure ElastiCache and EC2 will be in the SAME region

Step 2: Launch EC2 Instance

1. Go to **AWS Console** → **EC2**
2. Click **Launch instance**

Step 3: Configure Instance Basics

- Name: RedisClient-AL2023
- AMI: Select Amazon Linux 2023 AMI
- Instance Type: Select t3.micro (Free Tier eligible)
- Key Pair: Select an existing key pair OR create a new one (RSA, .pem)
- Network & Security
 - Network
 - VPC: Default VPC
 - Subnet: No preference
 - Auto-assign public IP: Enabled
 - Security Group (VERY IMPORTANT)
Create a new security group:
 - Security Group Name: SG-RedisClient
 - Description: Security group for Redis client EC2

Inbound Rules

Type	Port	Source
SSH	22	Anywhere 0.0.0.0/0

Do NOT add Redis (6379) here (The client does not listen on 6379)
Outbound rules: Allow all (default)

Step 5: Storage

- Keep default

Step 6: Launch Instance

- Click Launch instance
- Wait until Instance State = Running

Step 7: Connect to EC2

1. EC2 → Instances → select RedisClient-AL2023
2. Click Connect
3. Choose EC2 Instance Connect
4. Click Connect

You are now inside the EC2 terminal.

Step 8: Install Valkey Client

Run the following commands:

```
sudo dnf update -y
```

```
sudo dnf install -y valkey
```

Verify installation:

```
valkey-cli --version
```

Expected Output (example)

```
valkey-cli 8.x.x
```

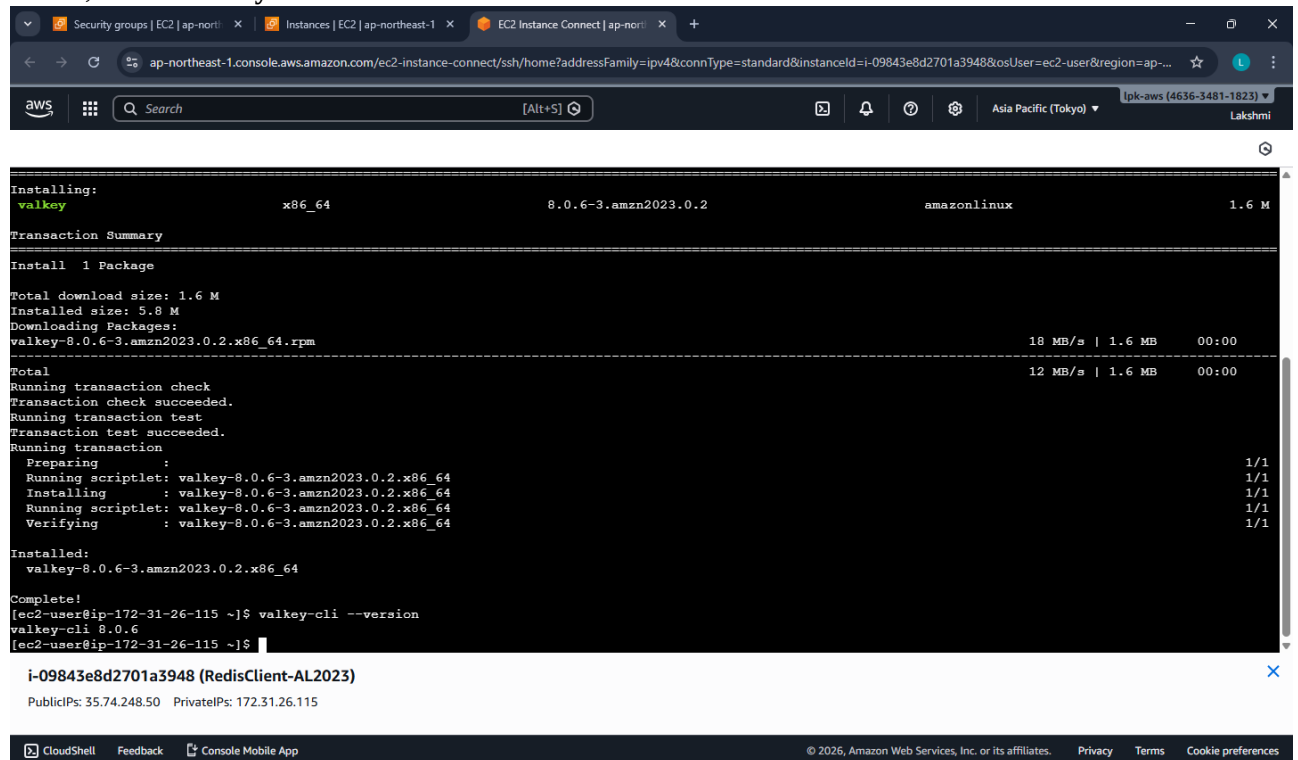
This confirms the EC2 is ready as a Redis-compatible client.

Note: why install valkey and not redis??

Amazon Linux 2023 does not provide redis-cli directly.

AWS now provides **Valkey**, which is fully Redis-protocol compatible.

Hence, we use **valkey-cli** to connect to ElastiCache Redis.



The screenshot shows the AWS Management Console interface for an EC2 instance. The top navigation bar includes tabs for 'Security groups', 'Instances', and 'EC2 Instance Connect'. The main content area displays the 'EC2 Instance Connect' terminal window for an instance named 'ap-northeast-1'. The terminal output shows the installation of 'valkey' on an Amazon Linux 2023 instance. The installation process includes downloading the package, running a transaction check, and successfully installing 'valkey-8.0.6-3.amzn2023.0.2.x86_64.rpm'. The final output shows 'valkey-cli 8.0.6'.

```
Installing:
valkey                                     x86_64                               8.0.6-3.amzn2023.0.2               amazonlinux                        1.6 M

Transaction Summary
-----
Install 1 Package

Total download size: 1.6 M
Installed size: 5.8 M
Downloading Packages:
valkey-8.0.6-3.amzn2023.0.2.x86_64.rpm                                18 MB/s | 1.6 MB  00:00
-----
Total                                                                12 MB/s | 1.6 MB  00:00
Running transaction check
Transaction check succeeded.
Running transaction test
Transaction test succeeded.
Running transaction
  Preparing                : valkey-8.0.6-3.amzn2023.0.2.x86_64                1/1
  Running scriptlet: valkey-8.0.6-3.amzn2023.0.2.x86_64                1/1
  Installing           : valkey-8.0.6-3.amzn2023.0.2.x86_64                1/1
  Running scriptlet: valkey-8.0.6-3.amzn2023.0.2.x86_64                1/1
  Verifying            : valkey-8.0.6-3.amzn2023.0.2.x86_64                1/1

Installed:
valkey-8.0.6-3.amzn2023.0.2.x86_64

Complete!
[ec2-user@ip-172-31-26-115 ~]$ valkey-cli --version
valkey-cli 8.0.6
[ec2-user@ip-172-31-26-115 ~]$
```

i-09843e8d2701a3948 (RedisClient-AL2023)
PublicIPs: 35.74.248.50 PrivateIPs: 172.31.26.115

Phase-2: Create Amazon ElastiCache (Redis OSS) Cluster

Purpose of Phase-2

To create an Amazon ElastiCache Redis OSS cluster that will act as an in-memory cache, accessible from the EC2 client created in Phase-1.

Step 1: Confirm AWS Region

- Ensure you are in the **same region** used in Phase-1
EC2 and ElastiCache must be in the SAME region and VPC.

Step 2: Open ElastiCache Console

1. AWS Console → **Services**

2. Select ElastiCache
3. Click Create cache

Step 3: Select Cache Engine

- **Engine: Redis OSS**

Step 4: Choose Deployment Settings

- Deployment option: Node-based cluster
- Creation method: Easy create

Easy create is used to avoid advanced production settings.

Step 5: Select Configuration

- **Configuration: Demo**

This automatically selects:

- A small, low-cost node (e.g., cache.t4g.micro)
- Suitable for labs and practice

Step 6: Provide Cluster Information

- Cache name: lab-redis
- Description: Optional (lab-redis)

Step 7: Configure Network and Subnet Group

Network

- Network type: IPv4

Subnet Group

- Select: Create a new subnet group
 - Subnet group name: redis-subnet-group
 - VPC: Default VPC
 - Subnets: Leave AWS auto-selected subnets unchanged

No manual subnet selection required.

Step 8: Configure Security

Security Group

- Create or select a security group:
 - Name: SG-RedisCache

Inbound Rule for Redis

Add **one inbound rule** to SG-RedisCache:

Type	Port	Source
Custom TCP	6379	SG-RedisClient

This allows only the EC2 client to access Redis.

Step 9: Authentication

- Authentication: Disabled

Step 10: Create Cache

1. Review all settings
2. Click Create

3. Wait until Status = Available

This may take a few minutes.

Step 11: Note the Redis Endpoint

1. Click on the cache name lab-redis
2. Copy the **Configuration Endpoint**
 - Example: lab-redis.xxxxxxx.cache.amazonaws.com

This endpoint will be used in **Phase-3** to connect from EC2.

Note:

We have created an in-memory Redis cache using Amazon ElastiCache.

It runs inside the AWS VPC and does not have a public IP.

Only our EC2 client is allowed to access it using port 6379.

The screenshot shows the AWS Management Console interface for an ElastiCache Redis cluster named 'lab-redis'. The breadcrumb navigation at the top indicates the path: **ElastiCache** > **Redis OSS caches** > **lab-redis**. The cluster status is 'Available'. The 'Cluster details' section provides the following information:

Cluster name	Description	Node type	Status
lab-redis	Easy created demo cluster on 2026-01-02T04:22:35.397Z	cache.t4g.micro	Available

Engine	Engine version	Global datastore	Global datastore role
Redis	7.1.0	-	-

Update status	Cluster mode	Shards	Number of nodes
Up to date	Enabled	1	1

Data tiering	Multi-AZ	Auto-failover	Encryption in transit
Disabled	Disabled	Enabled	Enabled

Encryption at rest	Parameter group	Outpost ARN	Transit encryption mode
Enabled	default.redis7.cluster.on	-	Required

Configuration endpoint	Primary endpoint	Reader endpoint	ARN
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379	-	-	arn:aws:elasticache:ap-northeast-1:463634811823:replicationgroup:lab-redis

At the bottom, the 'Data migration' status is 'No active migrations'. The console also shows tabs for 'Connectivity and security - new', 'Shards and nodes', 'Metrics', 'Logs', 'Maintenance and backups', 'Service updates', and 'Tags'.

Phase-3: Connect EC2 to ElastiCache Redis and Execute Cache Commands

Purpose of Phase-3

To connect the EC2 client (Amazon Linux 2023) to the ElastiCache Redis OSS cluster using valkey-cli and demonstrate basic in-memory cache operations.

Pre-checks - Before connecting, confirm:

- EC2 and ElastiCache are in the same AWS region
- EC2 security group = SG-RedisClient
- Redis security group = SG-RedisCache
- Redis security group allows port 6379 from SG-RedisClient
- Redis Status = Available
- You have copied the Primary Endpoint

Step 1: Connect to EC2

1. AWS Console → **EC2**
2. Select instance RedisClient-AL2023
3. Click **Connect**
4. Choose **EC2 Instance Connect**
5. Click **Connect**

You are now inside the EC2 terminal.

Step 2: Verify Valkey Client

Run: `valkey-cli --version`

Expected Output (example)

`valkey-cli 8.x.x`

This confirms the EC2 is ready to act as a Redis-compatible client.

Step 3: Connect to ElastiCache Redis

Use the Configuration Endpoint from Phase-2.

`valkey-cli -h <Configuration_ENDPOINT> -p 6379`

Example: `valkey-cli -h lab-redis.xxxxxx.cache.amazonaws.com -p 6379 --tls -c`

Expected Result

You should see a prompt like:

`lab-redis.xxxxxx.cache.amazonaws.com:6379>`

This means the connection is successful.

Step 4: Execute Redis / Valkey Commands

These commands simulate what an application does internally.

Test Connectivity

PING

Expected Output

PONG

Store and Retrieve Data (SET / GET)

`SET course "Cloud Computing"`

`GET course`

Expected Output

"Cloud Computing"

Demonstrates **key-value storage in memory**.

Counter Example (INCR)

`INCR visits`

`INCR visits`

`GET visits`

Expected Output

"2"

NOTE: Common real-world use - page views, hit counters.

4.4 Set Data with Expiry (TTL)

`SET notice "Results Published" EX 60`

TTL notice

GET notice

Expected Output

- TTL shows a value ≤ 60
- GET returns:

"Results Published"

Shows **temporary cache data**.

4.5 Verify Automatic Expiry

Wait ~60 seconds, then run:

GET notice

Expected Output

(nil)

Confirms data is **automatically removed from memory**.

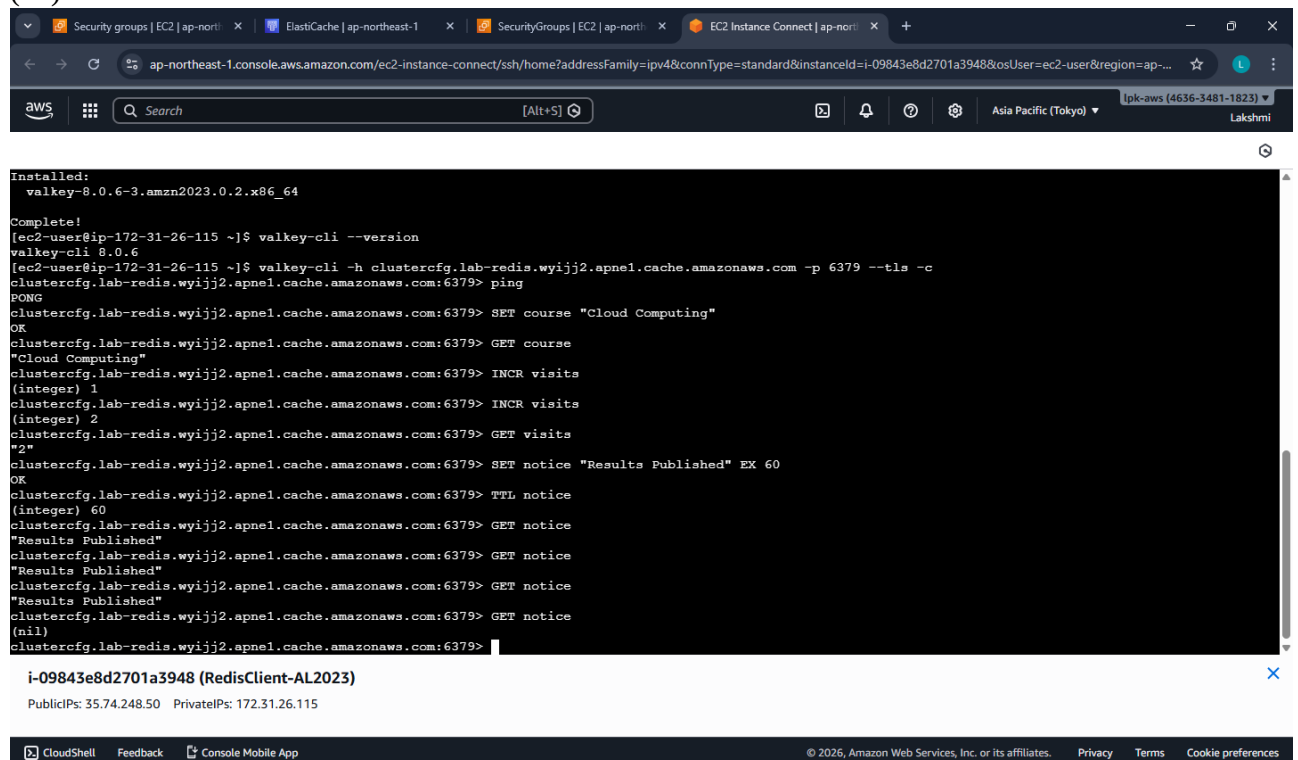
4.6 Delete Data Manually

DEL course

GET course

Expected Output

(nil)



```
Installed:
valkey-8.0.6-3.amzn2023.0.2.x86_64

Complete!
[ec2-user@ip-172-31-26-115 ~]$ valkey-cli --version
valkey-cli 8.0.6
[ec2-user@ip-172-31-26-115 ~]$ valkey-cli -h clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com -p 6379 --tls -c
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> ping
PONG
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> SET course "Cloud Computing"
OK
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> GET course
"Cloud Computing"
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> INCR visits
(integer) 1
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> INCR visits
(integer) 2
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> GET visits
"2"
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> SET notice "Results Published" EX 60
OK
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> TTL notice
(integer) 60
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> GET notice
"Results Published"
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> GET notice
"Results Published"
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> GET notice
"Results Published"
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379> GET notice
(nil)
clustercfg.lab-redis.wyijj2.apne1.cache.amazonaws.com:6379>
```

i-09843e8d2701a3948 (RedisClient-AL2023)

PublicIPs: 35.74.248.50 PrivateIPs: 172.31.26.115

CloudShell Feedback Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

Step 5: Exit Redis Client

EXIT

Note:

“The application first checks Redis.

If data is present, it is returned immediately from memory.

If not present, the application fetches data from the database and stores it in Redis with a TTL.

Redis automatically removes the data after expiry.”

Phase-3 Outcome

- EC2 successfully connected to ElastiCache Redis
- In-memory key-value operations verified
- Temporary storage and TTL behavior observed
- Redis used as a **cache**, not as a primary database

Common Errors & Quick Fix

Problem	Reason	Fix
Connection timeout	Wrong SG or region	Check SG-RedisCache inbound rule
Command hangs	Redis not Available	Wait & retry
(nil) output	Key expired	Expected behavior

Clean-Up (Mandatory)

Step 1: Delete Redis Cluster

- ElastiCache → Select lab-redis → Delete
- Disable snapshots

Step 2: Terminate EC2 Instance

- EC2 → Instances → Terminate RedisClient-EC2

Step 3: Optional

- Delete unused security groups

Redis Commands Explanation

- **SET** course "Cloud Computing"
- **GET** course
- **EXPIRE** course 30
- **TTL** course
- The above commands simulate application caching behavior.
The SET command represents storing frequently accessed data in cache.
The EXPIRE command shows that cached data is temporary and stored in memory.
After expiry, the application would fetch fresh data from the database again.

DATE: 12-12-25

Exercise–22: CloudFormation – Launch EC2 (Amazon Linux 2023) + Install Apache using UserData

Create an EC2 instance using AWS CloudFormation (YAML template) and automatically install Apache web server (httpd) using UserData, then verify the website from a browser.

Pre-requisites

- Region set to **Asia Pacific (Mumbai) – ap-south-1**
- Basic knowledge of EC2 and Security Groups

Part A — Create Key Pair (One-time setup)

Step A1: Open EC2 Key Pairs

1. AWS Console → Search **EC2**
2. Left menu → **Key Pairs** (under “Network & Security”)
3. Click **Create key pair**

Step A2: Create key pair

- Name: pemkeypair (any name is fine)
- Key pair type: **RSA**
- Private key file format:
 - **.pem** (recommended)

Click **Create key pair**

A file will download like: pemkeypair.pem

Note: CloudFormation uses **key pair NAME** (pemkeypair), not the file name.

Part B — Create CloudFormation Template File (Amazon Linux 2023 + Apache)

Step B1: Create YAML file on your computer

1. Open **Notepad**
2. Paste the full YAML template given below
3. Save as: **ec2-apache-al2023.yaml**
 - Save type: **All files**

- Encoding: UTF-8 (if asked)

Full CloudFormation Template (Amazon Linux 2023)

Important:

- Do **not** add .pem anywhere.
- You will select Key Pair from dropdown during stack creation.

AWSTemplateFormatVersion: "2010-09-09"

Description: Launch EC2 (Amazon Linux 2023) and install Apache (httpd) using UserData

Parameters:

KeyName:

Type: AWS::EC2::KeyPair::KeyName

Description: Select an existing EC2 Key Pair to enable SSH access

Resources:

WebServerSG:

Type: AWS::EC2::SecurityGroup

Properties:

GroupDescription: Allow SSH (22) and HTTP (80)

SecurityGroupIngress:

- IpProtocol: tcp

FromPort: 22

ToPort: 22

CidrIp: 0.0.0.0/0

- IpProtocol: tcp

FromPort: 80

ToPort: 80

CidrIp: 0.0.0.0/0

WebServerInstance:

Type: AWS::EC2::Instance

Properties:

InstanceType: t3.micro

KeyName: !Ref KeyName

SecurityGroups:

- !Ref WebServerSG

Amazon Linux 2023 AMI for Mumbai (ap-south-1)

ImageId: ami-02b49a24cfb95941c

UserData:

Fn::Base64: |

#!/bin/bash

dnf update -y

dnf install -y httpd

systemctl enable httpd

systemctl start httpd

echo "<h1>Apache Installed via CloudFormation UserData (Amazon Linux 2023)</h1>" >
/var/www/html/index.html

Outputs:

InstanceId:

Description: EC2 Instance ID

Value: !Ref WebServerInstance

WebsiteURL:

Description: Apache Website URL

Value: !Sub "http://\${WebServerInstance.PublicDnsName}"

Step C1: Open CloudFormation

1. AWS Console → Search **CloudFormation**
2. Click **Stacks**
3. Click **Create stack** → **With new resources (standard)**

Step C2: Prepare template (select correct options)

1. Under **Prepare template**:
Select **Choose an existing template**
 2. Under **Template source**:
Select **Upload a template file**
 3. Click **Choose file** → select ec2-apache-al2023.yaml
 4. Click **Next**
-

Part D — Specify Stack Details

Step D1: Stack name

- Stack name: EC2-Apache-AL2023

Click **Next**

Step D2: Parameters

- Under **KeyName**, select your key pair name from dropdown
Example: pemkeypair

Click **Next**

Part E — Configure Stack Options (keep default)

1. Leave everything as default
 2. Click **Next**
-

Part F — Review and Create

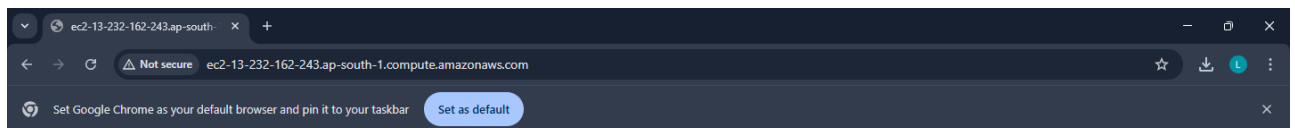
1. Scroll down
 2. Click **Create stack**
-

Part G — Monitor Stack Creation

1. Wait for Stack status to become:
CREATE_COMPLETE
2. Open the stack → click **Outputs** tab
3. Copy **WebsiteURL**
4. Paste URL in browser

Expected Output in browser:

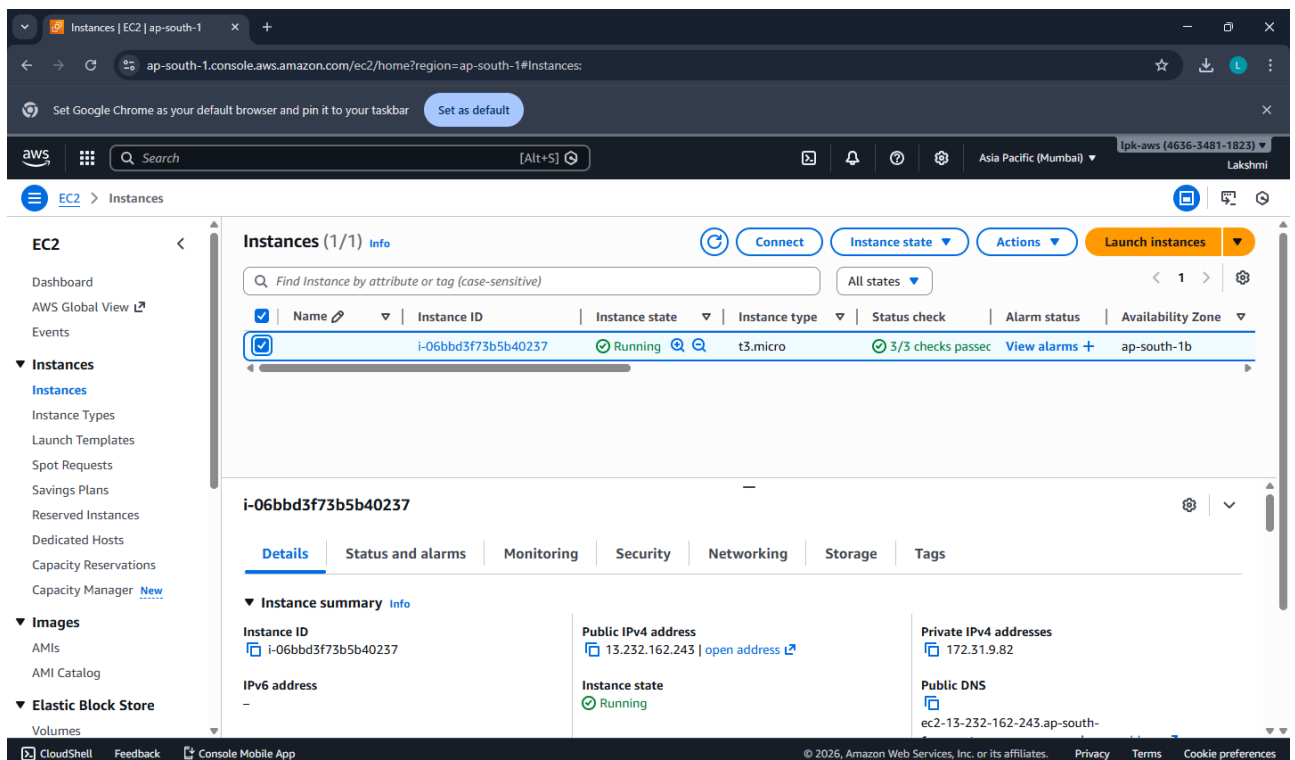
Apache Installed via CloudFormation UserData (Amazon Linux 2023)!



Apache Installed via CloudFormation UserData (Amazon Linux 2023)!

Part H — Verify in EC2 (Optional)

1. Go to EC2 → **Instances**
2. Instance should be running
3. Security group should allow:
 - SSH 22
 - HTTP 80



Troubleshooting (Common Errors)

1) Website not opening

Check:

- EC2 instance status checks are **2/2 passed**
- Security group has **port 80 open**
- Wait 2–3 minutes (UserData takes time)

2) Stack went to ROLLBACK

Go to stack → **Events tab** → check the failure reason

Most common reason:

- Wrong Key Pair selected / key pair does not exist

Part I — Clean-up

To avoid charges:

1. CloudFormation → Stacks
2. Select EC2-Apache-AL2023
3. Click **Delete**
4. Confirm

This deletes EC2 + Security Group automatically.

DATE: 17-12-25

Exercise–23: EC2 + S3 (Static Content Pulled from S3 using CloudFormation)

Create an **S3 bucket** using CloudFormation.

Upload a static HTML file (index.html) to S3.

Launch **EC2 (Amazon Linux 2023) + Apache** using CloudFormation.

EC2 will **pull index.html from S3** and host it via Apache.

Pre-requisites

- Region: **Mumbai (ap-south-1)**
 - A Key Pair exists (example: pemkeypair)
-

PART 0 — One-time: Create Key Pair

EC2 → Key Pairs → Create key pair

- Name: pemkeypair
 - Format: .pem
- Download it and keep safe.
-

PART 1 — Stack-1: Create S3 Bucket (CloudFormation)

Step 1.1: Create S3 template file

Create a file: **stack1-s3-bucket.yaml** and paste:

AWSTemplateFormatVersion: "2010-09-09"

Description: Create an S3 bucket to store website content (index.html)

Resources:

WebsiteBucket:

Type: AWS::S3::Bucket

Outputs:

BucketName:

Description: S3 Bucket Name (use this to upload index.html)

Value: !Ref WebsiteBucket

Step 1.2: Create stack

CloudFormation → Stacks → Create stack → With new resources

- Choose an existing template
- Upload a template file → stack1-s3-bucket.yaml
- Stack name: S3-Website-Bucket
- Create stack

Step 1.3: Copy bucket name

Open stack → **Outputs** → copy **BucketName**

PART 2 — Upload Static Content to S3 (Manual)

Step 2.1: Create index.html on your PC

Create a file named **index.html** with this content:

```
<!DOCTYPE html>

<html>

<head>

  <title>EC2 + S3 Demo</title>

</head>

<body>

  <h1>Hello! This page was pulled from S3 to EC2 automatically.</h1>

  <p>Deployed using CloudFormation + UserData</p>

</body>

</html>
```

Step 2.2: Upload to S3 bucket

S3 → Buckets → open your bucket (from Output) → Upload

- Upload **index.html**
- Keep it at **root** (no folder)

- Upload

Now S3 has: `s3://<your-bucket-name>/index.html`

PART 3 — Stack-2: Launch EC2 + Apache + Pull from S3

This stack will:

- Create IAM Role (permission to read the bucket)
- Create Security Group (22, 80)
- Launch EC2 (Amazon Linux 2023)
- Install Apache
- Copy index.html from S3 to `/var/www/html/index.html`

Step 3.1: Create EC2 template file

Create a file: **stack2-ec2-pull-from-s3.yaml** and paste:

```
AWSTemplateFormatVersion: "2010-09-09"
Description: Launch EC2 (Amazon Linux 2023), install Apache, and pull index.html from S3

Parameters:
  KeyName:
    Type: AWS::EC2::KeyPair::KeyName
    Description: Select an existing EC2 Key Pair to enable SSH access

  BucketName:
    Type: String
    Description: Enter the S3 bucket name created in Stack-1 (Output)

  SubnetId:
    Type: AWS::EC2::Subnet::Id
    Description: Select a PUBLIC subnet (default VPC public subnet recommended)

  VpcId:
    Type: AWS::EC2::VPC::Id
    Description: Select the VPC (default VPC recommended)

Resources:
  WebServerRole:
    Type: AWS::IAM::Role
    Properties:
      AssumeRolePolicyDocument:
        Version: "2012-10-17"
        Statement:
          - Effect: Allow
```

Principal:

Service: ec2.amazonaws.com

Action: sts:AssumeRole

Policies:

- PolicyName: S3ReadOnlyForWebsiteBucket

PolicyDocument:

Version: "2012-10-17"

Statement:

- Effect: Allow

Action:

- s3:GetObject

- s3:ListBucket

Resource:

- !Sub "arn:aws:s3:::\${BucketName}"

- !Sub "arn:aws:s3:::\${BucketName}/*"

WebServerInstanceProfile:

Type: AWS::IAM::InstanceProfile

Properties:

Roles:

- !Ref WebServerRole

WebServerSG:

Type: AWS::EC2::SecurityGroup

Properties:

GroupDescription: Allow SSH (22) and HTTP (80)

VpcId: !Ref VpcId

SecurityGroupIngress:

- IpProtocol: tcp

FromPort: 22

ToPort: 22

CidrIp: 0.0.0.0/0

- IpProtocol: tcp

FromPort: 80

ToPort: 80

CidrIp: 0.0.0.0/0

WebServerInstance:

Type: AWS::EC2::Instance

Properties:

InstanceType: t2.micro

KeyName: !Ref KeyName

SubnetId: !Ref SubnetId

SecurityGroupIds:

- !Ref WebServerSG

IamInstanceProfile: !Ref WebServerInstanceProfile

```
# Amazon Linux 2023 AMI for Mumbai (ap-south-1)
ImageId: ami-02b49a24cfb95941c
```

UserData:

```
Fn::Base64: !Sub |
  #!/bin/bash
  dnf update -y
  dnf install -y httpd
  systemctl enable httpd
  systemctl start httpd

  # pull index.html from S3 to Apache web root
  aws s3 cp s3://${BucketName}/index.html /var/www/html/index.html

  systemctl restart httpd
```

Outputs:

WebsiteURL:

Description: Open this URL in browser

Value: !Sub "http://\${WebServerInstance.PublicDnsName}"

Step 3.2: Create stack

CloudFormation → Stacks → Create stack

- Upload template → stack2-ec2-pull-from-s3.yaml
- Stack name: EC2-Pull-S3-Website
- Parameters:
 - **KeyName:** select your key pair (e.g., pemkeypair)
 - **BucketName:** paste bucket name from Stack-1 Output
 - **VpcId:** select **default VPC**
 - **SubnetId:** select a **PUBLIC subnet** in default VPC
(usually the subnet name shows “public” or you can pick any default subnet that gives public IP; default subnets generally work)

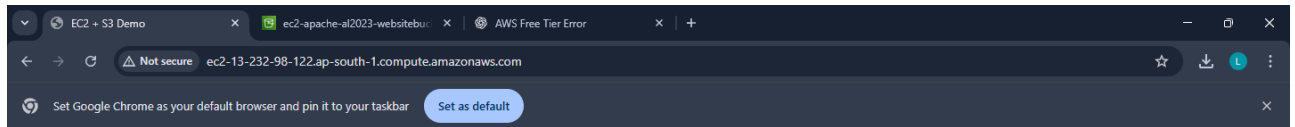
Create stack → wait for **CREATE_COMPLETE**

PART 4 — Verify Output

Stack-2 → **Outputs** → copy **WebsiteURL** → open in browser.

✅ Expected page:

“Hello! This page was pulled from S3 to EC2 automatically...”



Hello! This page was pulled from S3 to EC2 automatically.

Deployed using CloudFormation + UserData

Troubleshooting (common)

1) Website not opening

- Wait 1–2 minutes (UserData takes time)
- Ensure Security Group has **HTTP 80 open**
- Ensure you selected a **public subnet** (needs internet to reach S3)

2) Stack-2 fails with S3 access error

- BucketName typed wrong
- index.html not uploaded at root of bucket

Clean-up

Delete Stack-2 first

CloudFormation → select EC2-Pull-S3-Website → Delete

Empty the S3 bucket

S3 → bucket → delete index.html (empty bucket)

Delete Stack-1

CloudFormation → select S3-Website-Bucket → Delete

Viva-ready questions

1. What is the purpose of **UserData** in EC2?
2. Why do we need an **IAM Role + Instance Profile**?
3. Why is S3 bucket not made public in this lab?
4. What happens when a CloudFormation stack is deleted?
5. Which ports are required for web hosting and SSH?

DATE: 19-12-25

Exercise–24: AWS Lambda: Input Processing, Business Logic Execution, and CloudWatch Logging

Create one Lambda function that:

1. prints a welcome message
2. reads JSON input event
3. performs business logic (grade calculation)
4. writes logs → view them in CloudWatch Logs

Yes — this is the **first combined Lambda lab exercise** (basics + input + logic + logging). Below is a **complete step-by-step procedure** using **AWS Console**.

Create the Lambda Function

Step 1: Open Lambda

AWS Console → Search Lambda → Open AWS Lambda

Step 2: Create function

Click Create function

- Select - Author from scratch
- Function name: StudentGradeLogger
- Runtime: Python 3.11
- Architecture: x86_64 (default)

Step 3: Permissions (Execution Role)

Under “Permissions”

- Choose: Create a new role with basic Lambda permissions

This automatically gives permission to write logs to CloudWatch.

Click **Create function**

Add Code (Business Logic + Logs)

Step 4: Paste this code

In **Code** tab → lambda_function.py → paste and **Deploy**

```
import json

def lambda_handler(event, context):
    # Welcome message
    print("Lambda invoked successfully")

    # Read input from event
    student_name = event["StudentName"]
    marks = event["Marks"]

    # Business logic: grade calculation
    if marks >= 75:
        result = "Pass"
        grade = "A"
```

```
else:
    result = "Fail"
    grade = "F"

# Log output
print("Student:", student_name)
print("Marks:", marks)
print("Grade:", grade)
print("Result:", result)

# Return response
return {
    "statusCode": 200,
    "body": json.dumps({
        "StudentName": student_name,
        "Marks": marks,
        "Grade": grade,
        "Result": result
    })
}
```

Click **Deploy**.

Test with Input Events (JSON)

Step 5: Create a test event (Valid case)

Go to Test (top right) → Configure test event

- Event name: ValidInput
- Paste this JSON:

```
{
  "StudentName": "Anita",
  "Marks": 86
}
```

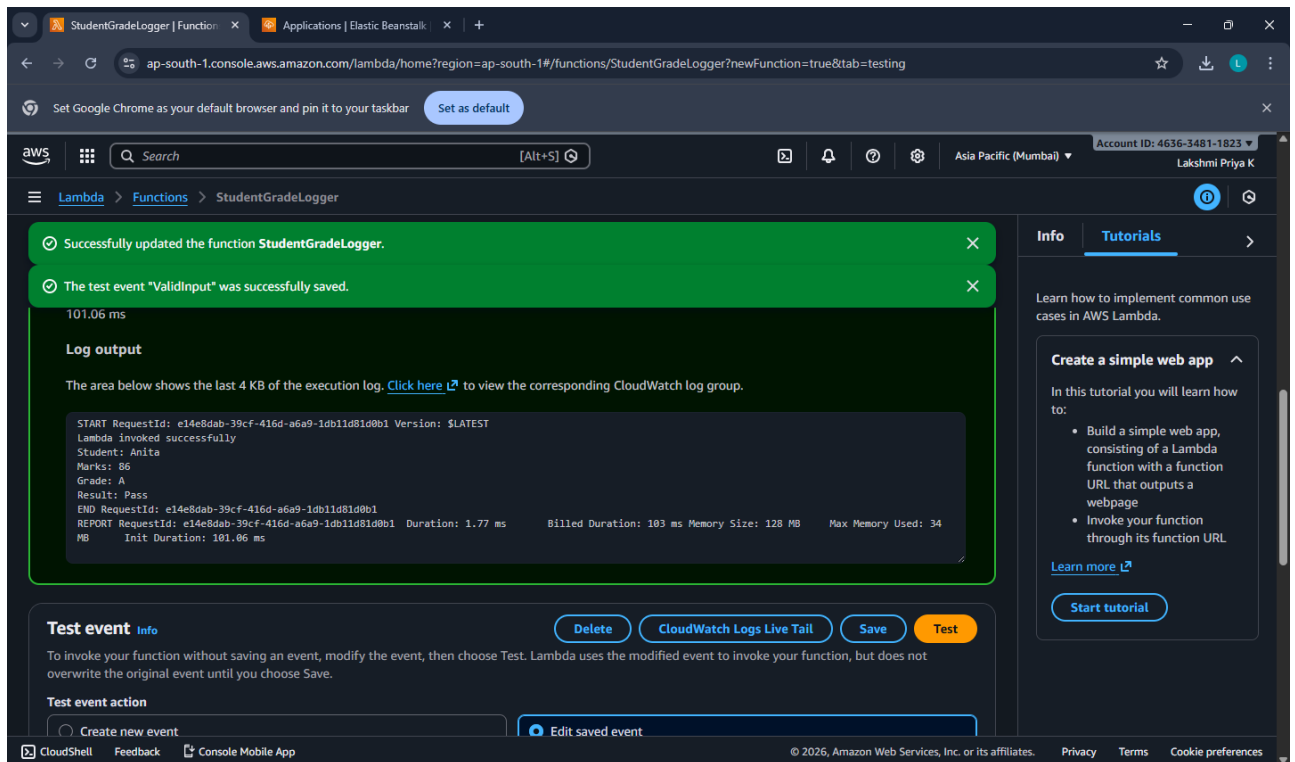
Click **Save**

Step 6: Run test

Click **Test**

Expected response (sample):

- statusCode: 200
- body will include Grade A and Result Pass



Step 7: Test invalid input (Missing Marks)

Create another test event:

- Event name: MissingMarks

```
{
  "StudentName": "Rahul"
}
```

Run **Test**

Expected:

- statusCode: 400
- message: Missing 'Marks'

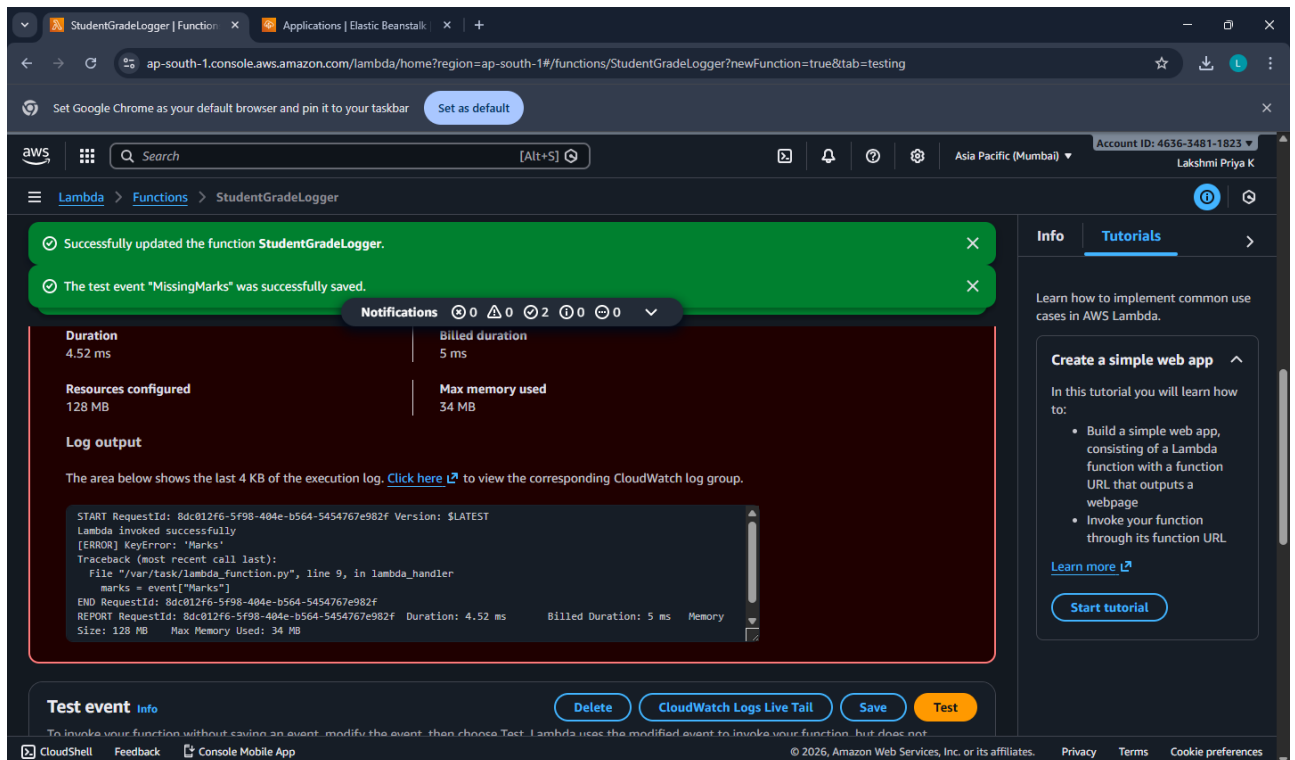
Step 8: Test invalid marks range

Event name: InvalidMarks

```
{
  "StudentName": "Priya",
  "Marks": 150
}
```

Expected:

- statusCode: 400
- message: Marks must be 0–100



View Logs in CloudWatch

Step 9: Open logs from Lambda directly

On the Lambda function page:

- Go to **Monitor** tab
- Click **View CloudWatch logs**

You will see:

- Log group: /aws/lambda/StudentGradeLogger
- Open latest **log stream**
- You should see all print() outputs:
 - “Lambda invoked successfully!”
 - “Received event...”
 - grade/result logs
 - error logs for invalid tests

Cleanup (Avoid charges, keep account clean)

Lambda itself usually costs nothing in small use, but cleanup is good practice:

1. Lambda → Functions → select StudentGradeLogger → **Delete**
2. CloudWatch → Log groups → find /aws/lambda/StudentGradeLogger → **Delete**
3. IAM role created (optional):
 - IAM → Roles → search role name created for Lambda → delete (only if not used elsewhere)

DATE: 24-12-25

Exercise–25: Mini Project: Event-Driven Notification System using AWS Lambda and Amazon SNS. Simulating real-time alerts from events using serverless computing.

- An event in JSON format is given as input to an AWS Lambda function.
- The Lambda function processes the event and generates a notification message.
- The message is published to an Amazon SNS topic.
- SNS sends the notification to the subscribed email address.
- The execution details are verified using CloudWatch Logs.

Create SNS Topic + Email Subscription

Step 1: Open SNS

AWS Console → Search SNS → Open Simple Notification Service

Step 2: Create a Topic

SNS → Topics → Create topic

- Type: Standard
 - Name: NotificationTopic
- Click Create topic

Step 3: Copy Topic ARN

Open the created topic → copy Topic ARN

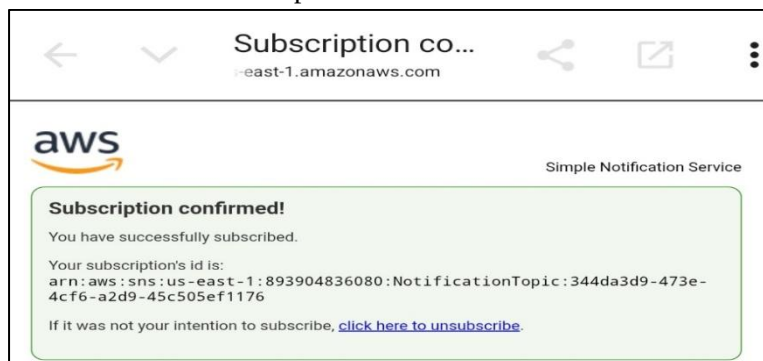
(You will paste it into Lambda code later)

Step 4: Create an Email Subscription

Inside the same topic:

Click Create subscription

- Protocol: Email
 - Endpoint: (your email id)
- Click Create subscription



Step 5: Confirm Subscription

Open your email inbox → find AWS SNS confirmation mail → click Confirm subscription
→ Status in SNS should become Confirmed.

Create Lambda Function

Step 6: Open Lambda

AWS Console → Search Lambda → Open AWS Lambda

Step 7: Create function

Click Create function

- Select: Author from scratch
 - Function name: NotificationSimulator
 - Runtime: Python 3.11
 - Permissions: Create a new role with basic Lambda permissions
- Click Create function

Add Lambda Code (with SNS Publish)

Step 8: Paste code in lambda_function.py

Lambda → **Code** tab → open lambda_function.py → remove existing code → paste:

Replace <SNS_TOPIC_ARN> with your copied Topic ARN.

```
import json
import boto3

sns = boto3.client('sns')

TOPIC_ARN = "<SNS_TOPIC_ARN>"

def lambda_handler(event, context):
    print("Notification Simulator invoked")

    event_type = event["eventType"]
    user = event["user"]

    if event_type == "ORDER_PLACED":
        message = f"Hi {user}, your order is placed successfully."
        priority = "NORMAL"

    elif event_type == "PAYMENT_FAILED":
        message = f"Hi {user}, your payment has failed. Please retry."
        priority = "HIGH"

    elif event_type == "LOW_ATTENDANCE":
        message = f"Hi {user}, your attendance is low. Please take action."
        priority = "HIGH"

    else:
        message = f"Hi {user}, unknown event received."
        priority = "LOW"

    print("Event Type:", event_type)
    print("Message:", message)
    print("Priority:", priority)

    # Publish to SNS (Actual notification delivery)
    sns.publish(TopicArn=TOPIC_ARN, Message=message, Subject=f"{event_type} [{priority}]")
```

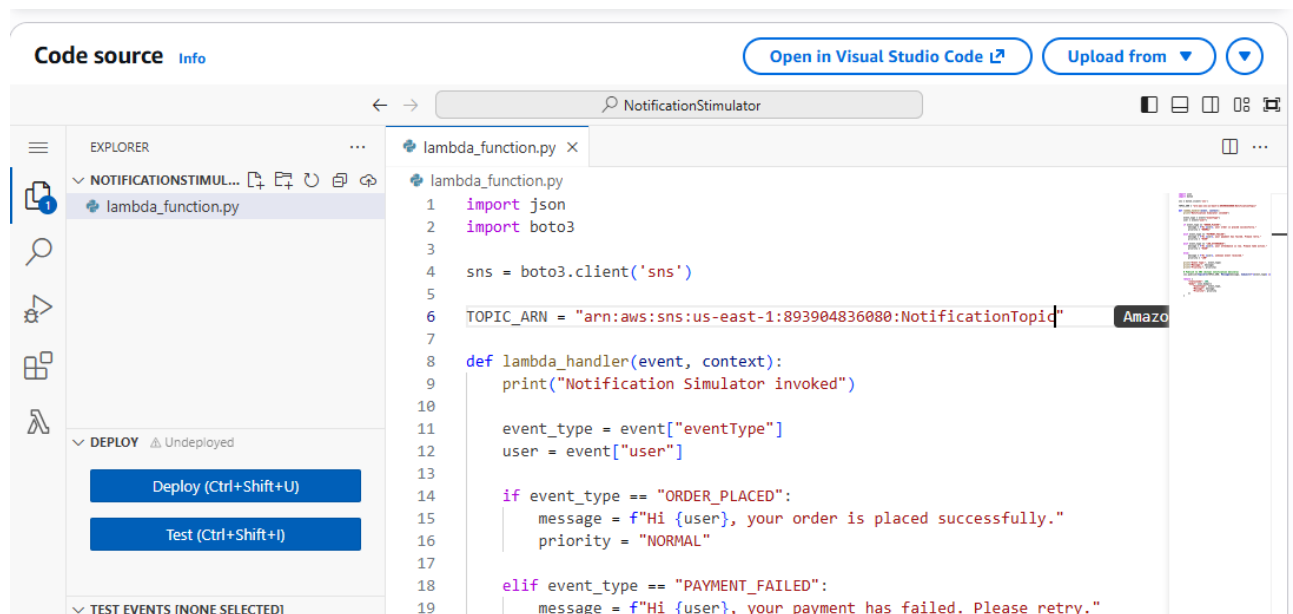
```

return {
    "statusCode": 200,
    "body": json.dumps({
        "EventType": event_type,
        "Message": message,
        "Priority": priority
    })
}

```

Click **Deploy**

Wait for “Successfully updated function...”



Give Lambda Permission to Publish to SNS

Step 10: Open Lambda Execution Role

Lambda → Configuration → Permissions

Click the Role name (execution role link)

Step 11: Attach SNS Publish Policy

IAM Role page → Add permissions → Attach policies

Attach: AmazonSNSFullAccess

Click Add permissions

Now Lambda can publish to SNS.

Test the Mini Project

Step 12: Create a test event

Lambda → Test tab → Create new test event

Event name: PaymentFailed

Paste:

```

{
    "eventType": "PAYMENT_FAILED",
    "user": "Anita"
}

```

}

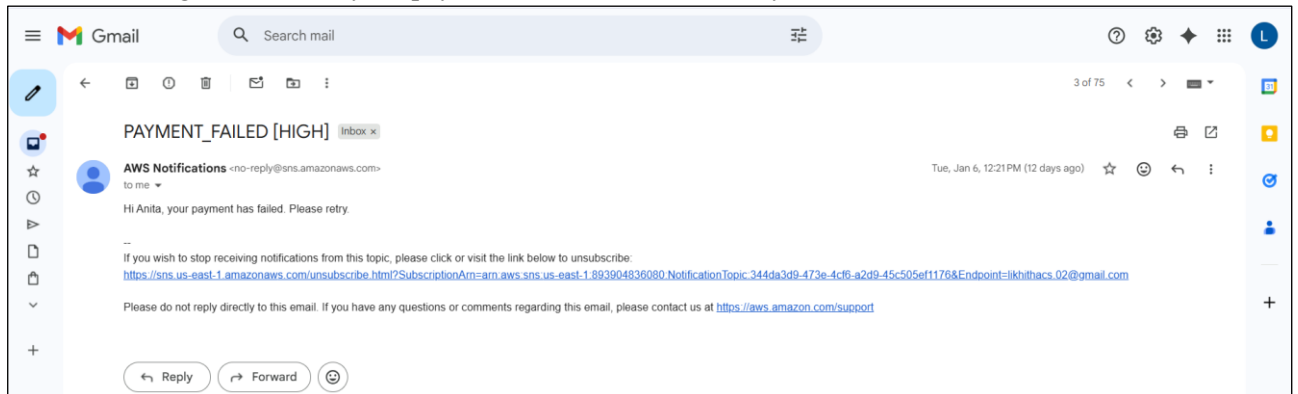
Click **Save**

Step 13: Run Test

Click **Test**

Expected:

- Execution: **Succeeded**
- Email should arrive within a few seconds:
Subject like: PAYMENT_FAILED [HIGH]
Message: “Hi Anita, your payment has failed. Please retry.”



Verify CloudWatch Logs

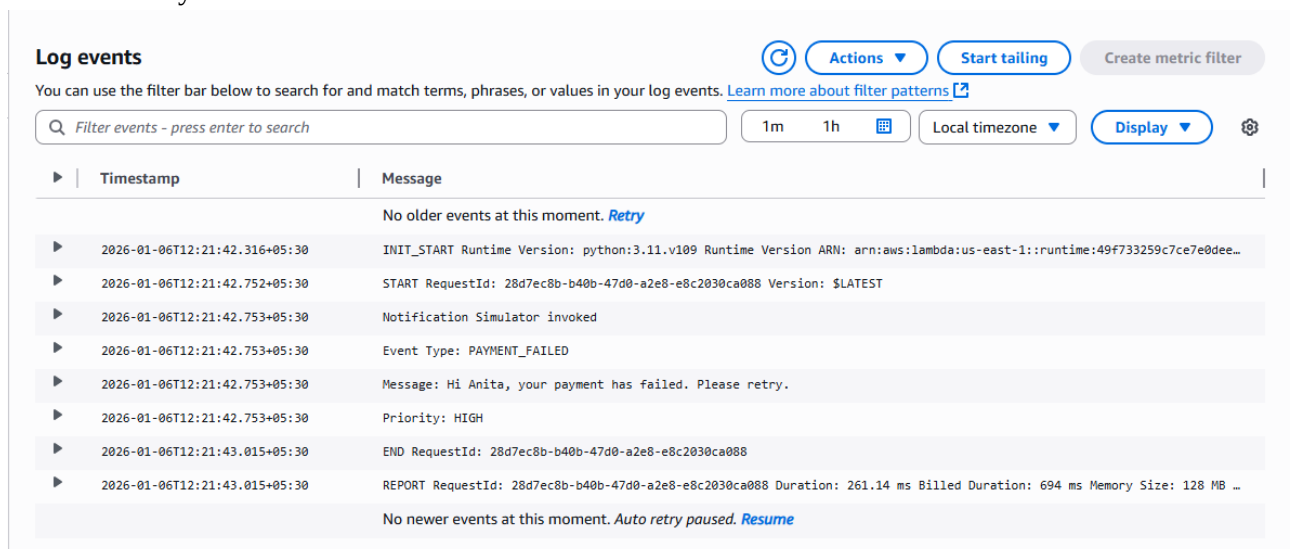
Step 14: View logs

Lambda → Monitor → View CloudWatch logs

Open latest log stream.

Verify these prints exist:

- Notification Simulator invoked
- Event Type:
- Message:
- Priority:



Expected Outputs

1. Lambda test status: Succeeded
2. Email received from SNS with correct message
3. CloudWatch logs showing event type and generated message

Cleanup

1. Delete Lambda function: NotificationSimulator
2. SNS → delete topic NotificationTopic (subscriptions removed automatically)
3. CloudWatch → delete log group for Lambda
4. (Optional) Delete IAM role if not used elsewhere

DATE: 26-12-25

Exercise–26: Analyze a CSV File in S3 Using Amazon Athena (No Server)

Upload a small CSV to S3, then use Athena to run SQL queries like count, average, max, group by.

Part A — Create Sample CSV (on your PC)

1. Open **Notepad**
2. Paste this data and save as: **students.csv**

```
StudentID,Name,Dept,Marks,Result
```

```
101,Anita,MCA,85,Pass
```

```
102,Ravi,MCA,72,Pass
```

```
103,Meera,MBA,64,Pass
```

```
104,John,MCA,35,Fail
```

```
105,Sneha,MBA,91,Pass
```

```
106,Arun,MCA,49,Fail
```

```
107,Kiran,MBA,58,Pass
```

```
108,Divya,MCA,77,Pass
```

Part B — Create S3 Bucket and Upload CSV

1. Go to **AWS Console** → S3
2. Click **Create bucket**
3. Bucket name: athena-lab-<yourname>-<number> (must be globally unique)
4. Keep defaults → Click **Create bucket**
5. Open the bucket → Click **Upload**
6. Upload **students.csv**
7. Click **Upload**

Now your CSV is in S3.

Upload: status

Close

After you navigate away from this page, the following information is no longer available.

Summary

Destination

s3://athena-lab-lcs-050

Succeeded

1 file, 215.0 B (100.00%)

Failed

0 files, 0 B (0%)

Files and folders

Configuration

Files and folders (1 total, 215.0 B)

Find by name

< 1 >

Name	Folder	Type	Size	Status	Error
students.csv	-	text/csv	215.0 B	Succeeded	-

Part C — Open Athena and Set Query Result Location (IMPORTANT)

- Go to **AWS Console** → **Amazon Athena**
- If it shows “Get started” / “Query editor”, open it.
- It will ask to set a **Query result location**
- Click the link/button like **Settings / Manage / Edit**
- Set query result location to something like:
 - s3://your-bucket-name/athena-results/
- Click **Save**

This is required so Athena can store query outputs.

Amazon Athena > Query editor

Query in Amazon SageMaker Unified Studio

Use your IAM role to analyze and build with your existing AWS resources in a single data and AI development environment

Set up. LA

Editor

Recent queries

Saved queries

Query settings

Workgroup primary

Before you run your first query, you need to set up a query result location in Amazon S3.

Edit settings

Athena now supports typeahead code suggestions to speed up SQL query development

Typeahead suggestions are turned on by default. You can change this setting in query editor preferences.

Edit preferences

Data

Data source

AwsDataCatalog

Catalog

None

Database

Choose a database

Tables and views

Create

Query 1

1

CloudShell

Feedback

Console Mobile App

© 2026, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

17°C Mostly sunny

Search

ENG IN 09:40 09-01-2026

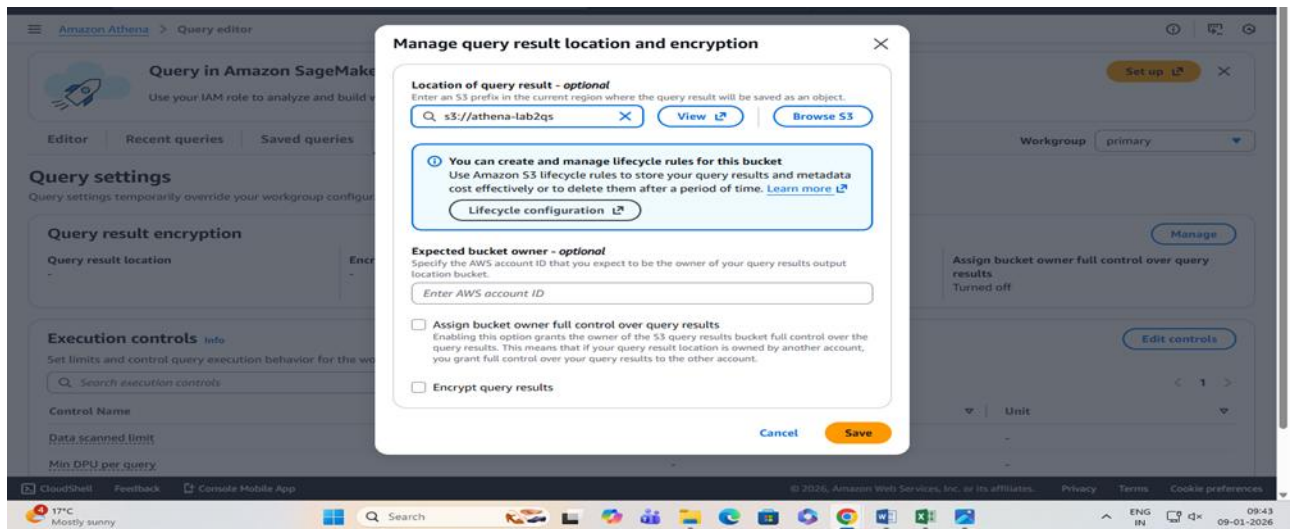
Part D — Create a Database in Athena

In Athena query editor, run:

```
CREATE DATABASE labdb;
```

Then on the left side, select:

- **Data source:** AwsDataCatalog
- **Database:** labdb



Part E — Create a Table for the CSV in S3

Run this (replace YOUR_BUCKET_NAME with your actual bucket name):

```
CREATE EXTERNAL TABLE IF NOT EXISTS students (
  StudentID int,
  Name string,
  Dept string,
  Marks int,
  Result string
)
ROW FORMAT SERDE 'org.apache.hadoop.hive.serde2.OpenCSVSerde'
WITH SERDEPROPERTIES (
  'separatorChar' = ',',
  'quoteChar' = '"',
  'escapeChar' = '\\'
)
STORED AS TEXTFILE
LOCATION 's3://YOUR_BUCKET_NAME/'
TBLPROPERTIES ('skip.header.line.count'='1');
```

This tells Athena: “My CSV is in S3, treat it like a table.”

Part F — Run Simple Analysis Queries (Core Part)

1) View all rows

```
SELECT * FROM students;
```

Expected: 8 records.

The screenshot shows the Amazon Athena Query Editor interface. On the left, there's a sidebar with 'Tables and views' and a search bar. The main area displays the SQL query editor with the query `SELECT * FROM students;` entered. Below the editor, the 'Query results' tab is active, showing a table with 8 rows of data. The table has columns: #, studentid, name, dept, marks, and result. The results are as follows:

#	studentid	name	dept	marks	result
1	101	Anita	MCA	85	Pass
2	102	Ravi	MCA	72	Pass
3	103	Meera	MBA	64	Pass
4	104	John	MCA	35	Fail
5	105	Sneha	MBA	91	Pass
6	106	Arun	MCA	49	Fail
7	107	Kiran	MBA	58	Pass
8	108	Dhrya	MCA	77	Pass

The interface also shows 'Query stats' with metrics like 'Time in queue: 54 ms', 'Run time: 375 ms', and 'Data scanned: 0.21 KB'.

2) Total students

```
SELECT COUNT(*) AS total_students FROM students;
```

Expected: 8

3) Pass vs Fail count

```
SELECT Result, COUNT(*) AS cnt
```

```
FROM students
```

```
GROUP BY Result;
```

Expected (based on data): Pass = 6, Fail = 2

4) Average marks overall

```
SELECT AVG(Marks) AS avg_marks
```

```
FROM students;
```

5) Department-wise average marks

```
SELECT Dept, AVG(Marks) AS avg_marks
```

```
FROM students
```

```
GROUP BY Dept;
```

6) Top scorer

```
SELECT Name, Dept, Marks
```

```
FROM students
```

```
ORDER BY Marks DESC
```

LIMIT 1;

Expected: Sneha (91)

7) List failed students

SELECT StudentID, Name, Dept, Marks

FROM students

WHERE Result = 'Fail';

Expected: John, Arun

The screenshot displays the Amazon Athena Query Editor. On the left sidebar, the 'Database' is set to 'labddb'. Under 'Tables and views', the 'students' table is listed. The main area shows the SQL query: 'SELECT StudentID, Name, Dept, Marks FROM students WHERE Result = 'Fail';'. The query has been executed successfully, as indicated by the 'Completed' status. The 'Query results' tab shows two rows of data:

#	StudentID	Name	Dept	Marks
1	104	John	MCA	35
2	106	Arun	MCA	49

The interface also shows query statistics: 'Time in queue: 54 ms', 'Run time: 438 ms', and 'Data scanned: 1.11 KB'. At the bottom, there is a Windows taskbar with various application icons and a system clock showing 10:39 on 09-01-2026.

Cleanup (to avoid any charges)

1. In Athena, you can keep DB/table (no cost by itself), but clean S3:
2. Go to **S3 bucket**
3. Delete:
 - students.csv
 - athena-results/ folder contents (query outputs)
4. Delete the bucket (must be empty to delete)

Cost Note (Simple)

- **S3 storage:** tiny (almost negligible for this)
- **Athena:** charges based on **data scanned**; with this tiny CSV it's usually minimal, but **always delete outputs and bucket** after lab.